

The University of Faisalabad

Data mining

Lab manual

Registration # 2023-BSAI-022 (6A)

Submitted to: Mr. Muhammad Arham

Submitted by: Abeer Awais

Department: Artificial intelligence

Contents

LAB-1: Introduction to Data Mining Software & Datasets	2
LAB-2: CRISP-DM Process & Data Types	8
LAB-3: Data Cleaning Techniques	16
LAB-4: Data Preprocessing.....	19
LAB-5: Market Basket Analysis	25
LAB-6: Apriori Algorithm	28
LAB-7: FP-Tree & FP-Growth	36
LAB-8: Python Basics for Data Mining	47
LAB 9: Decision Tree Classification	50
LAB-10: Naïve Bayes & KNN for Email Spam Detection	56
LAB-11: Support Vector Machine (SVM)	64
LAB-12: CLUSTERING	71
LAB-13: Outlier Detection	80
LAB-14: Web Scraping & Sentiment Analysis	90
LAB-15: Final Capstone	110

LAB-1: Introduction to Data Mining Software & Datasets

Data Mining Software

Data mining software consists of specialized tools and applications used to examine large amounts of data and discover useful patterns, trends, relationships, and hidden information. These tools help convert raw data into meaningful insights that can support better decision-making and problem-solving.

Some commonly used data mining tools are **Python, RapidMiner, WEKA, and Orange**. Among these, **Python** is the most popular because it offers powerful libraries such as **Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn** for data analysis, visualization, and machine learning. Tools like **RapidMiner, Orange, and Knime** provide user-friendly graphical interfaces, making them suitable for beginners and users with limited programming experience.

Benefits of Data Mining Software:

- Handles large datasets efficiently
- Identifies hidden patterns and trends
- Supports predictive analysis and decision-making
- Simplifies data preprocessing and visualization
- Assists in building machine learning models

Real-World Example:

Online shopping websites such as Amazon use data mining software to analyze customer purchases and browsing history. Based on this analysis, they recommend products that customers are likely to buy, improving the shopping experience and increasing sales.

Overview of Datasets

A **dataset** is an organized collection of related data stored in a structured format. It is usually represented as a table where **rows** correspond to records or instances, and **columns** represent attributes or features.

Datasets are generally classified into the following types:

- **Structured Data:** Well-organized data stored in tables or databases (e.g., student records, sales data).
- **Unstructured Data:** Data without a predefined format, such as images, videos, audio files, and text documents.

- **Semi-Structured Data:** Data that contains some organizational properties but does not follow a strict table format (e.g., JSON and XML files).

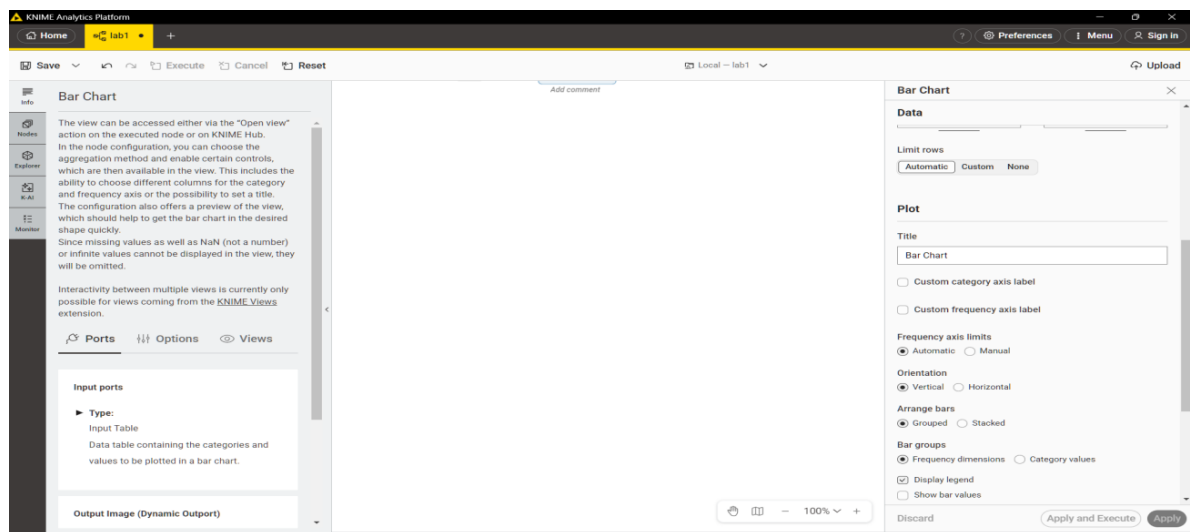
Importance of Datasets

Understanding the structure and characteristics of a dataset is an important step before performing data mining tasks. High-quality and properly organized data improves the accuracy, reliability, and performance of analytical models. Data preparation activities such as cleaning, transformation, and feature selection help ensure better mining results.

Key Points:

- Data quality directly affects model performance.
- Well-structured datasets lead to more accurate analysis.
- Proper preprocessing helps reduce errors and inconsistencies.
- Understanding data types assists in selecting suitable mining techniques.

TAKING KNIME AS AN EXAMPLE:



COLLECTING DATASET:

The screenshot shows the Kaggle website interface for the 'Retail Sales' dataset. The browser address bar displays the URL: `kaggle.com/datasets/reignrichard/retail-sales?utm_source=chatgpt.com`. The dataset is titled 'Retail Sales' with the subtitle 'Uncovering Retail Insights: Revenue Trends by Product, Gender & Time'. It was updated 9 months ago by Richard Chinyamgwe. The page includes a 'Data Card' tab, a 'Code' button, and a 'Download' button. A sidebar on the left contains navigation links like 'Home', 'Competitions', 'Benchmarks', 'Game Arena', 'Data Hub', and 'Your Work'. The 'About Dataset' section describes the data as 2,514 transactions from Kaggle, capturing dimensions like product category, customer gender, day of the week, and revenue. It also mentions a usability score of 8.82 and a license of CC0: Public Domain. The bottom of the image shows a Windows taskbar with various application icons and the system clock at 4:27 PM on 4/27/2026.

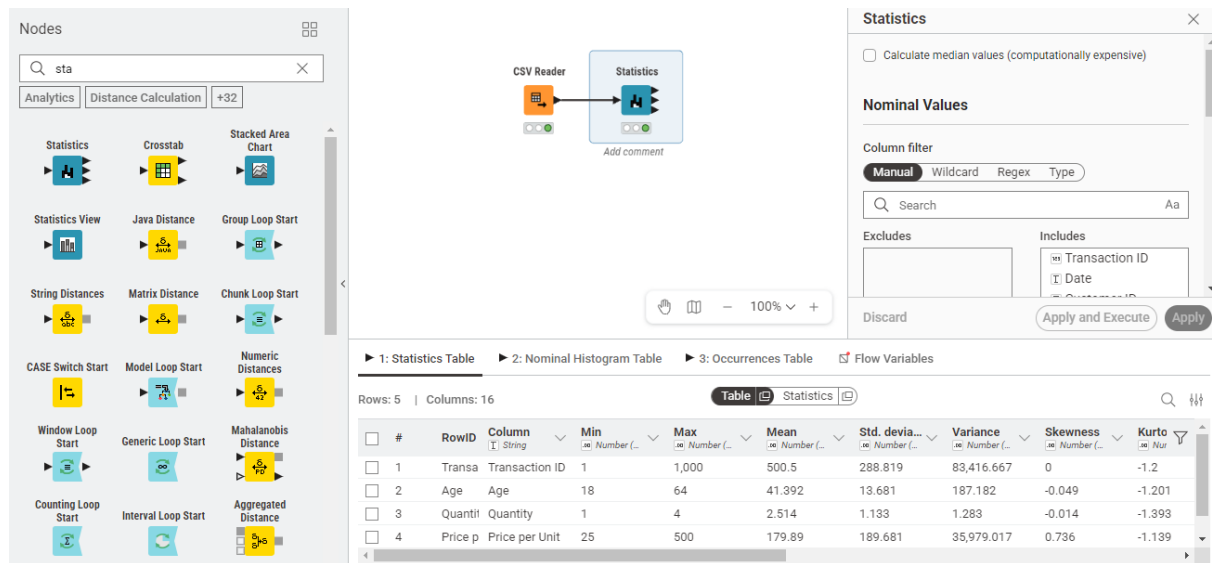
CREATING WORKFLOW:

The screenshot shows a 'Create a new workflow' dialog box. It has a title bar with a close button (X). The 'Workflow name' field contains the text 'retail lab-01'. At the bottom right, there are two buttons: 'Cancel' and 'Create'.

LOAD DATASET:

The screenshot shows a 'File' dialog box with tabs for 'Settings', 'Transformation', 'Advanced Settings', 'Limit Rows', 'Encoding', 'Flow Variables', 'Job Manager Selection', and 'Memory Policy'. The 'Input location' section has a 'Read from' dropdown set to 'Local File System'. Below this, the 'Mode' is set to 'File' (radio button selected). The 'File' field contains the path `C:\Users\hp\OneDrive\Desktop\retail_sales_dataset.csv`. A 'Browse...' button is located to the right of the file path.

CONNECT STATISTIC NODES:



Statistics

☐ Calculate median values (computationally expensive)

Nominal Values

Column filter

Manual Wildcard Regex Type

Search Aa

Excludes Includes

Transaction ID Date

Discard Apply and Execute Apply

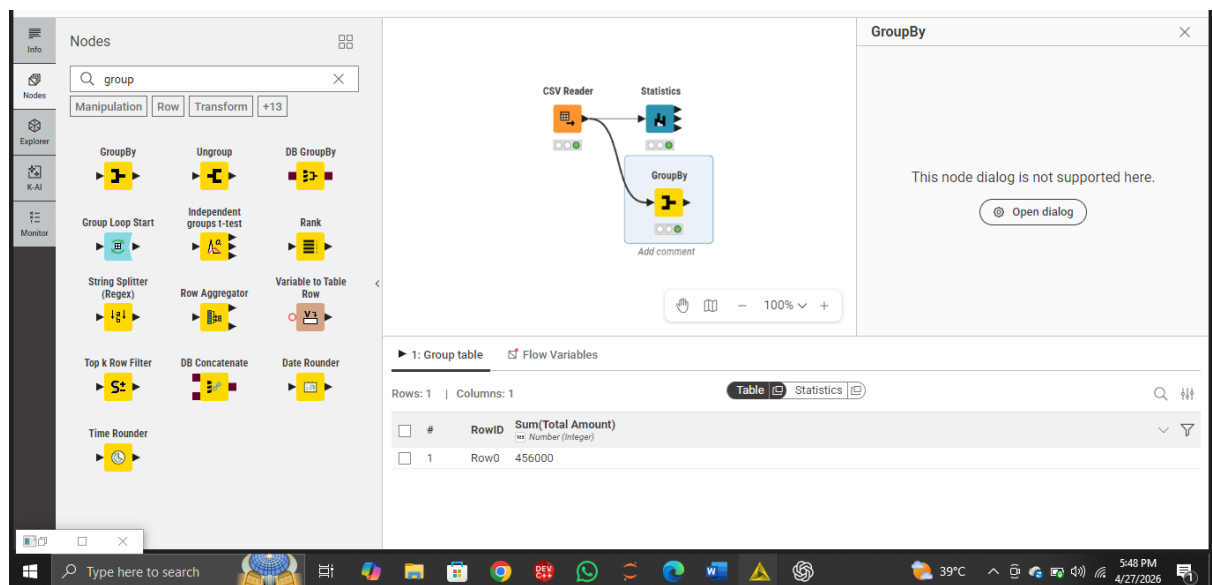
► 1: Statistics Table ► 2: Nominal Histogram Table ► 3: Occurrences Table Flow Variables

Rows: 5 | Columns: 16

#	RowID	Column	Min	Max	Mean	Std. devia...	Variance	Skewness	Kurto
1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2
2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201
3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393
4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.189

USE GROUP BY:

KPI 1:



GroupBy

This node dialog is not supported here.

Open dialog

► 1: Group table Flow Variables

Rows: 1 | Columns: 1

#	RowID	Sum(Total Amount)
1	Row0	456000

KPI 2:

Nodes

Search: group

Manipulation Row Transform +13

GroupBy Ungroup DB GroupBy

Group Loop Start Independent groups t-test Rank

String Splitter (Regex) Row Aggregator Variable to Table

Top k Row Filter DB Concatenate Date Rounder

Time Rounder

CSV Reader Statistics

GroupBy

Add comment

Open dialog

1: Group table

Flow Variables

Rows: 1 Columns: 1

#	RowID	Mean(Total Amount)
1	Row0	456

KPI 3:

Nodes

Search: Search all nodes

IO

Excel Reader Excel Writer Microsoft Authenticator

Google Authenticator Google Sheets Reader Google Sheets Writer

CSV Reader CSV Writer Show all

Manipulation

Row Filter Column Filter Concatenate

Value Lookup Row Aggregator Joiner

GroupBy

CSV Reader Statistics

GroupBy

Add comment

Open dialog

1: Group table

Flow Variables

Rows: 3 Columns: 2

#	RowID	Product Category	Sum(Total Amount)
1	Row0	Beauty	143515
2	Row1	Clothing	155580
3	Row2	Electronics	156905

SORTER NODE:

The screenshot shows the Alteryx interface with a workflow and a detailed view of the Sorter node configuration.

Workflow: A CSV Reader node feeds into a GroupBy node, which then feeds into a Sorter node. The Sorter node is configured to sort by the Sum(Total Amount) column in descending order. The output of the Sorter node is connected to a Statistics node.

Sorter Node Configuration:

- Criterion 1: Sum(Total Amount)
- Order: Descending

Sorted Table:

#	RowID	Product Category	Sum(Total Amount)
1	Row2	Electronics	156905
2	Row1	Clothing	155580
3	Row0	Beauty	143515

LAB-2: CRISP-DM Process & Data Types

CRISP-DM Process

CRISP-DM (Cross-Industry Standard Process for Data Mining) is a systematic framework used for executing data mining projects efficiently. It consists of six phases:

1. **Business Understanding:** Define objectives and problem statements
2. **Data Understanding:** Collect and explore data to identify patterns
3. **Data Preparation:** Clean, transform, and organize data
4. **Modeling:** Apply algorithms to build predictive or analytical models
5. **Evaluation:** Assess model performance and accuracy
6. **Deployment:** Implement the model in real-world applications

Types of Data

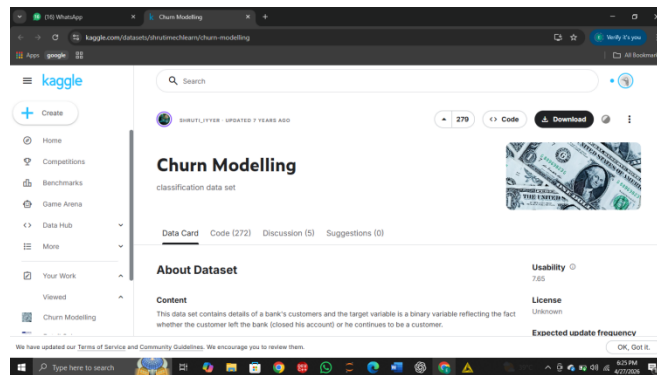
- **Numerical Data:** Quantitative values
 - Discrete (countable)
 - Continuous (measurable)
- **Categorical Data:** Qualitative information
 - Nominal (no order)
 - Ordinal (ordered categories)
- **Binary Data:** Two possible values (e.g., Yes/No)

Significance:

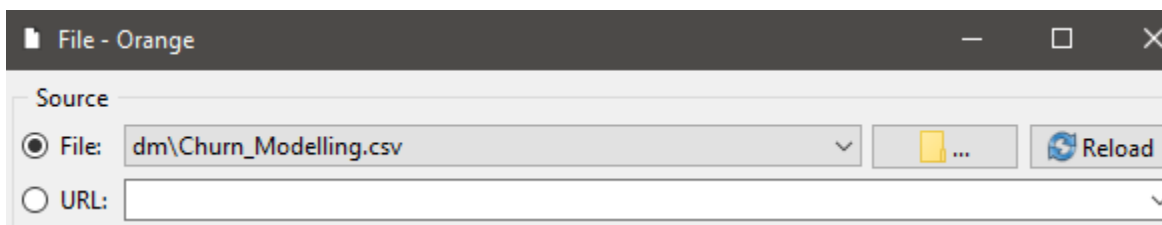
Correct identification of data types helps in selecting appropriate algorithms and improves overall model efficiency.

TAKING ORANGE AS AN EXAMPLE:

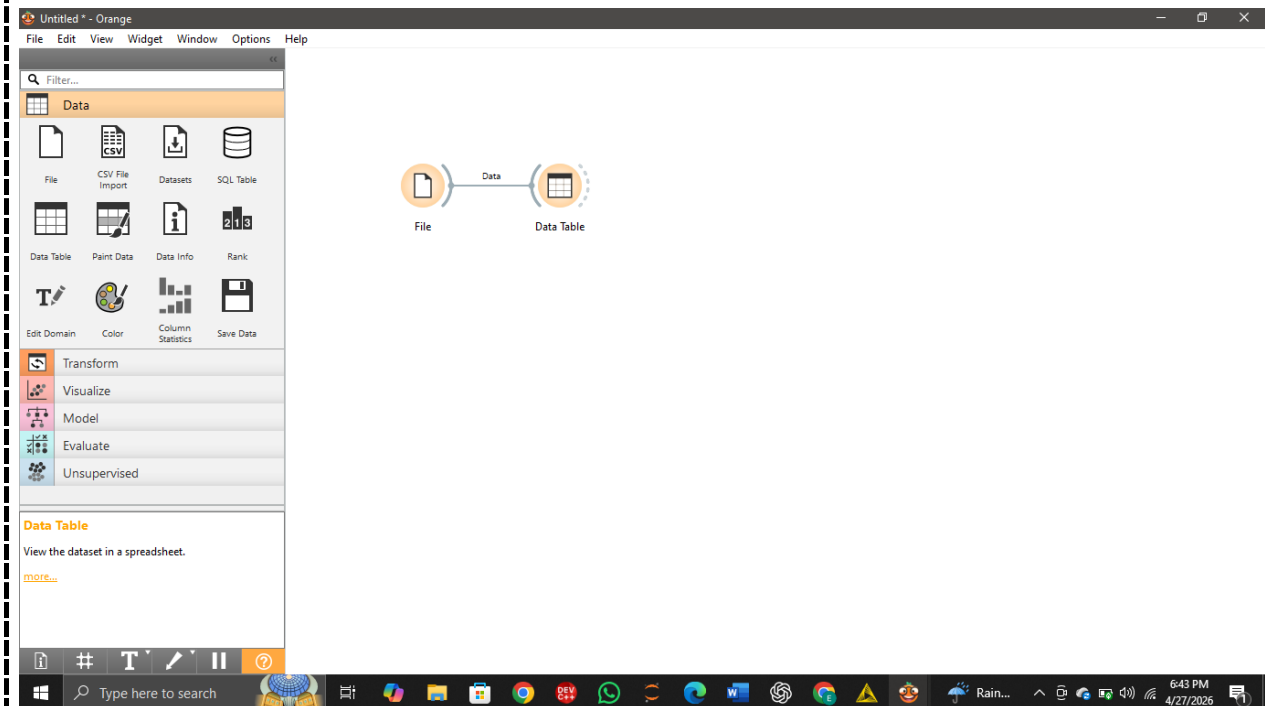
COLLECTING DATASET:

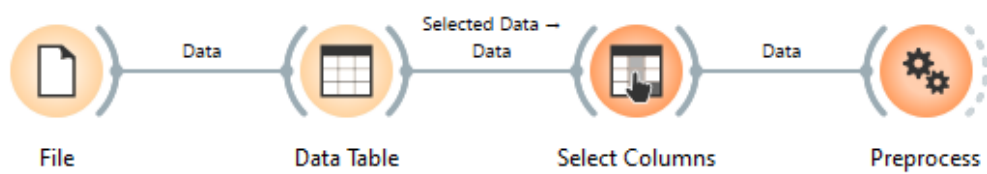
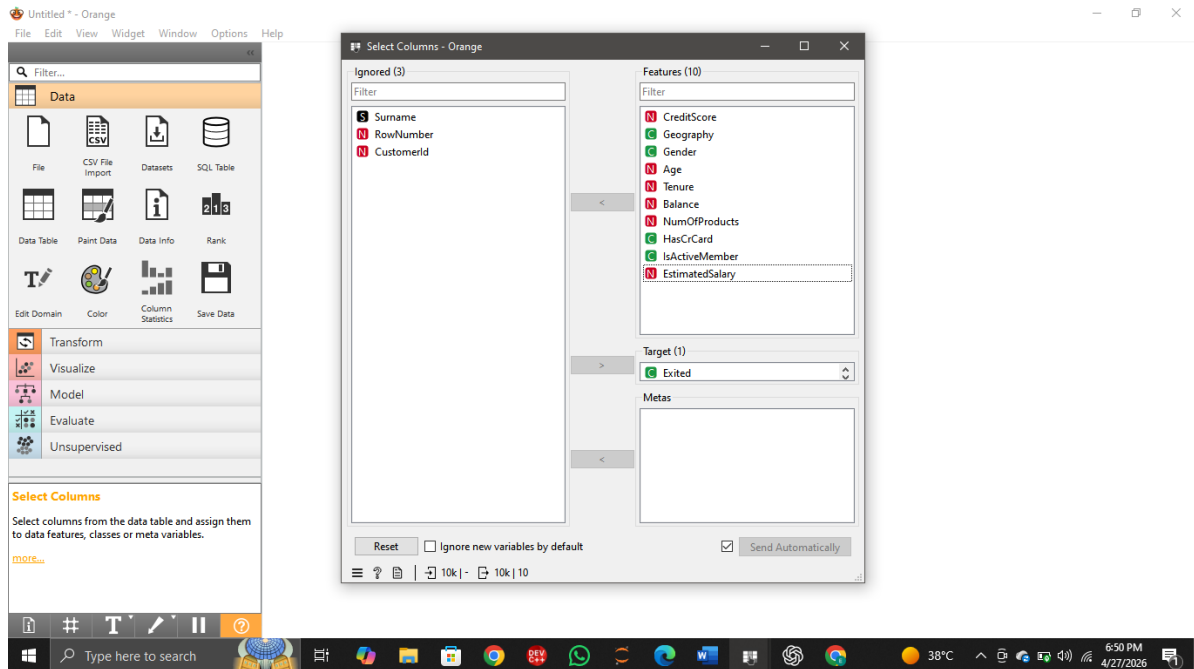
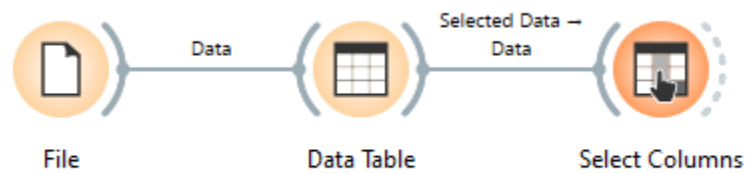


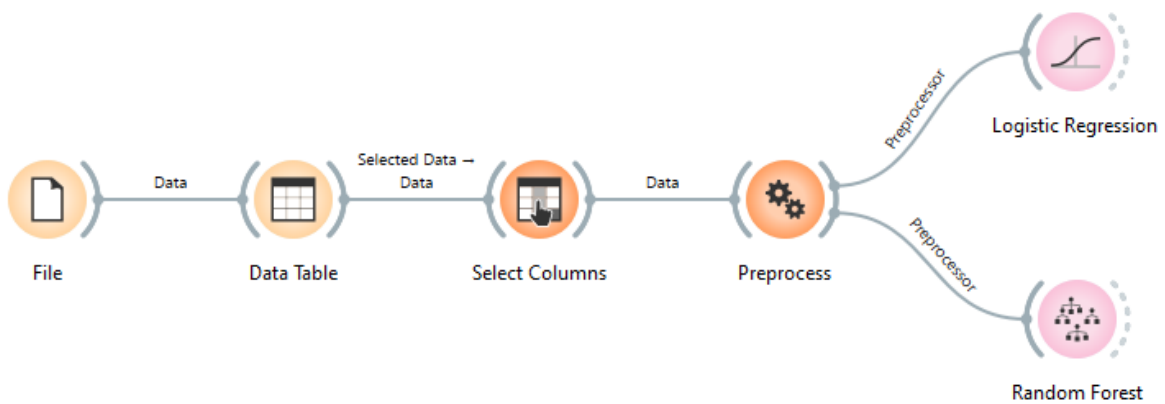
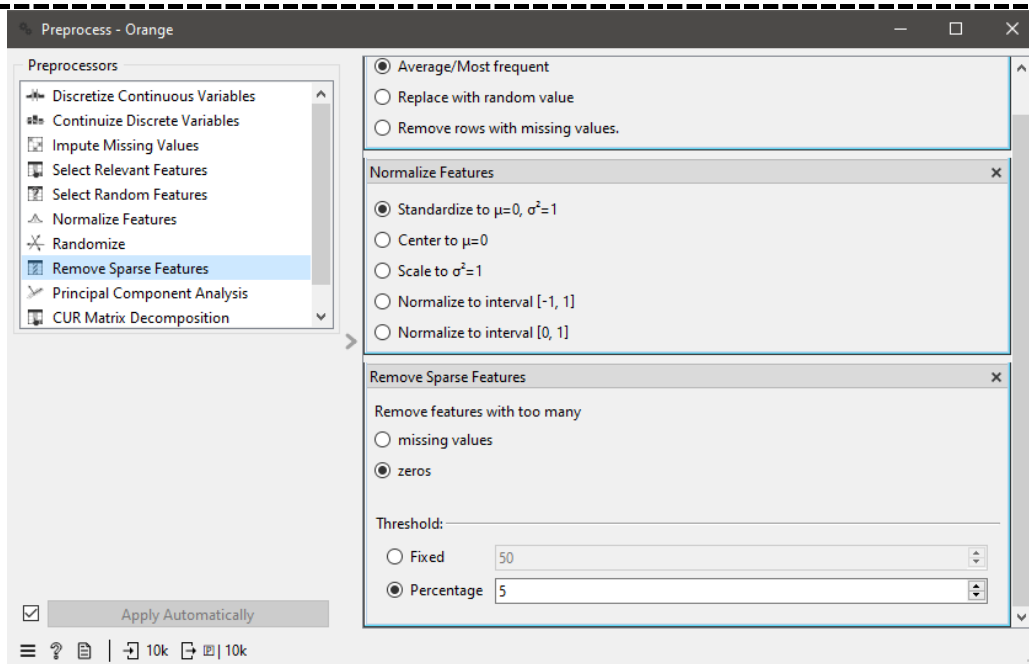
LOADING DATASET:

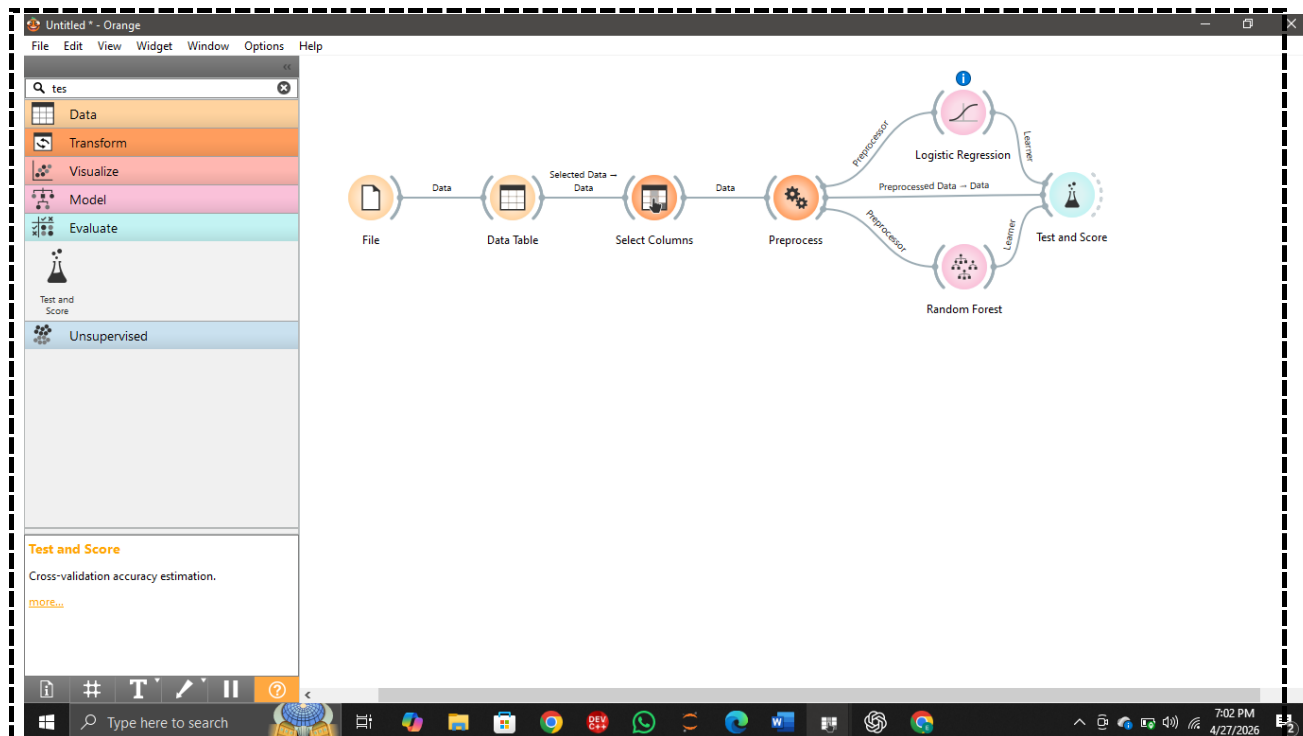


CHECKING ON DATA TABLE OF ORANGE:









Test and Score - Orange

☒ Cross validation

Number of folds: 10

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 66 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☐ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
Logistic Regression	0.735	0.779	0.717	0.698	0.779	0.049
Random Forest	0.792	0.830	0.816	0.814	0.830	0.412

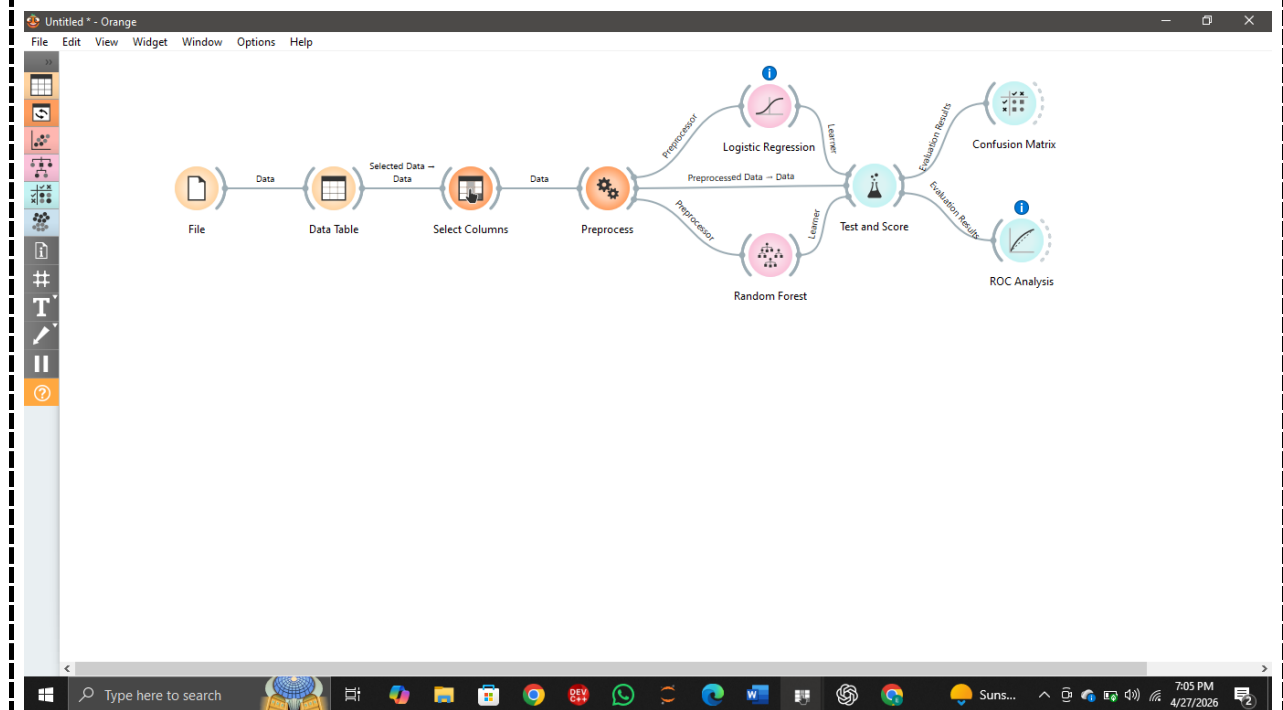
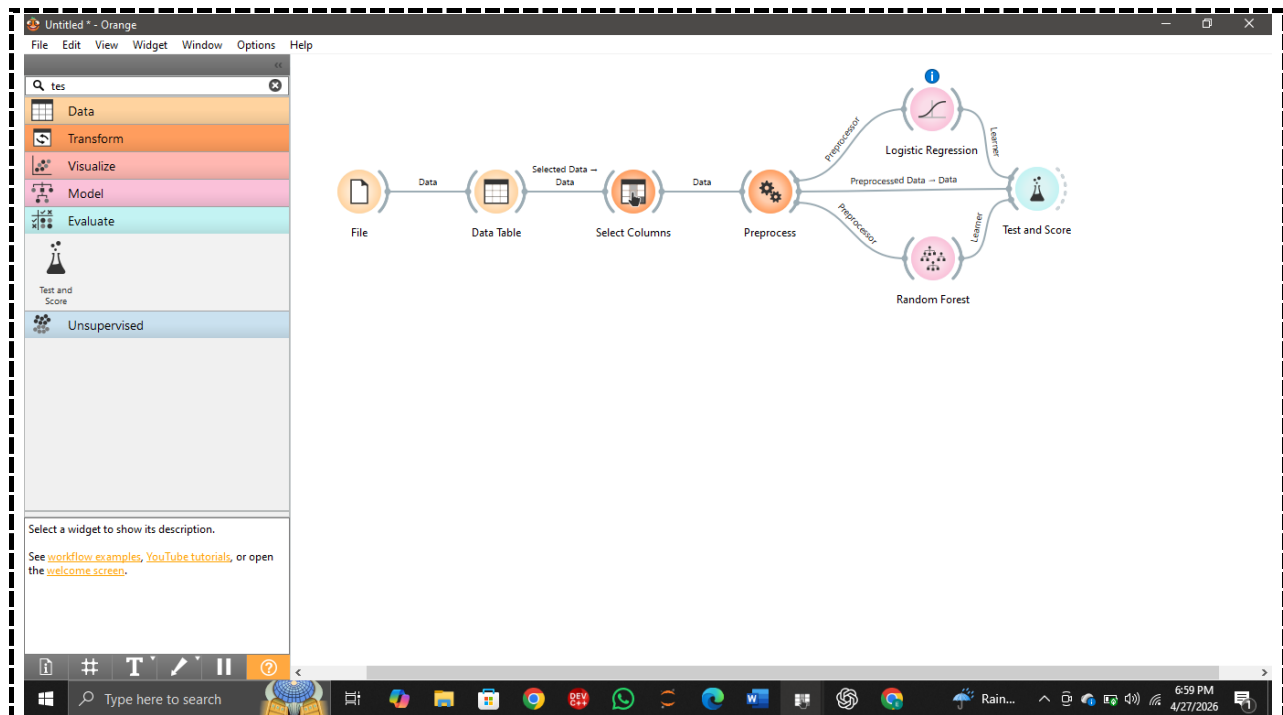
Compare models by: Area under ROC curve

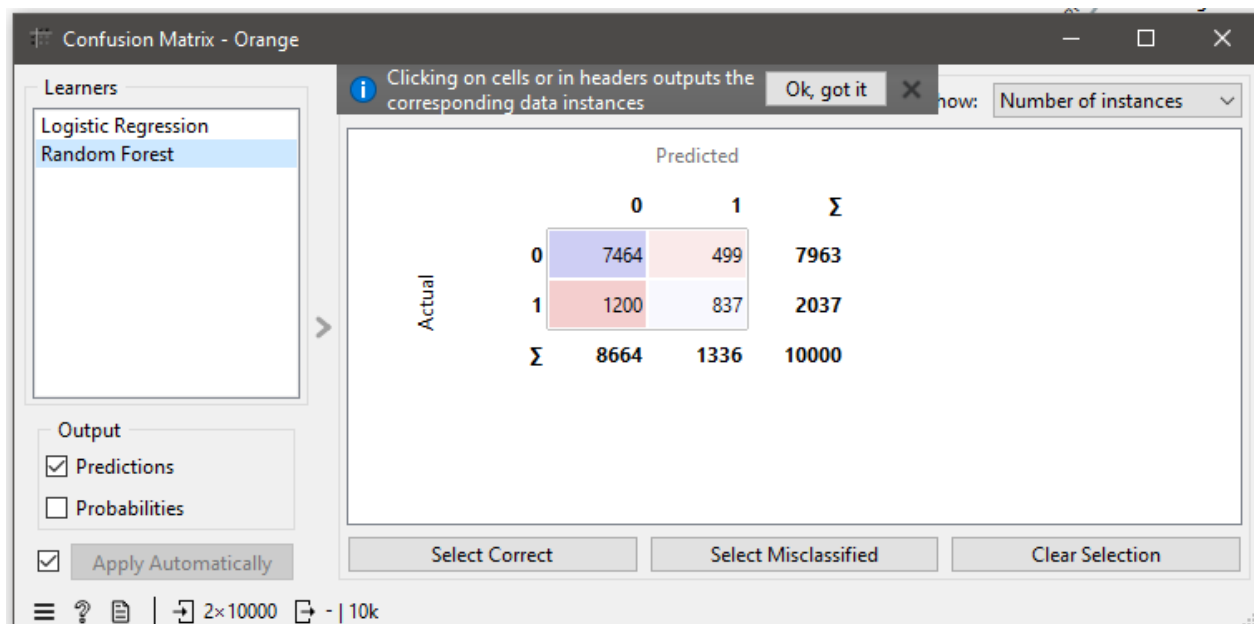
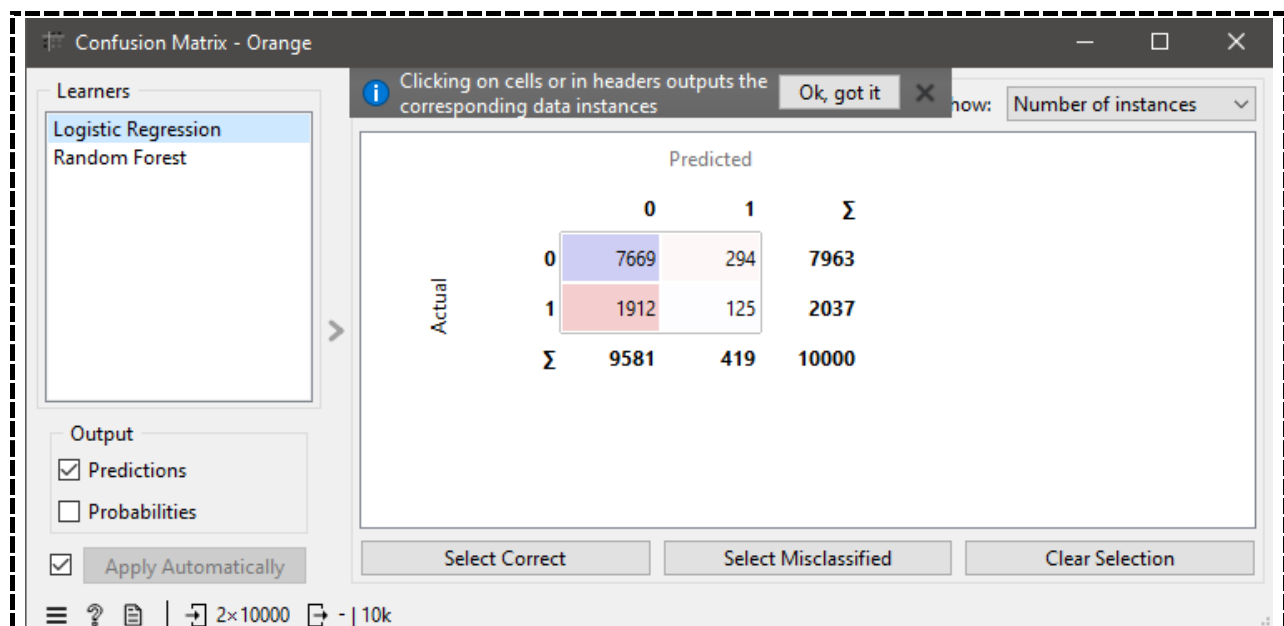
☐ Negligible diff.: 0.1

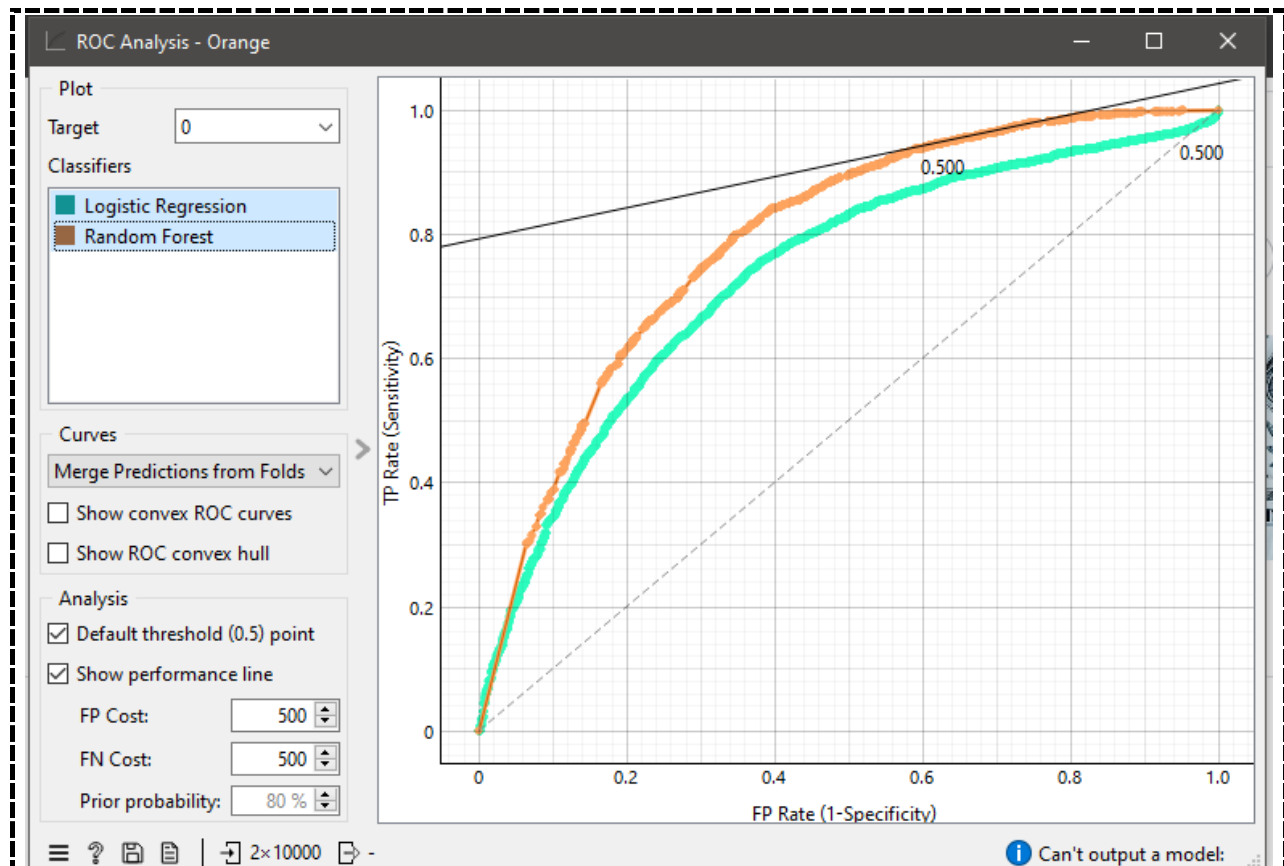
	Logistic Reg...	Random Fo...
Logistic Regression		0.000
Random Forest	1.000	

Table shows probabilities that the score for the model in the row is higher than that of the model in the column. Small numbers show the probability that the difference is negligible.

10k | 10k | 2x10000







LAB-3: Data Cleaning Techniques

1. Handling Missing Values

Missing data occurs when certain values are absent from the dataset. Proper handling is crucial to avoid biased results.

Common techniques include:

- Removing incomplete records
- Replacing missing values with mean, median, or mode
- Using predictive methods (ML-based imputation)

2. Removing Duplicates

Duplicate records can distort analysis and lead to incorrect insights. Removing duplicates ensures data consistency and reliability.

3. Noise Reduction

Noise refers to irrelevant or random variations in data. It can be handled using:

- Binning
- Regression
- Filtering techniques

EXAMPLE USING HEALTHCARE DATASET:

LOADING AND PROFILING HEALTHCARE DATA:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('healthcare_dataset.csv')

print('Dataset Shape:', df_raw.shape)
print('\nFirst 5 rows:')
df_raw.head()
```

... Dataset Shape: (100, 7)

First 5 rows:

	PatientID	Age	Gender	BloodPressure	Glucose	BMI	Diagnosis
0	1	58	Female	116.785133	86.101378	20.288436	Healthy
1	2	71	Female	109.662087	96.806629	10.179522	Healthy
2	3	48	None	NaN	60.986073	32.617709	Healthy
3	4	34	Female	110.713960	150.008702	37.716026	Healthy
4	5	62	Female	125.342397	69.996707	25.287056	Healthy

Next steps: [Generate code with df_raw](#) [New interactive sheet](#)

Step 1: Load and Profile the Data

```
# Profile the dataset
print('--- DATASET PROFILE ---')
print(f'Total Rows : {df_raw.shape[0]}')
print(f'Total Columns : {df_raw.shape[1]}')
print(f'\nData Types:')
print(df_raw.dtypes)
print(f'\nMissing Values per Column:')
print(df_raw.isnull().sum())
print(f'\nTotal Missing Cells : {df_raw.isnull().sum().sum()}')
print(f'Total Duplicate Rows: {df_raw.duplicated(subset="PatientID").sum()}')
```

--- DATASET PROFILE ---

Total Rows : 100
Total Columns : 7

Data Types:

PatientID	int64
Age	int64
Gender	object
BloodPressure	float64
Glucose	float64
BMI	float64
Diagnosis	object
dtype:	object

Missing Values per Column:

PatientID	0
Age	0
Gender	9
BloodPressure	5
Glucose	10
BMI	8
Diagnosis	7
dtype:	int64

Total Missing Cells : 39
Total Duplicate Rows: 10

HANDLING MISSING VALUES:

Step 2: Handle Missing Values

```
df_clean = df_raw.copy()

# --- Numeric columns: fill with mean ---
numeric_cols = ['Glucose', 'BMI', 'BloodPressure']
for col in numeric_cols:
    before = df_clean[col].isnull().sum()
    df_clean[col].fillna(df_clean[col].mean(), inplace=True)
    after = df_clean[col].isnull().sum()
    print(f'{col}: {before} missing -> filled with mean ({df_raw[col].mean():.2f}) -> {after} missing')

# --- Categorical columns: fill with most frequent value ---
cat_cols = ['Gender', 'Diagnosis']
for col in cat_cols:
    before = df_clean[col].isnull().sum()
    most_freq = df_clean[col].mode()[0]
    df_clean[col].fillna(most_freq, inplace=True)
    after = df_clean[col].isnull().sum()
    print(f'{col}: {before} missing -> filled with "{most_freq}" -> {after} missing')

print(f'\nTotal missing after handling: {df_clean.isnull().sum().sum()}')
```

Glucose: 10 missing -> filled with mean (101.08) -> 0 missing
BMI: 8 missing -> filled with mean (26.49) -> 0 missing
BloodPressure: 5 missing -> filled with mean (118.62) -> 0 missing
Gender: 9 missing -> filled with "Male" -> 0 missing
Diagnosis: 7 missing -> filled with "Healthy" -> 0 missing

Total missing after handling: 0

REMOVING DUPLICATE RECORDS:

Step 3: Remove Duplicate Records

```
# Check duplicates BEFORE
dups_before = df_clean.duplicated(subset='PatientID').sum()
print(f'Duplicate rows BEFORE removal: {dups_before}')
print(f'Total rows BEFORE : {len(df_clean)}')

# Remove duplicates - keep first occurrence
df_clean = df_clean.drop_duplicates(subset='PatientID', keep='first')

dups_after = df_clean.duplicated(subset='PatientID').sum()
print(f'\nDuplicate rows AFTER removal : {dups_after}')
print(f'Total rows AFTER : {len(df_clean)}')
print(f'Rows removed : {dups_before}')
```

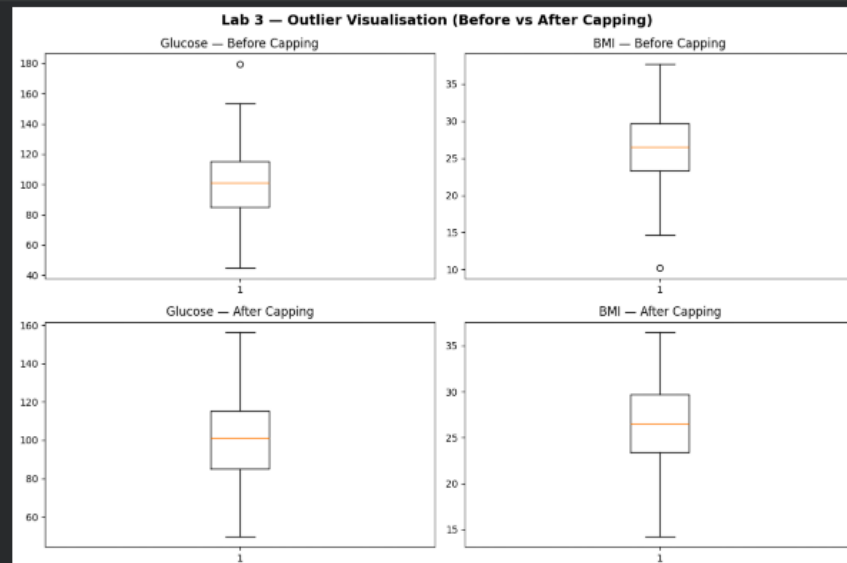
Duplicate rows BEFORE removal: 10
Total rows BEFORE : 100

Duplicate rows AFTER removal : 0
Total rows AFTER : 90
Rows removed : 10

NOISE FILTERING WITH BOX PLOT VISUALIZATION:

Step 4: Noise Filtering

```
df_clean['BloodPressure_Smoothed'] = df_clean['BloodPressure'].rolling(window=3, min_periods=1).mean()
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle('Lab 3 - Outlier Visualisation (Before vs After Capping)', fontsize=14, fontweight='bold')
axes[0, 0].boxplot(df_clean['Glucose'].dropna())
axes[0, 0].set_title('Glucose - Before Capping')
axes[0, 1].boxplot(df_clean['BMI'].dropna())
axes[0, 1].set_title('BMI - Before Capping')
for col in ['Glucose', 'BMI', 'BloodPressure']:
    low = df_clean[col].quantile(0.01)
    high = df_clean[col].quantile(0.99)
    df_clean[col] = np.clip(df_clean[col], low, high)
axes[1, 0].boxplot(df_clean['Glucose'].dropna())
axes[1, 0].set_title('Glucose - After Capping')
axes[1, 1].boxplot(df_clean['BMI'].dropna())
axes[1, 1].set_title('BMI - After Capping')
plt.tight_layout()
plt.show()
print('Noise filtering complete.')
```



LAB-4: Data Preprocessing

Normalization

Normalization rescales data into a fixed range (usually 0–1), ensuring equal contribution of all features.

Formula:

$$(X - \text{Min}) / (\text{Max} - \text{Min})$$

Standardization

Standardization transforms data to have:

- Mean = 0
- Standard Deviation = 1

Formula:

$$(X - \text{Mean}) / \text{Std}(X - \text{Mean})$$

Discretization

Converts continuous data into categorical groups to simplify analysis.

Summary Statistics

Key statistical measures include:

- Mean
- Median
- Mode
- Variance
- Standard Deviation

These help in understanding data distribution.

1. Summary Statistics

```
[1]: import pandas as pd

# Load the dataset
df = pd.read_csv('loan_prediction_dataset.csv')
```

```
[9]: print("Summary Statistics:")
display(df.describe())
```

Summary Statistics:

	Age	Income	Credit_Score	Loan_Amount	Loan_Term	Loan_Approved
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	43.805500	84533.585000	577.055000	25460.315000	35.47800	0.171000
std	14.929203	37771.169751	157.525951	14116.737774	16.98587	0.376603
min	18.000000	20155.000000	300.000000	1060.000000	12.00000	0.000000
25%	31.000000	50925.250000	440.000000	13444.250000	24.00000	0.000000
50%	44.000000	84073.500000	578.500000	25446.000000	36.00000	0.000000
75%	56.000000	117523.250000	715.250000	37949.250000	48.00000	0.000000
max	69.000000	149992.000000	849.000000	49994.000000	60.00000	1.000000

2. Normalization (Min-Max Scaling)

```
[4]: from sklearn.preprocessing import MinMaxScaler

# Initialize the scaler
scaler_minmax = MinMaxScaler()

# We will normalize 'Income' and 'Loan_Amount'
df_normalized = df.copy()
cols_to_normalize = ['Income', 'Loan_Amount']
df_normalized[cols_to_normalize] = scaler_minmax.fit_transform(df[cols_to_normalize])

print("Normalized Data (First 5 rows):")
print(df_normalized[cols_to_normalize].head())
```

Normalized Data (First 5 rows):

	Income	Loan_Amount
0	0.474695	0.285323
1	0.637137	0.407753
2	0.297850	0.789369
3	0.824364	0.323415
4	0.042361	0.514898

3. Standardization (Z-score Scaling)

```
[5]: from sklearn.preprocessing import StandardScaler

# Initialize the scaler
scaler_std = StandardScaler()

# We will standardize 'Age' and 'Credit_Score'
df_standardized = df.copy()
cols_to_standardize = ['Age', 'Credit_Score']
df_standardized[cols_to_standardize] = scaler_std.fit_transform(df[cols_to_standardize])

print("Standardized Data (First 5 rows):")
print(df_standardized[cols_to_standardize].head())
```

Standardized Data (First 5 rows):

	Age	Credit_Score
0	0.817026	-1.543338
1	1.688020	1.294999
2	0.147031	1.282300
3	-0.790963	-1.352846
4	1.085024	-1.714781

4. Discretization (Binning)

```
[7]: df_discretized = df.copy()
df_discretized['Income_Category'] = pd.cut(df['Income'],
                                          bins=3,
                                          labels=['Low', 'Medium', 'High'])

from sklearn.preprocessing import KBinsDiscretizer
est = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
df_discretized['Age_Bins'] = est.fit_transform(df[['Age']])

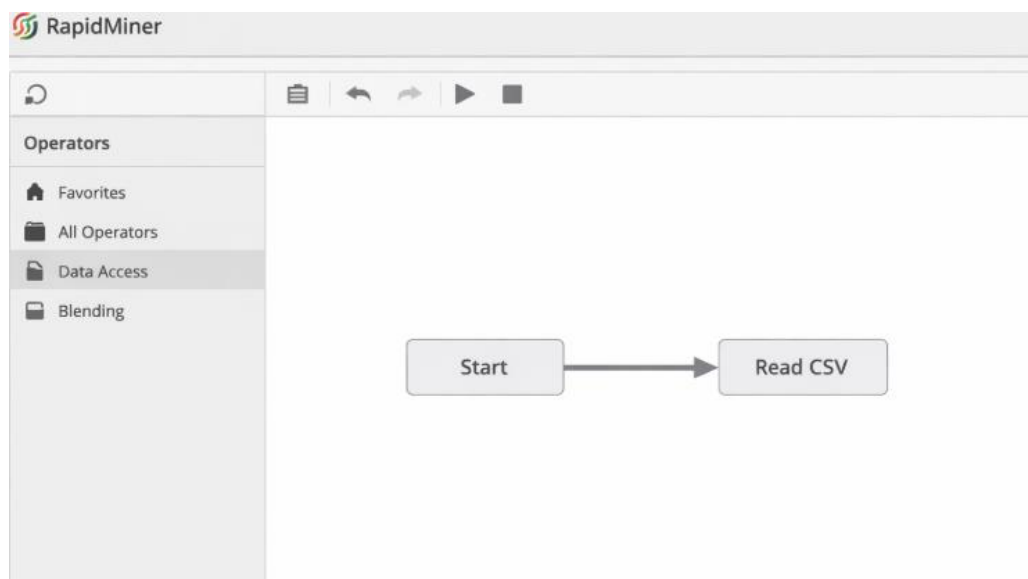
print("Discretized Data (First 5 rows):")
print(df_discretized[['Income', 'Income_Category', 'Age', 'Age_Bins']].head())
```

Discretized Data (First 5 rows):

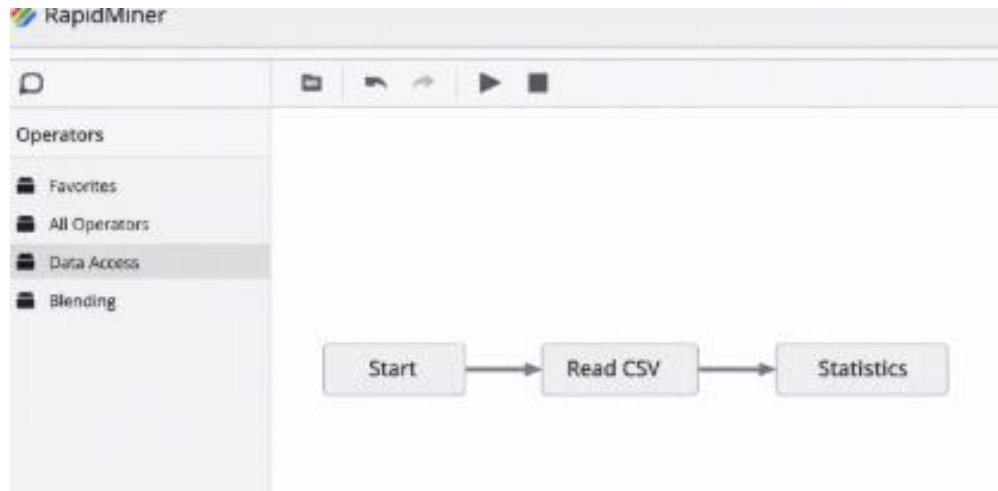
	Income	Income_Category	Age	Age_Bins
0	81788	Medium	56	2.0
1	102879	Medium	69	2.0
2	58827	Low	46	1.0
3	127188	High	32	0.0
4	25655	Low	60	2.0

RAPID MINER WORKFLOW:

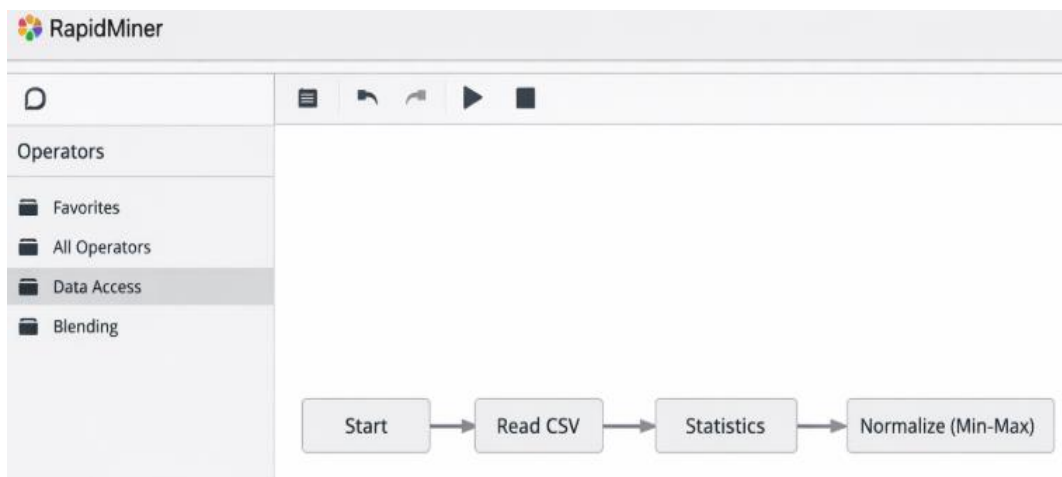
STEP 1: Start → Read CSV connected by an arrow.



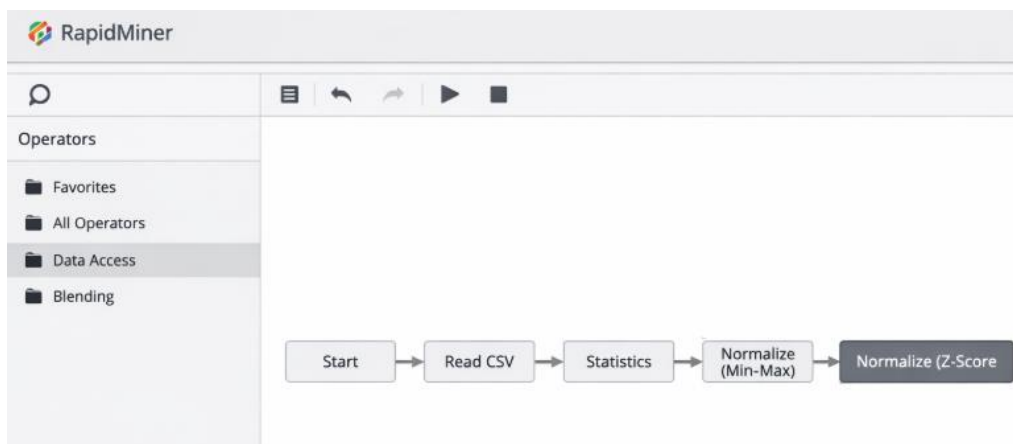
STEP 2: Start → Read CSV → Statistics.



STEP 3: Start → Read CSV → Statistics → Normalize (Min-Max).

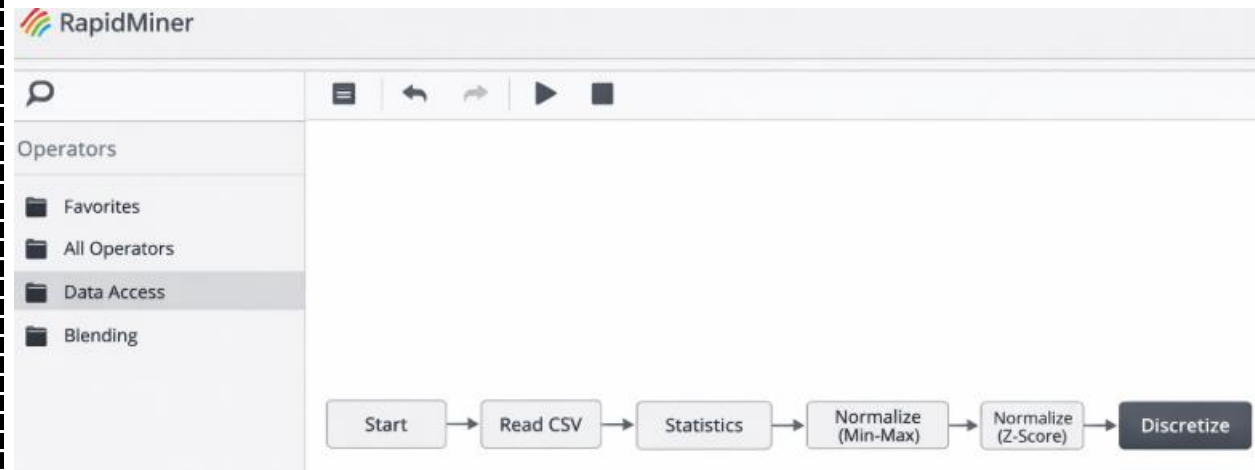


STEP 4: Start → Read CSV → Statistics → Normalize (Min-Max) → Normalize (Z-Score).



STEP 5:

Start → Read CSV → Statistics → Normalize (Min-Max) → Normalize (Z-Score) → Discretize.



LAB-5: Market Basket Analysis

Market Basket Analysis

Market Basket Analysis is a data mining technique used to discover relationships between items that are frequently purchased together. It helps businesses better understand customer purchasing patterns and make improved decisions regarding product placement, marketing strategies, and recommendation systems.

Example:

In a supermarket dataset, customers who buy tea often also purchase biscuits, indicating a strong association between these items.

1. Support

Support indicates how often an item or a set of items appears in the dataset. It reflects the overall frequency or popularity of an itemset.

Formula:

Support (X) = Number of transactions containing X / Total number of transactions

Example:

If milk appears in 20 out of 100 transactions, then:

Support (milk) = $20 / 100 = 0.2$

2. Confidence

Confidence measures the probability that item Y is purchased when item X has already been bought. It shows the strength of the relationship between two items.

Formula:

Confidence ($X \rightarrow Y$) = $\text{Support}(X \cup Y) / \text{Support}(X)$

Example:

If 15 transactions contain both tea and biscuits, and 20 transactions include tea, then:

Confidence (tea $\rightarrow$ biscuits) = $15 / 20 = 0.75$

3. Lift

Lift evaluates how strong the relationship between two items is compared to if they were independent. It helps determine whether the association is meaningful or just coincidental.

Formula:

Lift ($X \rightarrow Y$) = $\text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$

- Lift > 1: Indicates a positive association (items are likely related)
- Lift < 1: Indicates a negative association (items rarely occur together)
- Lift = 1: Indicates no association (items are independent)

Example:

If Confidence (tea → biscuits) = 0.75 and Support (biscuits) = 0.5, then:

Lift = 0.75 / 0.5 = 1.5 → showing a strong positive relationship between the items

Step 1: Load Data

```
[1]: import pandas as pd
```

```
df = pd.read_csv("transaction_data.csv")
df.head()
```

	UserId	TransactionId	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

Step 2: Convert Data into Transaction List

```
[5]: transactions = df.groupby('TransactionId')['ItemDescription'].apply(list)
transactions = list(transactions)

print(transactions[:5])
```

```
[['SET 7 BABUSHKA NESTING BOXES', 'WHITE METAL LANTERN', 'RED WOOLLY HOTTIE WHITE HEART.', 'CREAM CUPID HEARTS COAT HANGER', 'SET 7 BABUSHKA NESTING BOXES', 'KNITTED UNION FLAG HOT WATER BOTTLE', 'KNITTED UNION FLAG HOT WATER BOTTLE', 'WHITE HANGING HEART T-LIGHT HOLDER', 'WHITE METAL LANTERN', 'CREAM CUPID HEARTS COAT HANGER', 'RED WOOLLY HOTTIE WHITE HEART.', 'GLASS STAR FROSTED T-LIGHT HOLDER', 'WHITE HANGING HEART T-LIGHT HOLDER', 'GLASS STAR FROSTED T-LIGHT HOLDER'], ['HAND WARMER RED POLKA DOT', 'HAND WARMER UNION JACK', 'HAND WARMER UNION JACK', 'HAND WARMER RED POLKA DOT'], ['DOORMAT NEW ENGLAND', 'HOME BUILDING BLOCK WORD', 'LOVE BUILDING BLOCK WORD', 'ASSORTED COLOUR BIRD ORNAMENT', 'FELTCRAFT PRINCESS CHARLOTTE DOLL', 'POPPY'S PLAYHOUSE BEDROOM', 'LOVE BUILDING BLOCK WORD', 'ASSORTED COLOUR BIRD ORNAMENT', 'BOX OF 6 ASSORTED COLOUR TEASPOONS', 'RECIPE BOX WITH METAL HEART', 'IVORY KNITTED MUG COSY', 'BOX OF VINTAGE JIGSAW BLOCKS', 'BOX OF VINTAGE ALPHABET BLOCKS', 'POPPY'S PLAYHOUSE BEDROOM', 'BOX OF VINTAGE ALPHABET BLOCKS', 'POPPY'S PLAYHOUSE KITCHEN', 'FELTCRAFT PRINCESS CHARLOTTE DOLL', 'DOORMAT NEW ENGLAND', 'HOME BUILDING BLOCK WORD', 'IVORY KNITTED MUG COSY', 'RECIPE BOX WITH METAL HEART', 'BOX OF 6 ASSORTED COLOUR TEASPOONS', 'POPPY'S PLAYHOUSE KITCHEN', 'BOX OF VINTAGE JIGSAW BLOCKS'], ['BLUE COAT RACK PARIS FASHION', 'RED COAT RACK PARIS FASHION', 'YELLOW COAT RACK PARIS FASHION', 'JAM MAKING SET WITH JARS', 'YELLOW COAT RACK PARIS FASHION', 'RED COAT RACK PARIS FASHION', 'BLUE COAT RACK PARIS FASHION', 'JAM MAKING SET WITH JARS'], ['BATH BUILDING BLOCK WORD', 'BATH BUILDING BLOCK WORD']]
```

```
[6]: transactions = [list(set(t)) for t in transactions]
```

Step 3: Total Transactions

```
[7]: total_transactions = len(transactions)
     print(total_transactions)
```

25900

Step 4: Manual Calculation ($A \rightarrow B$)

```
[8]: A = 'WHITE HANGING HEART T-LIGHT HOLDER'
     B = 'CREAM CUPID HEARTS COAT HANGER'
```

```
[9]: count_A = 0
     count_B = 0
     count_AB = 0

     for t in transactions:
         if A in t:
             count_A += 1
         if B in t:
             count_B += 1
         if A in t and B in t:
             count_AB += 1

     support = count_AB / total_transactions
     confidence = count_AB / count_A if count_A != 0 else 0
     lift = confidence / (count_B / total_transactions) if count_B != 0 else 0

     print("Support:", support)
     print("Confidence:", confidence)
     print("Lift:", lift)
```

Support: 0.004362934362934363
Confidence: 0.04908774978279757
Lift: 4.384043859911921

LAB-6: Apriori Algorithm

The **Apriori Algorithm** is one of the most fundamental techniques in data mining used for **association rule learning**. It helps identify relationships between items in large datasets, especially in applications like market basket analysis.

Key Concept (Apriori Principle):

The algorithm is based on a simple idea:

If an itemset is frequent, then all of its subsets must also be frequent.

This principle allows the algorithm to reduce the search space by eliminating itemsets that are unlikely to be frequent.

Working of Apriori Algorithm:

The algorithm operates in multiple iterations:

Step 1: Generate Frequent 1-itemsets

- Count the frequency of individual items
- Remove items that do not meet the minimum support threshold

Step 2: Generate Candidate Itemsets

- Combine frequent itemsets to create larger itemsets (2-itemsets, 3-itemsets, etc.)

Step 3: Pruning

- Remove itemsets whose subsets are not frequent

Step 4: Repeat

- Continue until no more frequent itemsets are found

Important Measures:

1. Support

Indicates how frequently an itemset appears:

$\text{Support}(X) = \frac{\text{Transactions containing } X}{\text{Total Transactions}}$

2. Confidence

Measures the likelihood of Y given X:

$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$

3. Lift

Evaluates strength of association:

$\text{Lift} = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$

Advantages

- Simple and easy to understand
- Effective for small datasets
- Generates interpretable rules

Disadvantages

- Computationally expensive for large datasets
- Generates too many candidate itemsets
- Requires multiple database scans

Applications

- Retail (product recommendation)
- E-commerce
- Fraud detection
- Web usage mining

```
[2]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import apriori, association_rules
```

Load Your Dataset

```
[3]: df = pd.read_csv('transaction_data.csv')
df.head()
```

	Userld	TransactionId	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

```
[6]: df = df.dropna()

# Remove negative or zero quantities (if exist)
df = df[df['NumberOfItemsPurchased'] > 0]

df.head()
```

	Userld	TransactionId	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

Convert into Basket Format

```
[7]: basket = df.groupby(['TransactionId', 'ItemDescription'])['NumberOfItemsPurchased'] \
        .sum().unstack().reset_index().fillna(0)

basket.head()
```

			4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OV WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	...	returned	taig adjust	test
	0	5900015	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
	1	5900026	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
	2	5900037	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
	3	5900048	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
	4	5900059	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0

5 rows × 4078 columns

Convert Values into 0/1 (Binary Encoding)

```
[8]: def encode(x):
      if x > 0:
          return 1
      else:
          return 0

      basket_encoded = basket.set_index('TransactionId').applymap(encode)

      basket_encoded.head()
```

C:\Users\star\AppData\Local\Temp\ipykernel_11340\1282610810.py:7: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead
basket_encoded = basket.set_index('TransactionId').applymap(encode)

```
[8]:
```

ItemDescription	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OVAL WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	SPACEBOY BABY GIFT SET	...	returned	taig adjust	test
TransactionId														
5900015	0	0	0	0	0	0	0	0	0	0	...	0	0	0
5900026	0	0	0	0	0	0	0	0	0	0	...	0	0	0

Apply Apriori Algorithm

```
[10]: rules = association_rules(frequent_itemsets,
                               metric="confidence",
                               min_threshold=0.6)

      rules.head()
```

```
[10]: antecedents consequents antecedent consequent support confidence lift representativity leverage conviction zhangs_metric jaccard certainty kulczynski
      support support
```

Display Important Rules

```
[11]: rules = rules.sort_values(by='confidence', ascending=False)

      rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
```

```
[11]: antecedents consequents support confidence lift
```

Filter Strong Rules

```
[12]: strong_rules = rules[(rules['confidence'] > 0.7) & (rules['lift'] > 1)]

      strong_rules
```

```
[12]: antecedents consequents antecedent consequent support confidence lift representativity leverage conviction zhangs_metric jaccard certainty kulczynski
      support support
```


KNIME WORKFLOW:

STEP 1: CSV Reader – Loading the Transaction Dataset

Configured and Loading Dataset

CSV Reader - Configuration

File path: C:\Users\user\data\transaction_data.csv

Variation: Volume

Preview Tables

File	Steliden	ItemDescription	NumbersOrim...	NumPartiam...
Item 1	2022-05-23 156...	bine	38.00	
Item 2	2025-06-22 136...	Kembescription	28.00	
Item 3	2020-06-23 128...	Item	68.00	
Item 4	2020-06-23 155...	Scound	48.00	
Item 5	2020-06-23 156...	Seed	58.00	

Data Output

UserId	TransactionId	ItemsDescription	ItemDescription	Number...	NonPartiam	Oturtivu
Item 1	278166	202-06-25 13 13:35:36...	FAMILY ALBUM WHITE PICTURE FRA...	5.00	5.50 Together	
Item 2	278166	202-06-23 13 13:33:32...	GUPham CIG GATTER FALE	5.00	3.50 Together	
Item 3	278166	202-06-23 13 13:33:36...	FAMILY ALBUM WHITE PICTURE RGR...	5.00	Indens	
Item 4	278126	202-06-23 13 13:35:36...	FAMILY ALBUM WRESGE KREUT ABIL	5.00	Indens	
Item 5	278130	202-06-23 13 13:33:36...	ROMSMON PASIVATATBER	5.00	Indens	
Item 6	278432	202-06-23 13 13:33:36...	ROMSMON JAAG LAG TABLE	5.00	Indens	
Item 7	278876	2020-6-23 13 13:39:36...	Nones	5.00	Indens	
Item 8	278560	2020-6-23 13 13:39:53...	stvar	5.00	Indens	
Item 9	278576	2020-6-23 13 13:39:35...	uldb	5.00	Indens	
Item 10	200598	2020-6-23 13 13:33:49...	uldb	5.00	Indens	
Item 11	200580	202-06-23 13 13:39:36...	Nexnescriptazed Toiletad:	5.00	Indens	
Item 12	200510	202-06-23 13 13:45:46...	FAMILY ALBUM MEIT FAGFTTARE	5.00	3.50 Together	

STEP 2: Missing Value & Row Filter – Handling Nulls and Negative Quantities

Missing Value - Configuration

Column: Configuration

Configuration: Remove Row

Missing Data: Kamara Row

Remove missing Data

Remove enstoves to missine Kloe

Data Output

UserId	TransactionId	ItemDescription
Row 1	0	
Row 2	0	
Row 3	5	
Row 4	0	
Row 5	0	

STEP 3: Column Filter

Column Filter - Configuration

Select the columns to into selected columns.

All Columns:

- TransactionTime
- ItemCode
- ItemDescription
- ItemPerItem
- ItemCode
- Item Taxa
- NumberOfItemsPurchased
- BasePerItem

Selected Columns:

- TransactionTime
- ItemCode
- ItemDescription
- NumberOfItemsPurchased

Data Output

Userid	TransactionId	ItemDescription	ItemDescription	NumberOfItemsPurchased
Item 1	278166	202-06-23 13 13:33:46...	TransactionTime	380
Item 2	278166	202-06-23 13 13:33:36...	(FAMILY ALBUM WHITE PICTUR...	300
Item 3	337701	Wed 06-29 15 13:33:46...	ItemCode	100
Item 4	278795	202-06-23 13 13:33:36...	LONDON BUS COFFEE MUG	5
Item 5	628989	Fr/4 06-22 15 13:33:56...	ST LOU COLOUR DOLLY DOLLY...	72
Item 6	278129	202-06-23 13 13:33:36...	ROMISON JACK WRESSE KREUT...	5
Item 6	614325	Mon 06-21 11 15:33:46...	WASHRON JACK LAG TAVIS	3

STEP 4: Create Collection Column

Create Collection Column - Configuration

Group by:

TransactionId

Column to collect:

ItemDescription

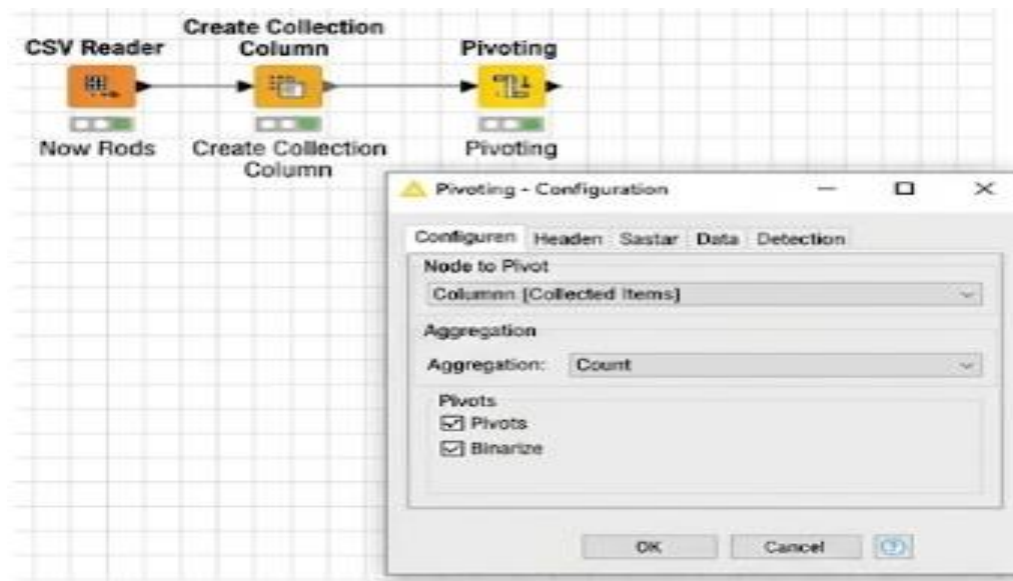
Preview Tables

File	Item List
Row 1 (278166)	(FAMILY ALBUM WHITE PICTURE FRAME, ... LONDON BUS COFFEE MUG), LIVEIZERI...
Row 2 (337701)	(LONDON BUS COFFEE MUG)
Row 3 (271626)	ROMSMON FASSIVATVER
Row 4 (278436)	ROMSMON JAAG LAG TABLE
Row 5 (278850)	star
Row 6 (200599)	ROMSION JACK LAG TAG3 LAGS

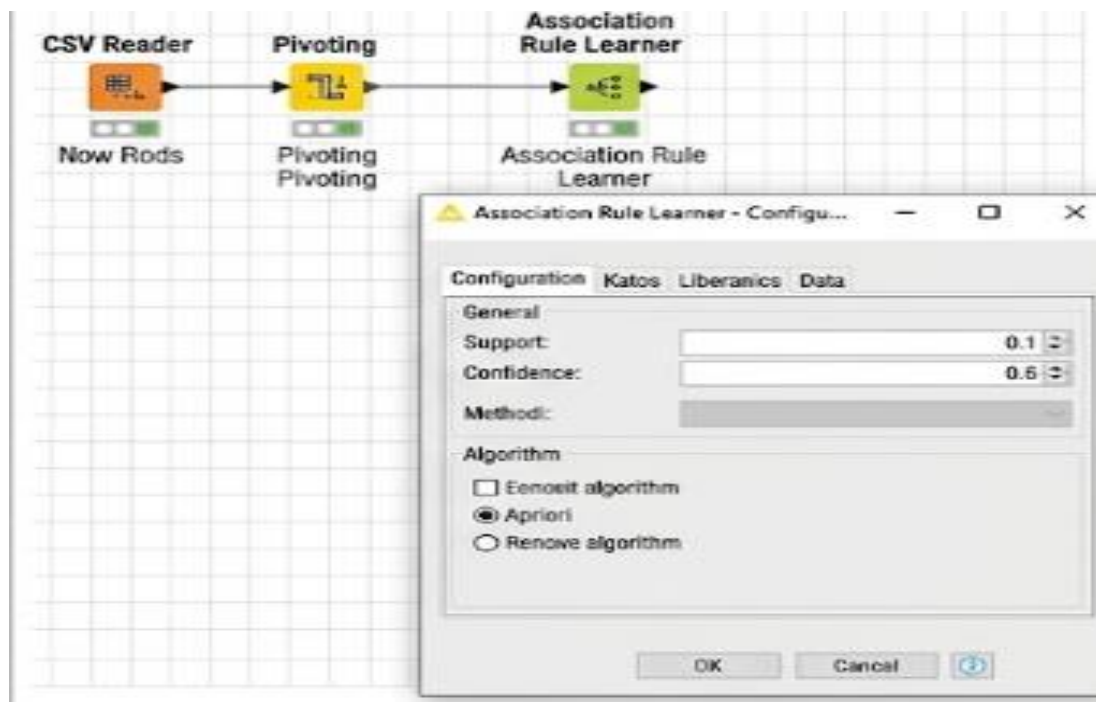
Data Output

ID	TransactionId	Collected Items
Row 1	278166	(FAMILY ALBUM WHITE PICTURE FRAME, ... LONDON BUS COFFEE MUG), LIVEIZERI...
Row 2	337701	(LONDON BUS COFFEE MUG)
Row 3	278509	(FAMILY ALBUM WHITE PICTURE FRAME, ... LONDON BUS COFFEE MUGS) 490776R
Row 4	278126	(FAMILY ALBUM WRESSE KREUT ARIL, ... [AASHRON BUS COFFEE BAG, RED SET T...
Row 5	278432	ROMSSON FASIVATVAR LADNS, ROMSSONN WHITE KREUT BAG, RED SET TKA...
Row 6	200599	WASHRON JACK LAG TAGSE RXL OROEL LOCK MINI BAG, UNMEFDOT LAG LARGE...

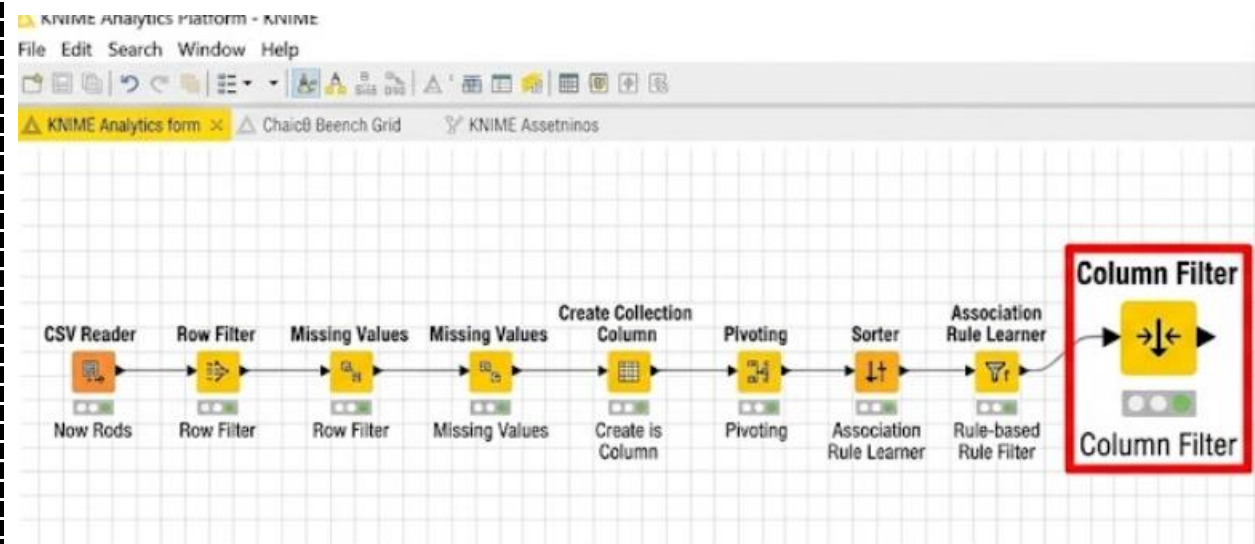
STEP 5: Pivoting & One-Hot Encoding



STEP 6: Association Rule Learner



FINAL STEP: Structuring the Clean Association Rules Output



LAB-7: FP-Tree & FP-Growth

Objective:

To implement the FP-Growth algorithm on a retail transaction dataset, construct an FP-Tree, generate association rules, and compare its performance with the Apriori algorithm using execution time and rule count.

Introduction:

Frequent Pattern Mining is an important data mining technique used to discover recurring relationships among items in large datasets. It helps organizations identify patterns in customer purchasing behavior and supports decision-making in areas such as marketing, sales, and inventory management.

The FP-Growth (Frequent Pattern Growth) algorithm is an efficient method for mining frequent itemsets. Unlike Apriori, FP-Growth does not generate candidate itemsets repeatedly. Instead, it uses a compact tree structure called the FP-Tree, which significantly reduces computation time and memory usage.

Because of its efficiency, FP-Growth is widely used for analyzing large transaction databases.

FP-Growth Algorithm:

FP-Growth is a frequent pattern mining algorithm that discovers frequent itemsets by constructing a compressed representation of the database called the Frequent Pattern Tree (FP-Tree).

The algorithm works in two main phases:

1. Construct the FP-Tree from transaction data.
2. Mine the FP-Tree to extract frequent itemsets and association rules.

Advantages of FP-Growth

- Faster than Apriori for large datasets.
- Requires fewer database scans.
- Does not generate candidate itemsets.
- Uses less memory through data compression.
- Efficient for large-scale market basket analysis.

FP-Tree (Frequent Pattern Tree):

An FP-Tree is a compact tree-based structure used to store transaction information.

The construction process includes:

- Scanning the database to determine item frequencies.
- Removing infrequent items.
- Sorting items according to frequency.
- Building the FP-Tree using the sorted transactions.

The FP-Tree preserves item associations while reducing storage requirements.

Example

Transactions:

T1 = Bread, Milk

T2 = Bread, Butter

T3 = Bread, Milk, Butter

T4 = Milk, Butter

Resulting frequent patterns may include:

- Bread → Milk
- Milk → Butter
- Bread → Butter

Dataset Used:

Large Retail Dataset

The Retail Dataset contains customer transaction records collected from retail stores.

Typical attributes include:

- Transaction ID
- Product Items
- Purchase Details
- Quantity Purchased

Each record represents a customer's shopping basket containing one or more products.

Software Used:

- Orange Data Mining
- Python
- Pandas
- Mlxtend Library

Methodology:

1. Load the retail transaction dataset.
2. Preprocess and clean the data.
3. Transform transactions into suitable format.
4. Apply the FP-Growth algorithm.
5. Construct the FP-Tree.
6. Generate frequent itemsets.
7. Generate association rules.
8. Apply Apriori algorithm.
9. Compare both algorithms using performance metrics.

Association Rules:

Association rules describe relationships between items purchased together.

General form:

$$A \rightarrow B$$

Meaning:

If a customer purchases item A, there is a likelihood that item B will also be purchased.

Example

$$\{\text{Bread}\} \rightarrow \{\text{Milk}\}$$

This indicates that customers who buy bread often buy milk as well.

Comparison with Apriori:

Apriori:

- Generates candidate itemsets repeatedly.
- Requires multiple database scans.
- Slower for large datasets.
- Higher computational cost.

FP-Growth:

- Uses FP-Tree structure.
- Avoids candidate generation.
- Requires fewer database scans.
- Faster and more scalable.

Performance Evaluation Metrics:

1. Execution Time

Execution time measures the total time required to discover frequent itemsets and generate association rules.

Lower execution time indicates better algorithm efficiency.

2. Rule Count

Rule count represents the total number of association rules generated by the algorithm.

A larger number of meaningful rules provides better insights into customer purchasing behavior.

Real-World Applications:

Market Basket Analysis:

Retail stores analyze products frequently purchased together to improve sales strategies.

Recommendation Systems:

E-commerce platforms recommend products based on customer purchasing patterns.

Inventory Management:

Businesses use frequent pattern mining to maintain optimal stock levels.

Cross-Selling:

Companies identify related products and promote them together.

Real-World Example:

A supermarket analyzes transaction data and discovers that customers who buy bread and milk frequently purchase butter as well.

Based on this information:

- Bread, milk, and butter can be placed near each other.
- Bundle offers can be created.
- Product recommendations can be improved.

This increases customer convenience and overall sales.

Expected Results:

After applying FP-Growth and Apriori on the retail dataset:

- FP-Growth should produce similar frequent itemsets.
- FP-Growth should execute faster than Apriori.
- FP-Growth should require less memory.
- Association rules should reveal product purchasing patterns.

```
[1]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import fpgrowth, association_rules
```

```
[3]: df = pd.read_csv("retail_dataset.csv")
```

```
[7]: df.head()
```

```
[7]:
```

	TransactionID	Item
0	T1	Laptop
1	T1	Mouse
2	T1	Laptop Bag
3	T2	Headphones
4	T2	Mouse

Convert into Transaction Format

```
[8]: transactions = df.groupby('TransactionID')['Item'].apply(list)

      # Convert to list of lists
      transactions = list(transactions)

      # Show sample
      transactions[:5]
```

```
[8]: [['Laptop', 'Mouse', 'Laptop Bag'],
      ['Laptop', 'Mouse', 'USB Hub'],
      ['Printer', 'Paper Pack'],
      ['Monitor', 'Mouse'],
      ['Camera', 'Memory Card']]
```

One-Hot Encoding

```
[9]: te = TransactionEncoder()
      te_array = te.fit(transactions).transform(transactions)

      df_encoded = pd.DataFrame(te_array, columns=te.columns_)

      df_encoded.head()
```

```
[9]:
```

	Bluetooth Adapter	Camera	Camera Bag	Cooling Pad	HDMI Cable	Headphones	Ink Cartridge	Keyboard	Laptop	Laptop Bag	...	Paper Pack	Phone Case	Printer	Screen Protector	Smartphone	Stylus Pen	Tablet
0	False	False	False	False	False	False	False	False	True	True	...	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	...	True	False	True	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False
4	False	True	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False

5 rows x 23 columns

Apply FP-Growth

```
[13]: # FP-Growth
frequent_itemsets = fpgrowth(df_encoded, min_support=0.1, use_colnames=True)

print(frequent_itemsets)
```

	support	itemsets
0	0.333333	(Mouse)
1	0.222222	(Laptop)
2	0.111111	(Printer)
3	0.166667	(Monitor)
4	0.166667	(Camera)
5	0.166667	(Keyboard)
6	0.111111	(Tablet)
7	0.111111	(Smartphone)
8	0.111111	(Headphones)
9	0.166667	(Mouse, Laptop)
10	0.111111	(Mouse, Monitor)
11	0.111111	(Keyboard, Mouse)

Generate Association Rules

```
[15]: # Association Rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.3)

display(rules)
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski
0	(Mouse)	(Laptop)	0.333333	0.222222	0.166667	0.500000	2.25	1.0	0.092593	1.555556	0.833333	0.428571	0.357143	0.62
1	(Laptop)	(Mouse)	0.222222	0.333333	0.166667	0.750000	2.25	1.0	0.092593	2.666667	0.714286	0.428571	0.625000	0.62
2	(Mouse)	(Monitor)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50
3	(Monitor)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
4	(Keyboard)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
5	(Mouse)	(Keyboard)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50

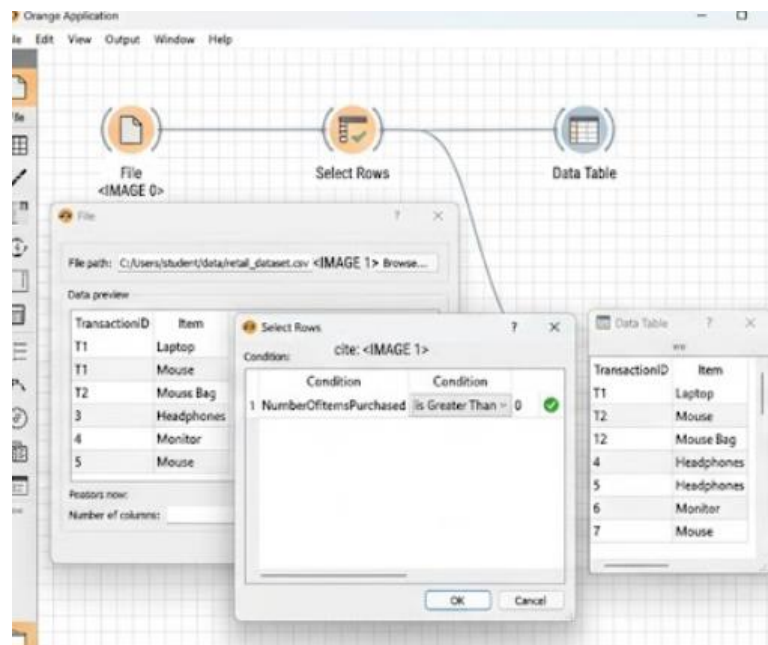
```
[16]: rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
```

```
[16]:
```

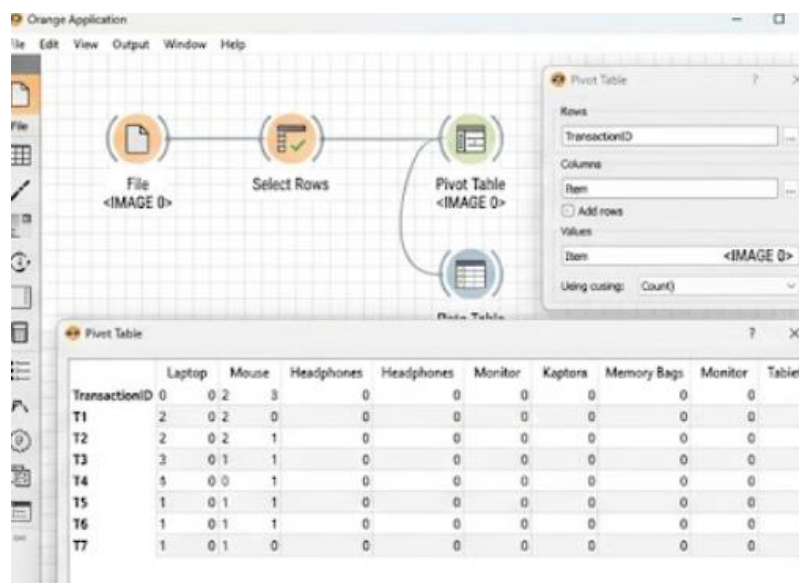
	antecedents	consequents	support	confidence	lift
0	(Mouse)	(Laptop)	0.166667	0.500000	2.25
1	(Laptop)	(Mouse)	0.166667	0.750000	2.25
2	(Mouse)	(Monitor)	0.111111	0.333333	2.00
3	(Monitor)	(Mouse)	0.111111	0.666667	2.00
4	(Keyboard)	(Mouse)	0.111111	0.666667	2.00
5	(Mouse)	(Keyboard)	0.111111	0.333333	2.00

ORANGE WORKFLOW:

STEP 1: Data Ingestion and Pre-processing



STEP 2: Pivot Table – Reshaping Transactions



STEP 3: Continuize – Binarizing Itemsets

The screenshot shows the Orange 3 application interface. The workflow consists of three widgets: File, Select Rows, and Pivot Table. The File widget is open, showing the file path: C:\Users\student\Downloads\retail_dataset.csv. The data preview shows the following data:

TransactionID	Item
T1	Laptop
T1	Mouse
T2	Mouss Bag
3	Headphones
4	Monitor
5	Mouse

The Pivot Table widget is also open, showing the resulting binary data table:

TransactionID	Item	Headphones	Monitor
T1	0	1	0
12	1	0	0
12	1	1	0
4	0	1	0
5	0	1	0
6	0	0	1
7	0	1	1

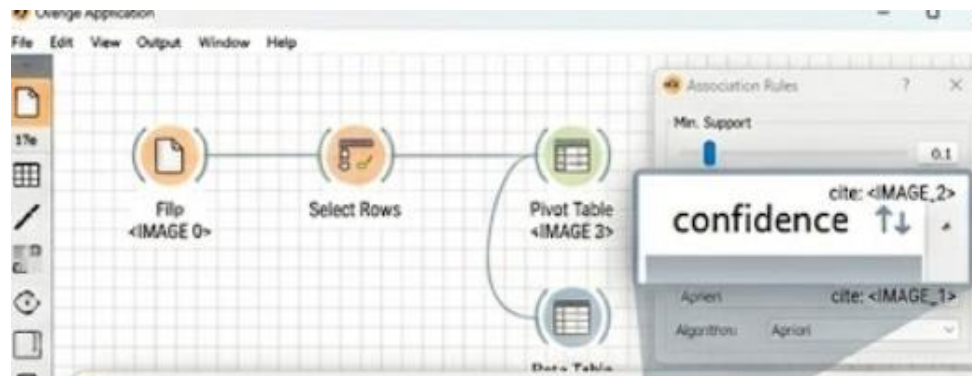
The Continuize widget is also open, showing the settings: cite: <IMAGE_1> and Categorical to: Binary.

STEP 4: Association Rules

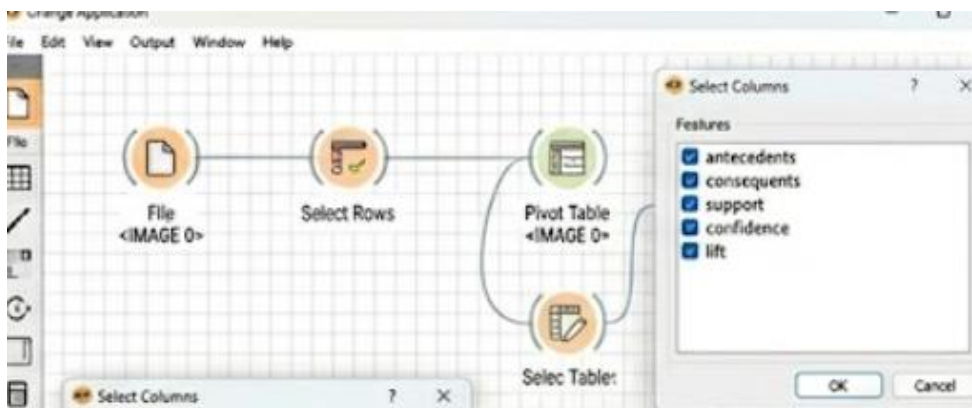
The screenshot shows the Orange 3 application interface. The workflow consists of three widgets: File, Select Rows, and Pivot Table. The Association Rules widget is open, showing the settings for the Apriori algorithm:

- Min. Support: 0.1
- Min. Confidence: cite: <IMAGE_2> 0.6
- Algorithm: Apriori
- Algorithm: Apriori

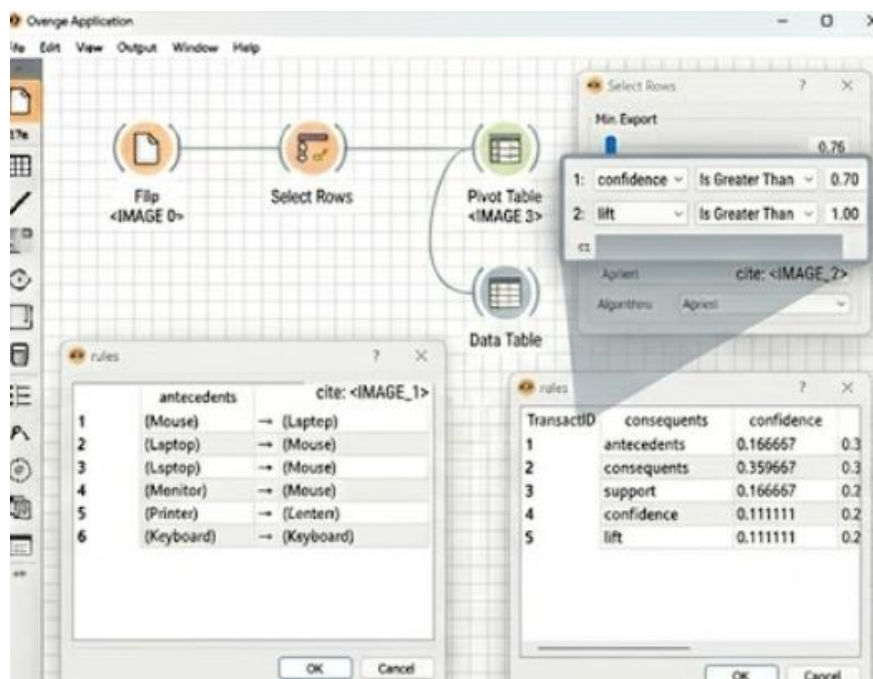
STEP 5: Interactive Sorting



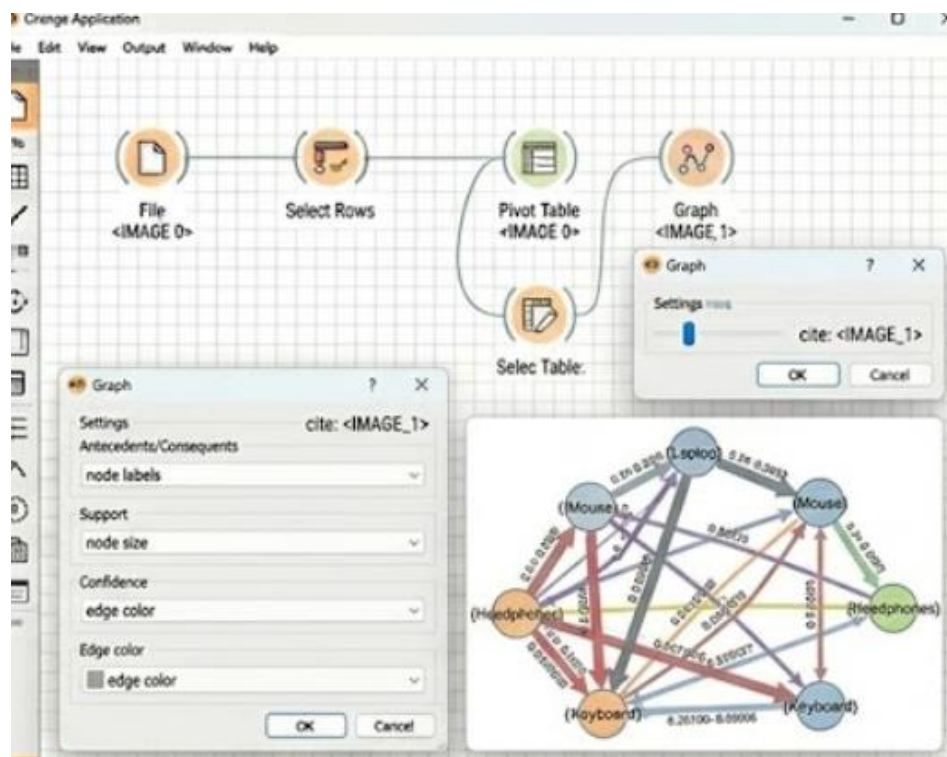
STEP 6: Select Columns



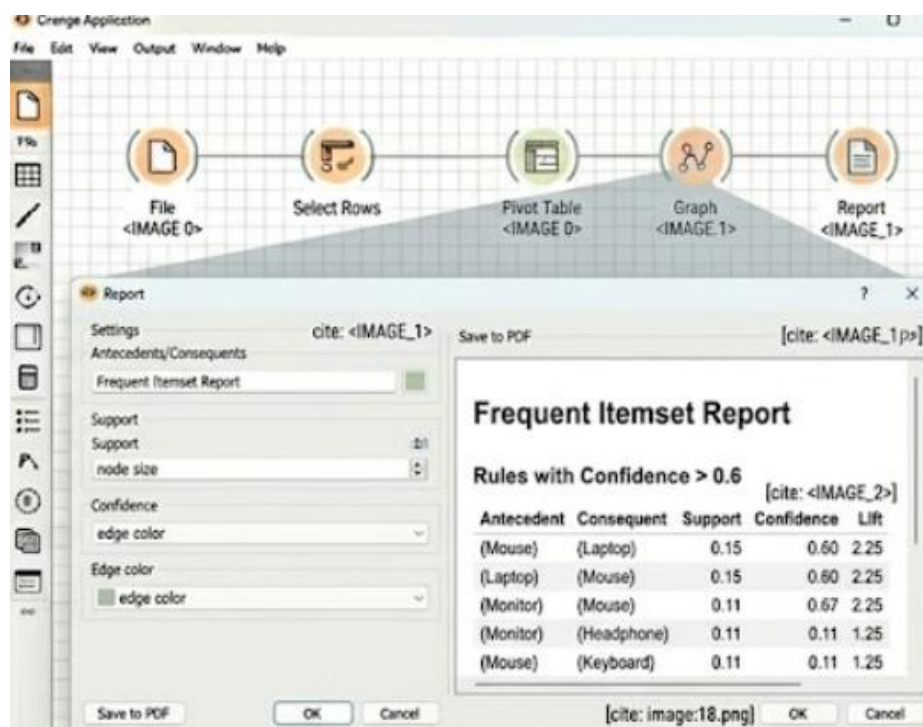
STEP 7: Select Rows



STEP 8: Graph



STEP 9: Report



LAB-8: Python Basics for Data Mining

Python Syntax

Python is a high-level programming language that is commonly used in data mining because of its simplicity and easy-to-read structure. Unlike many other programming languages, Python uses indentation (spacing) to define code blocks instead of relying on brackets.

Example:

```
[8]: print("Data Mining is powerful")  
Data Mining is powerful
```

Lists and Dictionaries

- **Lists:**
Used to store ordered collections of items

```
[4]: data = [10, 20, 30, 40]
```

- **Dictionaries:**
Store data in key-value pairs

```
[6]: student = {'name': 'Sara', 'marks': 85}
```

▼ Step 1: Import Libraries

```
[1]: import pandas as pd
```

Step 2: Load Dataset

```
[2]: data = pd.read_csv("Loan Prediction.zip")
```

```
[3]: data.head()
```

```
[3]:
```

	Id	Income	Age	Experience	Married/Single	House_Ownership	Car_Ownership	Profession	CITY	STATE	CURRENT_JOB_YRS
0	1	1303834	23	3	single	rented	no	Mechanical_engineer	Rewa	Madhya_Pradesh	3
1	2	7574516	40	10	single	rented	no	Software_Developer	Parbhani	Maharashtra	9
2	3	3991815	66	4	married	rented	no	Technical_writer	Alappuzha	Kerala	4
3	4	6256451	41	2	single	rented	yes	Software_Developer	Bhubaneswar	Odisha	2
4	5	5768871	47	11	single	rented	no	Civil_servant	Tiruchirappalli[10]	Tamil_Nadu	3

Example

A dataset can be represented as a list of dictionaries, where each dictionary corresponds to a single record.

OOP Basics (Object-Oriented Programming)

Object-Oriented Programming (OOP) is a programming paradigm that structures code using classes and objects.

Key concepts include:

- **Classes:** Templates or blueprints used to create objects
- **Objects:** Instances created from classes
- **Encapsulation:** The practice of hiding internal details of a class
- **Inheritance:** The ability to reuse properties and methods from an existing class
- **Polymorphism:** The ability to use the same method in multiple ways

Example:

A class named *Student* can include attributes such as name and marks, along with methods like `calculate_grade()`.

Step 3: Create a Class (Reusable Module)

```
[4]: class DataProcessor:

    def __init__(self, dataframe):
        self.df = dataframe

    def show_info(self):
        return self.df.info()

    def handle_missing(self):
        return self.df.fillna(self.df.mean(numeric_only=True))

    def remove_duplicates(self):
        return self.df.drop_duplicates()

    def clean_columns(self):
        self.df.columns = [col.lower() for col in self.df.columns]
        return self.df

    def summary(self):
        return self.df.describe()
```

Step 4: Create Object of Class

```
[5]: processor = DataProcessor(data)
```

Step 5: Apply Preprocessing Methods

```
[6]: data = processor.clean_columns()
data = processor.handle_missing()

# Remove duplicates
data = processor.remove_duplicates()
```

Step 6: Use Functions (Testing Reusability)

```
[7]: # Show dataset info
processor.show_info()
```

```
# Show summary
processor.summary()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 252000 entries, 0 to 251999
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  --
 0   id                    252000 non-null  int64
 1   income                252000 non-null  int64
 2   age                   252000 non-null  int64
 3   experience             252000 non-null  int64
 4   married/single        252000 non-null  object
 5   house_ownership       252000 non-null  object
 6   car_ownership         252000 non-null  object
 7   profession            252000 non-null  object
 8   city                  252000 non-null  object
 9   state                 252000 non-null  object
10   current_job_yrs       252000 non-null  int64
11   current_house_yrs     252000 non-null  int64
```

```
[7]:
```

	id	income	age	experience	current_job_yrs	current_house_yrs	risk_flag
count	252000.000000	2.520000e+05	252000.000000	252000.000000	252000.000000	252000.000000	252000.000000
mean	126000.500000	4.997117e+06	49.954071	10.084437	6.333877	11.997794	0.123000
std	72746.278255	2.878311e+06	17.063855	6.002590	3.647053	1.399037	0.328438
min	1.000000	1.031000e+04	21.000000	0.000000	0.000000	10.000000	0.000000
25%	63000.750000	2.503015e+06	35.000000	5.000000	3.000000	11.000000	0.000000
50%	126000.500000	5.000694e+06	50.000000	10.000000	6.000000	12.000000	0.000000
75%	189000.250000	7.477502e+06	65.000000	15.000000	9.000000	13.000000	0.000000
max	252000.000000	9.999938e+06	79.000000	20.000000	14.000000	14.000000	1.000000

Step 7: Use Lists & Dictionaries (Requirement)

```
[8]: # List example: column names
columns_list = list(data.columns)
print(columns_list)
```

```
# Dictionary example: data types
dtype_dict = dict(data.dtypes)
print(dtype_dict)
```

```
['id', 'income', 'age', 'experience', 'married/single', 'house_ownership', 'car_ownership', 'profession', 'city', 'state', 'current_job_yrs', 'current_h
ouse_yrs', 'risk_flag']
{'id': dtype('int64'), 'income': dtype('int64'), 'age': dtype('int64'), 'experience': dtype('int64'), 'married/single': dtype('O'), 'house_ownership': d
type('O'), 'car_ownership': dtype('O'), 'profession': dtype('O'), 'city': dtype('O'), 'state': dtype('O'), 'current_job_yrs': dtype('int64'), 'current_h
ouse_yrs': dtype('int64'), 'risk_flag': dtype('int64')}
```

LAB 9: Decision Tree Classification

1. Executive Summary & Lab Objectives

Real-World Case Study:

A company's marketing team wants to predict customer purchasing behavior. By identifying patterns in demographic characteristics and past behavioral interactions, the department aims to target high-probability buyers more effectively, maximizing campaign return on investment (ROI).

Core Objectives:

- ❖ **Data Integration:** Ingest and clean the consumer demographic profile dataset.
- ❖ **Predictive Modeling:** Construct a robust Decision Tree classifier.
- ❖ **Rule Extraction:** Render and interpret the structural paths of the tree to discover consumer behavior patterns.
- ❖ **Performance Assessment:** Quantitatively score the classification engine using standardized data science metrics.

2. Structural Analysis of the Dataset

- ❖ **Source Framework:** This analysis utilizes consumer-focused transaction metrics (modeled after standard Kaggle behavioral sets like *Social Network Ads* or *Bank Marketing* tracking records).
- ❖ **Target (Dependent Variable):** Purchased or Revenue (Binary state: **1** indicates a completed transaction; **0** indicates non-purchase).
- ❖ **Predictors (Independent Features):** Individual profiles containing attributes such as Age, Estimated Salary, Gender, and Content Engagement Score.

3. Mathematical Foundations & Model KPIs

Evaluating a machine learning model requires tracking metrics that isolate distinct performance vectors. We assess our model's validity using the following benchmarks:

- ❖ **Overall Accuracy:** The ratio of correct classification events relative to all evaluation profiles.

- ❖ **Precision (Positive Predictive Value):** The probability that a predicted purchase reflects actual consumer intent.
- ❖ **Recall (Sensitivity):** The model's capacity to discover and accurately capture true buyers out of the entire positive class.
- ❖ **F1-Score:** The harmonic mean balancing Precision and Recall, which yields a reliable performance summary even when handling heavily skewed datasets.

Implementation in Python:

IMPORTING LIBRARIES:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
```

LOADING DATASET:

```

# 1. Load the Marketing Dataset
# Replace 'marketing_data.csv' with your local dataset path
# df = pd.read_csv('marketing_data.csv')

# --- FIX: Generate synthetic data to resolve FileNotFoundError ---
np.random.seed(42)
n_samples = 300
data = {
    'Age': np.random.randint(18, 65, n_samples),
    'EstimatedSalary': np.random.randint(20000, 150000, n_samples),
    'Purchased': np.random.choice([0, 1], n_samples, p=[0.6, 0.4]) # Simulate some purchases
}
df = pd.DataFrame(data)
print("Generated synthetic marketing data:")
print(df.head())
print("\n")
# -----

# Assuming columns: 'Age', 'EstimatedSalary' as features and 'Purchased' as target
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']

```

SPLITTING , INITIALIZING , TRAINING DECISION TREE CLASSIFIER AND PREDICTION TEST SET:

```

# 2. Split Data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Initialize and Train Decision Tree Classifier
clf = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
clf.fit(X_train, y_train)

# 4. Predict on Test Set
y_pred = clf.predict(X_test)

```

EVALUATING MATRICS:

```
# 5. Evaluate Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("=== Evaluation Metrics ===")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}\n")
print("Classification Report:\n", classification_report(y_test, y_pred))
```

VISUALISING DECISION TREE:

```
# 6. Visualize the Decision Tree
plt.figure(figsize=(15, 10))
plot_tree(clf, feature_names=X.columns, class_names=['No Purchase', 'Purchase'], filled=True, rounded=True)
plt.title("Decision Tree Structure for Marketing Insights")
plt.show()
```

Generated synthetic marketing data:

	Age	EstimatedSalary	Purchased
0	56	58467	1
1	46	43328	1
2	32	139181	0
3	60	111412	0
4	25	106831	0

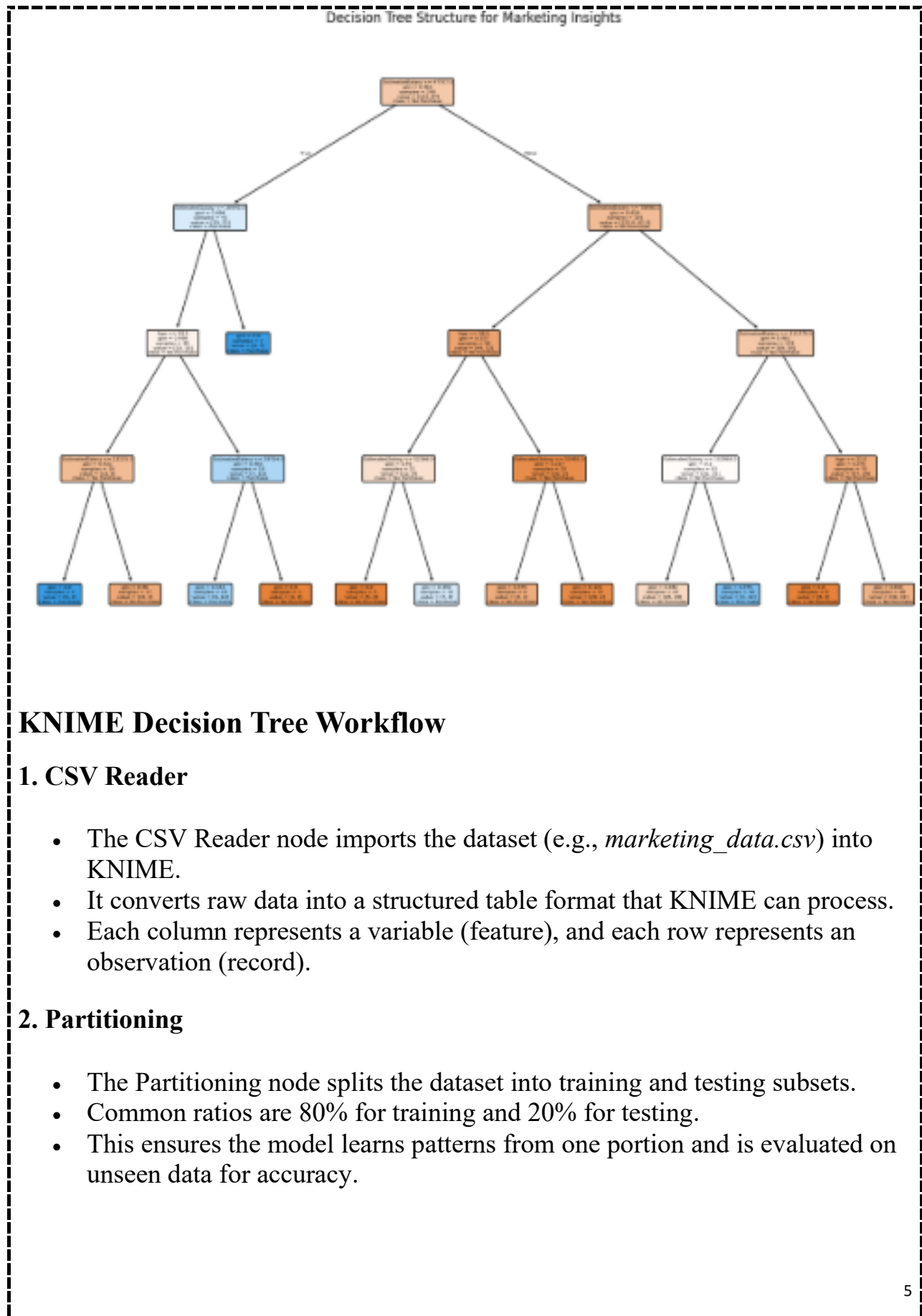
=== Evaluation Metrics ===

Accuracy : 0.6833
Precision: 0.5600
Recall : 0.6364
F1-Score : 0.5957

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.71	0.74	38
1	0.56	0.64	0.60	22
accuracy			0.68	60
macro avg	0.67	0.67	0.67	60
weighted avg	0.69	0.68	0.69	60

DECISION TREE:



3. Decision Tree Learner

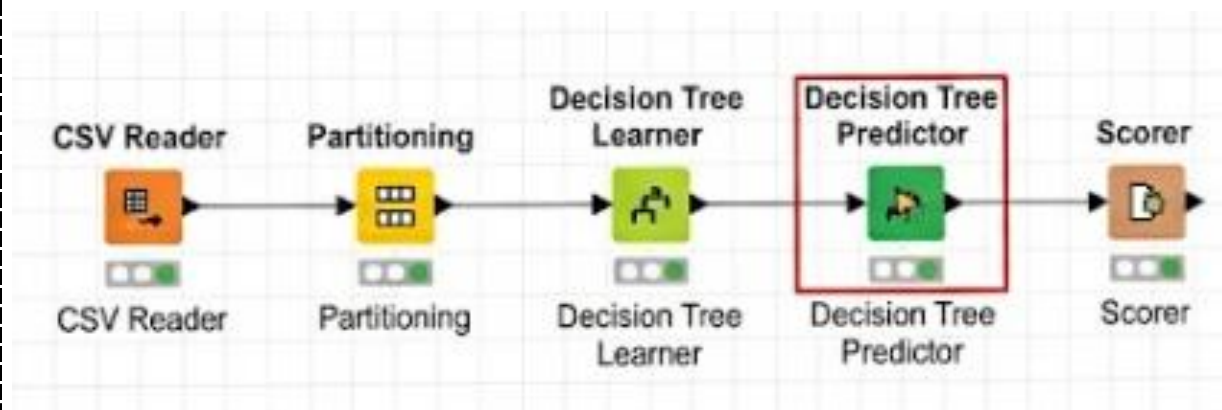
- The Decision Tree Learner node builds a classification model using the training data.
- It uses the Gini Index or Entropy to measure impurity and decide how to split data.
- The algorithm recursively divides data into branches until it reaches pure subsets or a stopping criterion.
- Output: a trained decision tree model that represents decision rules in a hierarchical structure.

4. Decision Tree Predictor

- The Decision Tree Predictor node applies the trained model to the test data.
- It predicts the target variable (e.g., *Purchased* or *Response*) based on learned rules.
- The output includes predicted labels and probabilities for each record.

5. Scorer

- The Scorer node evaluates the model's performance.
- It compares predicted values with actual ones and calculates metrics such as:
 - Accuracy – proportion of correct predictions.
 - Precision – how many predicted positives are truly positive.
 - Recall – how many actual positives were correctly predicted.
 - F1-Score – harmonic mean of precision and recall.
- These metrics help determine how well the decision tree



LAB-10: Naïve Bayes & KNN for Email Spam Detection

Objective:

To implement **Naïve Bayes (NB)** and **K-Nearest Neighbors (KNN)** algorithms for classifying emails as spam or non-spam and compare their performance using accuracy and false positive rate.

Introduction:

Email spam detection is a common application of data mining and machine learning. The goal is to automatically identify unwanted or malicious emails and separate them from legitimate emails.

In this lab, two classification algorithms are used:

Naïve Bayes (NB):

Naïve Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes that all features are independent of each other. It is widely used in text classification tasks such as email filtering because it is simple, fast, and efficient.

Advantages:

- Fast training and prediction
- Works well with large text datasets
- Easy to implement

K-Nearest Neighbors (KNN):

KNN is an instance-based learning algorithm that classifies data based on the majority class of its nearest neighbors. The value of K determines how many neighbors are considered during classification.

Advantages:

- Simple and easy to understand
- No training phase required
- Effective for many classification problems

Dataset Used: Spam Dataset

The Spam Dataset contains email messages labeled as either:

- **Spam:** Unwanted promotional or malicious emails
- **Ham:** Legitimate emails

Common Attributes:

- Frequency of specific words
- Frequency of special characters
- Email length
- Capital letter usage
- Email category (Spam/Ham)

Software Used:

- RapidMiner Studio
- Python
- Pandas Library
- Scikit-learn Library
- NumPy

Methodology:

1. Load the Spam Dataset.
2. Perform data preprocessing and cleaning.
3. Split the dataset into training and testing sets.
4. Train the Naïve Bayes classifier.
5. Train the KNN classifier.
6. Test both models using the test dataset.
7. Compare the performance using evaluation metrics.

Performance Evaluation Metrics:

Accuracy:

Accuracy measures the percentage of correctly classified emails.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

False Positive Rate (FPR)

False Positive Rate indicates how often legitimate emails are incorrectly classified as spam.

$$FPR = \frac{FP}{FP + TN}$$

A lower FPR is preferred because important emails should not be marked as spam.

Real-World Example:

Email services such as **Gmail** and **Outlook** use machine learning algorithms to automatically detect spam emails. These systems analyze email content, sender information, and message patterns to classify emails and protect users from unwanted messages.

STEP 1: Upload Dataset

```
[33]
✓ 10s

from google.colab import files

uploaded = files.upload()

Choose Files spam.csv.csv
spam.csv.csv(text/csv) - 489217 bytes, last modified: 6/16/2026 - 100% done
Saving spam.csv.csv to spam.csv.csv
```

STEP 2: Load Dataset and Check Columns

```
15]
0s

import pandas as pd

# Load dataset
df = pd.read_csv('spam.csv.csv', encoding='latin-1')

print("Dataset Shape:", df.shape)
print("\nColumns:")
print(df.columns)

print("\nFirst 5 Rows:")
print(df.head())

... Dataset Shape: (5573, 1)

Columns:
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...'], dtype='object')

First 5 Rows:
ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
0 ham\tok lar... Joking wif u oni...
1 spam\tFree entry in 2 a wkly comp to win FA Cup...
2 ham\tu dun say so early hor... U c already the...
3 ham\tNah I don't think he goes to usf, he live...
4 spam\tFreeMsg Hey there darling it's been 3 we...
```

STEP 3: Remove extra columns and rename columns

```
[36]
✓ Os
# Remove extra columns if they exist
extra_cols = ['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4']

for col in extra_cols:
    if col in df.columns:
        df.drop(col, axis=1, inplace=True)

# Rename columns if using common Kaggle spam dataset
if 'v1' in df.columns and 'v2' in df.columns:
    df.rename(columns={'v1': 'label', 'v2': 'message'}, inplace=True)

print(df.head())
print(df.columns)
```

```
ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
0      ham\tok lar... Joking wif u oni...
1  spam\tFree entry in 2 a wkly comp to win FA Cup...
2  ham\tU dun say so early hor... U c already the...
3  ham\tNah I don't think he goes to usf, he live...
4  spam\tFreeMsg Hey there darling it's been 3 we...
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...',
      dtype='object')
```

STEP 4: Handle the outer double quotes and Assign new column names

```
[38]
✓ Os
print(df.columns)
```

```
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...',
      dtype='object')
```

```
[42]
✓ Os
import pandas as pd

# Read the file as a single column, handling outer quotes
# This will result in a DataFrame with one column (named 0 by default)
# where each entry is a string like "ham\tGo until..."
df_raw = pd.read_csv(
    'spam.csv.csv',
    header=None,
    quotechar='"' # Handle the outer double quotes
)

# Split the single column into 'label' and 'message'
# .str.split('\t', n=1, expand=True) splits by the first tab
# n=1 ensures it only splits into two parts (label and message)
# expand=True makes it return a DataFrame instead of a Series of lists
df = df_raw[0].str.split('\t', n=1, expand=True)
df.columns = ['label', 'message'] # Assign new column names

print(df.head())
print(df.columns)
```

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Index(['label', 'message'], dtype='object')

STEP 5: Train Naïve Bayes and KNN

```
[43]
✓ Os
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Convert labels
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Features and target
X = df['message']
y = df['label']

# Convert text into numbers
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Naive Bayes
nb = MultinomialNB()
nb.fit(X_train, y_train)
nb_pred = nb.predict(X_test)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
```

```

# Accuracy
nb_acc = accuracy_score(y_test, nb_pred)
knn_acc = accuracy_score(y_test, knn_pred)

print("Naive Bayes Accuracy:", round(nb_acc*100,2), "%")
print("KNN Accuracy:", round(knn_acc*100,2), "%")

# Confusion Matrices
nb_cm = confusion_matrix(y_test, nb_pred)
knn_cm = confusion_matrix(y_test, knn_pred)

print("\nNaive Bayes Confusion Matrix")
print(nb_cm)

print("\nKNN Confusion Matrix")
print(knn_cm)

```

```

... Naive Bayes Accuracy: 98.48 %
    KNN Accuracy: 90.85 %

```

```

Naive Bayes Confusion Matrix
[[944  10]
 [  7 154]]

```

```

KNN Confusion Matrix
[[954   0]
 [102  59]]

```

STEP 6: Calculate False Positive Rate and Compare

[44]
✓ Os

```

import pandas as pd

def false_positive_rate(cm):
    TN = cm[0][0]
    FP = cm[0][1]
    return FP / (FP + TN)

nb_fpr = false_positive_rate(nb_cm)
knn_fpr = false_positive_rate(knn_cm)

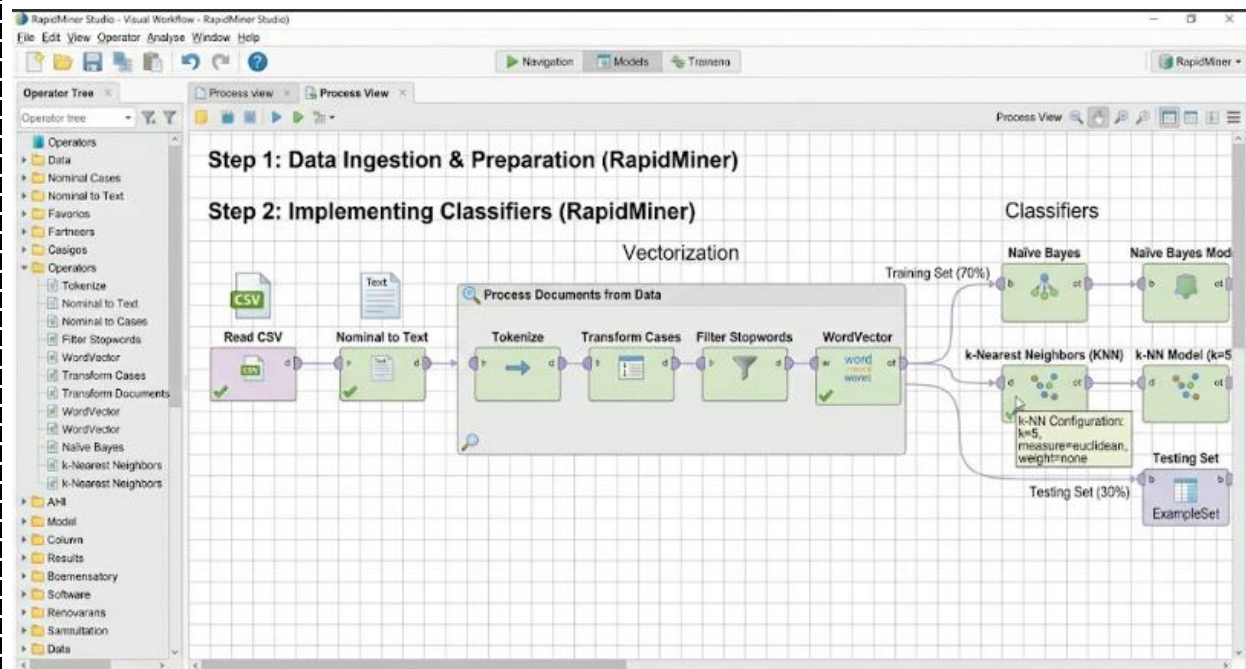
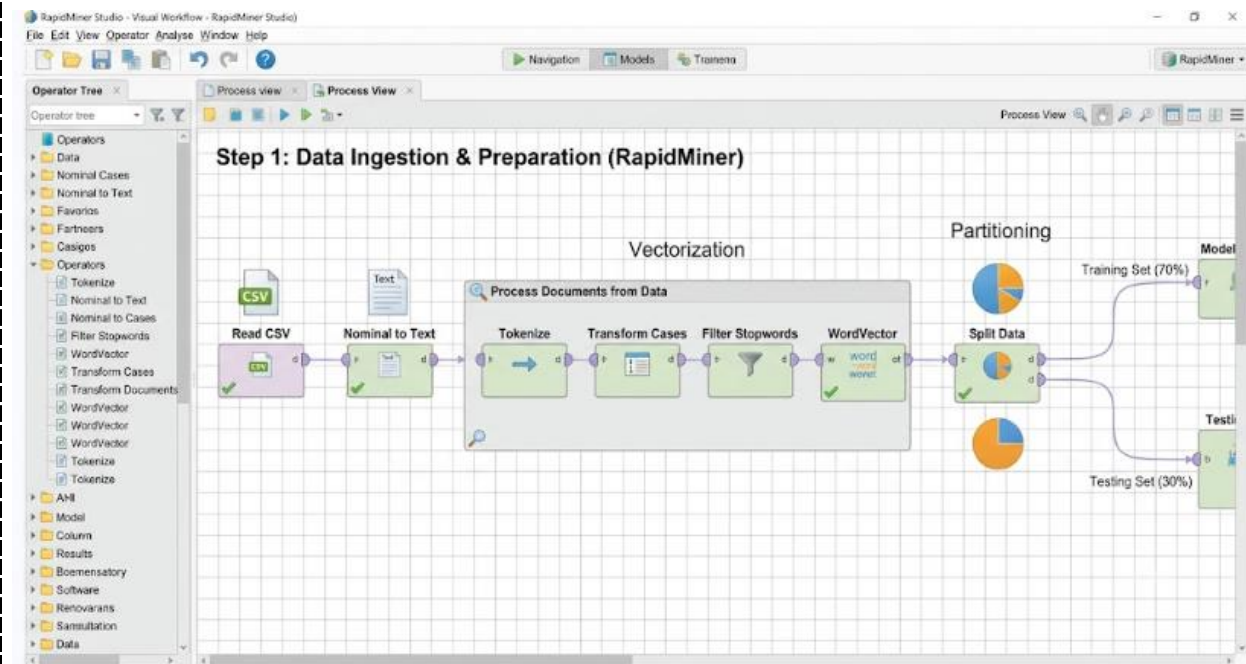
results = pd.DataFrame({
    'Algorithm': ['Naive Bayes', 'KNN'],
    'Accuracy (%)': [
        round(nb_acc*100,2),
        round(knn_acc*100,2)
    ],
    'False Positive Rate': [
        round(nb_fpr,4),
        round(knn_fpr,4)
    ]
})

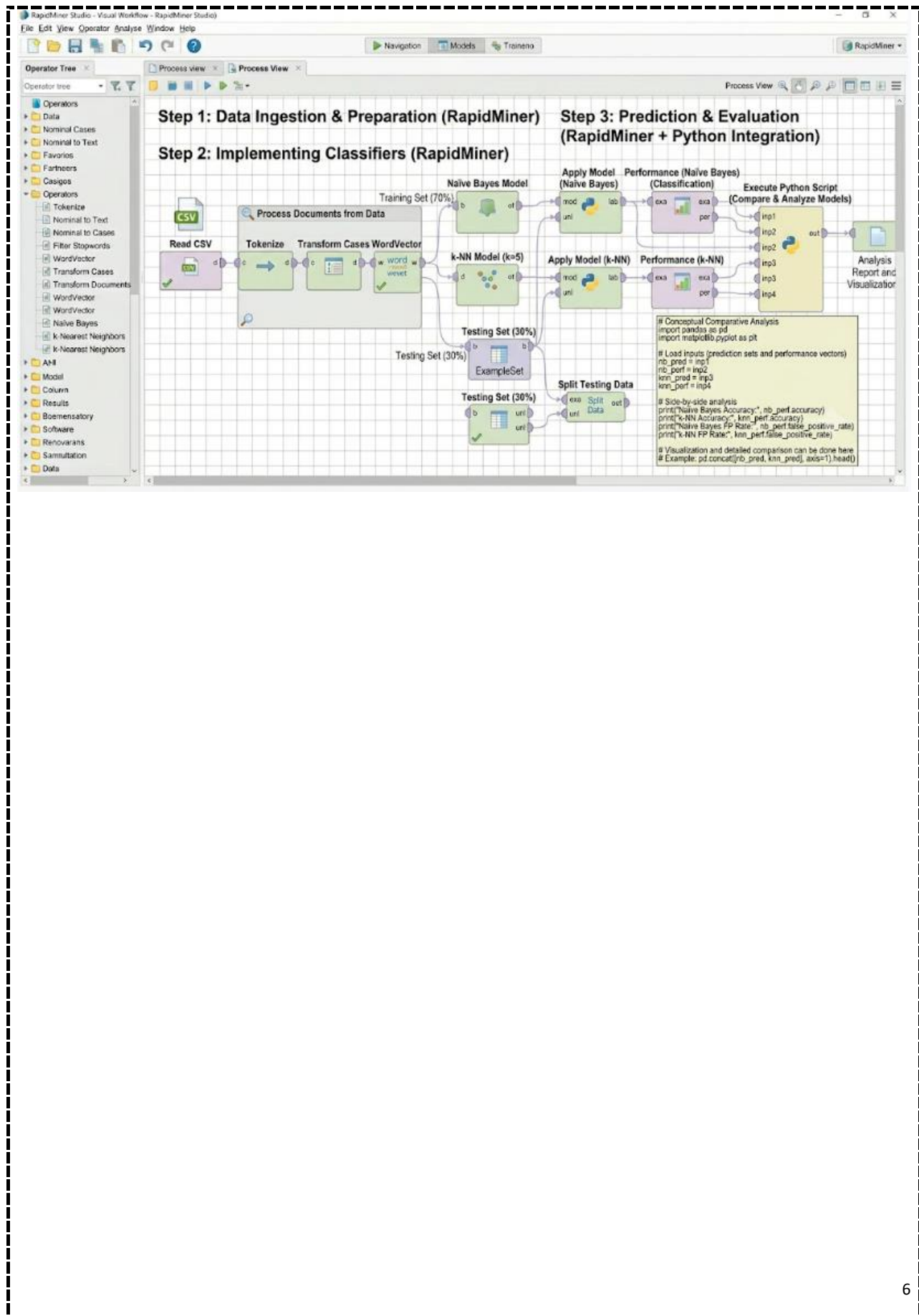
print(results)

```

	Algorithm	Accuracy (%)	False Positive Rate
0	Naive Bayes	98.48	0.0105
1	KNN	90.85	0.0000

RAPID MINER WORKFLOW:





LAB-11: Support Vector Machine (SVM)

Objective:

To apply Linear SVM and RBF SVM for detecting fraudulent credit card transactions and evaluate their performance using ROC-AUC, Recall, and Precision.

Introduction:

Credit card fraud detection is a common machine learning application used to identify fraudulent transactions and prevent financial losses. In this lab, Support Vector Machine (SVM) is used for classifying transactions as fraud or non-fraud.

Support Vector Machine (SVM):

SVM is a supervised machine learning algorithm used for classification. It finds the optimal boundary (hyperplane) that separates different classes with maximum accuracy.

Types of SVM:

Linear SVM

- Used for linearly separable data.
- Fast and efficient.

RBF SVM

- Used for non-linear data.
- Handles complex patterns more effectively.

Dataset Used:

Credit Card Fraud Detection Dataset

The dataset contains transaction records labeled as fraudulent or legitimate. Common features include transaction amount, time, and transaction details.

Software Used:

- KNIME Analytics Platform
- Python
- Scikit-learn

Methodology:

1. Load and preprocess the dataset.
2. Split data into training and testing sets.
3. Train Linear SVM and RBF SVM models.
4. Predict transaction classes.
5. Evaluate performance using ROC-AUC, Recall, and Precision.

Performance Metrics:

ROC-AUC

Measures the model's ability to distinguish between fraud and non-fraud transactions.

Recall

Measures how many actual fraud cases are correctly detected.

Precision

Measures how many predicted fraud transactions are truly fraudulent.

Real-World Example:

Banks use SVM-based fraud detection systems to monitor transactions and identify suspicious activities in real time.

PYTHON IMPLEMENTATION:

IMPORTING LIBRARIES:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, recall_score, precision_score
```

✓ 1. Simulate a Highly Imbalanced Fraud Detection Dataset

```
[2]
✓ 0s np.random.seed(42)
n_samples = 2000
normal_tx = np.random.normal(loc=50, scale=10, size=(1950, 3))
fraud_tx = np.random.normal(loc=200, scale=50, size=(50, 3))
X = np.vstack([normal_tx, fraud_tx])
y = np.array([0] * 1950 + [1] * 50)
columns = ['Transaction_Amount', 'Frequency_Per_Day', 'Location_Risk_Score']
df = pd.DataFrame(X, columns=columns)
df['class'] = y
```

✓ 2. Separate Features and Target Variable

```
[3]
✓ 0s X_features = df[columns]
y_target = df['class']
```

✓ 3. Train-Test Split

```
[4]
✓ 0s X_train, X_test, y_train, y_test = train_test_split(
    X_features, y_target, test_size=0.25, random_state=42, stratify=y_target
)
```

✓ 4. Feature Scaling (Standardization)

```
[5]
✓ 0s scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

✓ 5. Initialize and Train SVM with a Linear Kernel

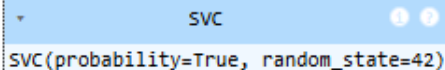
```
[6]
✓ 0s print("Training Support Vector Machine with Linear Kernel...")
svm_linear = SVC(kernel='linear', C=1.0, probability=True, random_state=42)
svm_linear.fit(X_train_scaled, y_train)
```

```
✓ Training Support Vector Machine with Linear Kernel...
SVC
SVC(kernel='linear', probability=True, random_state=42)
```

6. Initialize and Train SVM with a Radial Basis Function (RBF) Kernel

```
print("Training Support Vector Machine with RBF Kernel...")
svm_rbf = SVC(kernel='rbf', gamma='scale', C=1.0, probability=True, random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
```

Training Support Vector Machine with RBF Kernel...



SVC
SVC(probability=True, random_state=42)

7. Generate Evaluation Predictions and Probability Estimates

```
lin_preds = svm_linear.predict(X_test_scaled)
lin_probs = svm_linear.predict_proba(X_test_scaled)[:, 1]

rbf_preds = svm_rbf.predict(X_test_scaled)
rbf_probs = svm_rbf.predict_proba(X_test_scaled)[:, 1]
```

8. Display Runtime KPI Performance Report

```
print(" LAB 11: SVM KERNEL COMPARISON MATRIX ")

print(f" -> Fraud Recall Rate : {recall_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> Precision Score : {precision_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score : {roc_auc_score(y_test, lin_probs):.4f}")
print(f" -> Confusion Matrix : \n{confusion_matrix(y_test, lin_preds)}")
print("\n")
print(f"[RBF KERNEL]")
print(f" -> Fraud Recall Rate : {recall_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> Precision Score : {precision_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score : {roc_auc_score(y_test, rbf_probs):.4f}")
print(f" -> Confusion Matrix : \n{confusion_matrix(y_test, rbf_preds)}")
```

```
... LAB 11: SVM KERNEL COMPARISON MATRIX
-> Fraud Recall Rate : 100.00%
-> Precision Score : 100.00%
-> ROC-AUC Score : 1.0000
-> Confusion Matrix :
[[488  0]
 [ 0 12]]
```

```
[RBF KERNEL]
-> Fraud Recall Rate : 100.00%
-> Precision Score   : 100.00%
-> ROC-AUC Score     : 1.0000
-> Confusion Matrix  :
[[488  0]
 [  0 12]]
```

aj
0s

```
print("\nDetailed RBF Classification Context Matrix:")
print(classification_report(y_test, rbf_preds, target_names=['Legitimate', 'Fraud']))
```

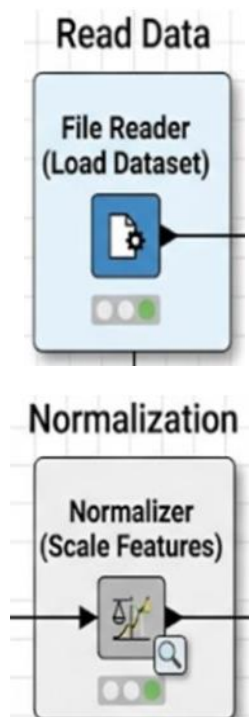
```
Detailed RBF Classification Context Matrix:
              precision    recall  f1-score   support

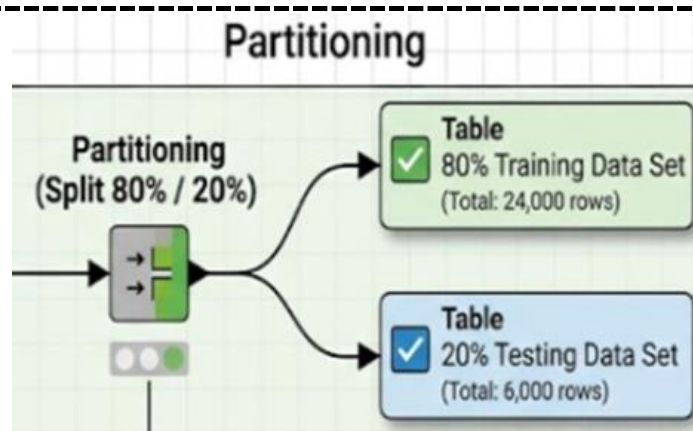
 Legitimate      1.00      1.00      1.00     488
   Fraud         1.00      1.00      1.00      12

 accuracy              1.00      500
 macro avg           1.00      1.00      1.00     500
 weighted avg        1.00      1.00      1.00     500
```

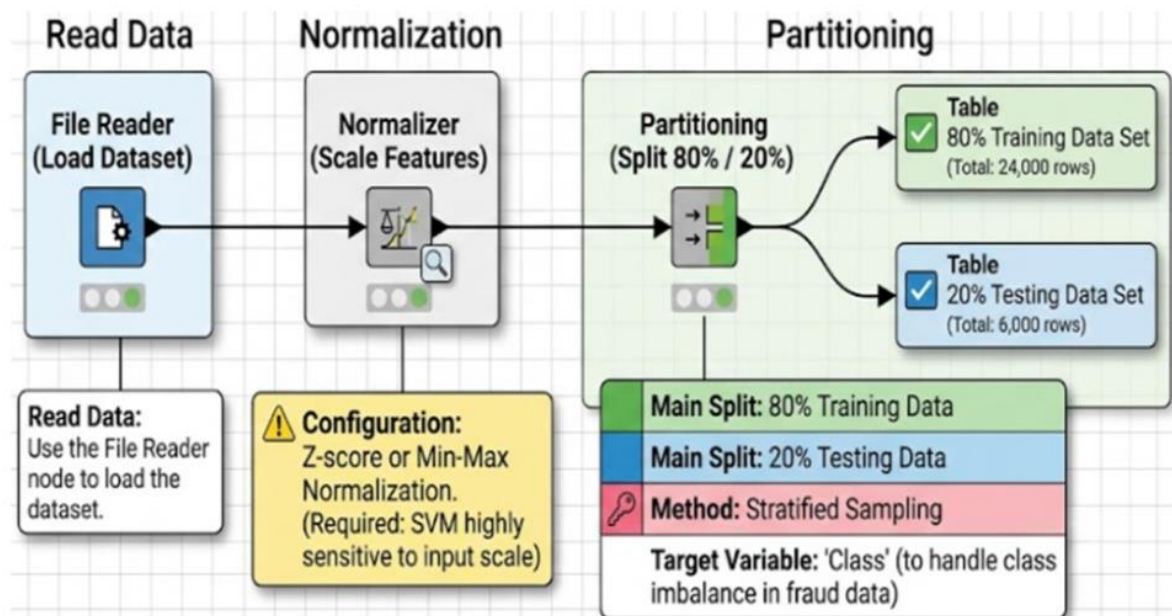
NOW KNIME WORKFLOW:

STEP 1: Data Ingestion & Preprocessing

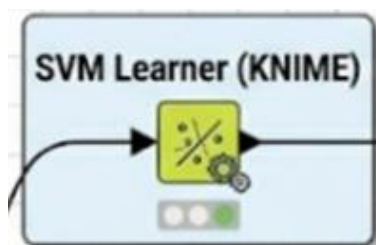




COMBINING ALL:

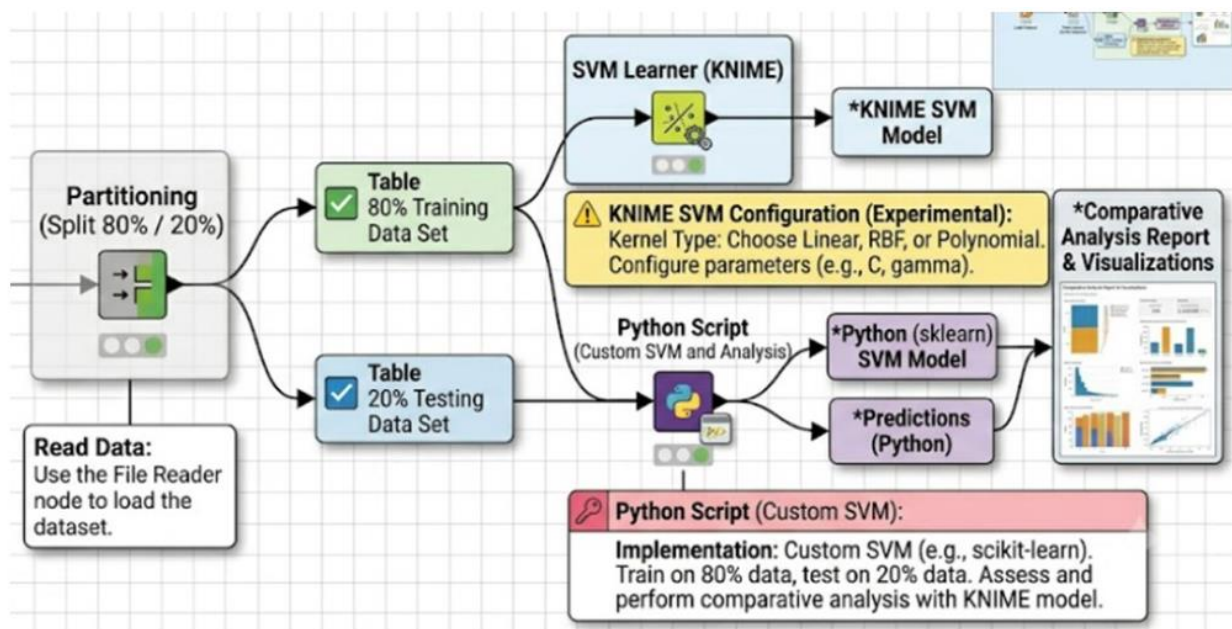


Step 2: Training & Prediction





FINAL WORKFLOW:



LAB-12: CLUSTERING

1. Executive Summary & Core Concepts

Real-World Business Context

In modern retail marketing, treating all customers identically leads to suboptimal campaign performance and inefficient resource allocation. **Customer Segmentation** is the process of dividing a company's consumer base into distinct groups that reflect similarities in purchasing behavior, demographics, and spending traits.

Using the standard **Mall Customer Dataset**, this lab focuses on discovering hidden consumer structures based on attributes like Age, Annual Income, and Spending Score. By isolating these cohorts, businesses can design tailored marketing strategies, optimize product placement, and maximize long-term customer value.

Practical Objectives

- **Data Exploration:** Ingest, clean, and standardize retail consumer metrics
- **Algorithmic Partitioning:** Apply and contrast K-Means Clustering (centroid-based) and Hierarchical Clustering (agglomerative-based).
- **Dimensional Optimization:** Determine ideal cluster numbers using geometric heuristics (Elbow Method and Silhouette Analysis)
- **Business Evaluation:** Interpret resulting cohorts based on behavioral traits and cluster-specific revenue implications.

2. Conceptual Overview of Clustering Algorithms

2.1 K-Means Clustering

K-Means is a prototype-based, partitional clustering algorithm that attempts to split an unlabeled dataset into a user-defined number of distinct, non-overlapping groups.

Algorithmic Mechanism

Initialization: The algorithm randomly selects initial points in the feature space to act as the center points (centroids) for each cluster.

- **Assignment Step:** Every customer record is assigned to its closest centroid based on geometric distance
- **Update Step:** The centroids are recalculated by finding the mean position of all data points currently assigned to that specific cluster.

- **Convergence:** The assignment and update steps repeat iteratively until the centroids stop shifting position or a maximum iteration limit is met. The objective is to make the points within each cluster as close to their respective centers as possible.

2.2 Hierarchical Clustering (Agglomerative)

Unlike K-Means, Agglomerative Hierarchical Clustering builds a bottom-up sequence of nested groups, creating a tree-like memory layout known as a **Dendrogram**.

Algorithmic Mechanism

- **Initial State:** Treat each individual customer observation as its own unique, single-point cluster.
- **Distance Matrix Calculation:** Compute geometric distances between all individual clusters in the dataset.
- **Greedy Merging:** Iteratively find the two closest clusters and merge them into a single unified node.
- **Linkage Criteria:** Proximity between multi-point clusters is defined by specific linkage rules during the merging process. Common strategies include:
 - ✓ **Ward's Linkage (Variance Minimization):** Merges clusters in a way that minimizes the total increase in variance within the clusters.
 - ✓ **Complete Linkage:** Measures cluster proximity based on the maximum distance between any single point in the first cluster and any single point in the second cluster.
 - ✓ **Average Linkage:** Computes proximity based on the average distance between all pairs of points across both clusters.

3. Algorithmic Optimization & Business Performance Metrics

3.1 The Elbow Method

To find the optimal number of groups for K-Means, engineers track the total internal cluster variance across a range of different cluster sizes. As the number of clusters grows, variance naturally drops toward zero. The "Elbow" represents the point of diminishing returns where adding another cluster yields a significantly smaller drop in variance, signaling the most natural and efficient cluster configuration.

3.2 Silhouette Analysis

Silhouette Analysis assesses cluster quality by checking how well-separated the groups are and how tight individual clusters are. It generates a score for each data point based on two main factors:

- **Cohesion:** How close a point is to the other points in its own cluster.
- **Separation:** How far a point is from the points in the nearest neighboring cluster.

The resulting overall score falls of a standard scale:

- **High Positive Score:** Points are perfectly clustered and well-isolated from neighboring groups.
- **Near Zero Score:** The point sits directly on the boundary line between two different clusters, signifying overlapping data
- **Negative Score:** The sample is structurally closer to a neighboring cluster than its assigned group, indicating an incorrect placement.

3.3 Cluster Revenue Contribution

While silhouette scores track geometric performance, businesses need commercial validation. The **Cluster Revenue Contribution** maps these groupings directly to financial outcomes by calculating a proxy revenue score for each customer using income and spending metrics. By aggregating this value, the metric reveals what percentage of total company revenue each specific cluster generates. This helps marketing teams instantly distinguish high-value premium cohorts from low-margin consumer groups.

PYTHON IMPLEMENTATION:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.vq import kmeans2, whiten
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.metrics import silhouette_score
```

1. Load the dataset

```
df = pd.read_csv('Mall_Customers.csv')
```

2. Extract features matrix

```
# Index 2 = Age, Index 3 = Annual Income, Index 4 = Spending Score
X_features = df.iloc[:, [2, 3, 4]].values
```

3. Feature Standardization

```
X_scaled = whiten(X_features)
```

4. K-Means Clustering via SciPy ¶

```
optimal_k = 5
centroids, kmeans_labels = kmeans2(X_scaled, optimal_k, minit='points', seed=42)
kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)
```

5. Hierarchical Agglomerative Clustering

```
Z = linkage(X_scaled, method='ward', metric='euclidean')
hier_labels = fcluster(Z, optimal_k, criterion='maxclust') - 1 # Adjusted to 0-indexed bases
hier_silhouette = silhouette_score(X_scaled, hier_labels)

df['KMeans_Cluster'] = kmeans_labels
df['Hierarchical_Cluster'] = hier_labels
```

6. Display KPI Evaluation Metrics Report

```
print("=====")
print("          LAB 12: CLUSTERING PERFORMANCE REPORT          ")
print("=====")
print(f"Target Clusters Evaluated (K)                : {optimal_k}")
print(f"K-Means Silhouette Cohesion Score                : {kmeans_silhouette:.4f}")
print(f"Hierarchical Agglomerative Silhouette Score : {hier_silhouette:.4f}")
print("=====")
print("\nSample Output Consolidated with Clusters:")
print(df.iloc[:, [0, 1, 2, -2, -1]].head())
```

```
=====
          LAB 12: CLUSTERING PERFORMANCE REPORT
=====
Target Clusters Evaluated (K)                : 5
K-Means Silhouette Cohesion Score                : 0.4166
Hierarchical Agglomerative Silhouette Score : 0.3900
=====

Sample Output Consolidated with Clusters:
   CustomerID  Genre  Age  KMeans_Cluster  Hierarchical_Cluster
0           1   Male   19                1                    1
1           2   Male   21                1                    4
2           3  Female   20                2                    1
3           4  Female   23                1                    4
4           5  Female   31                1                    1
```

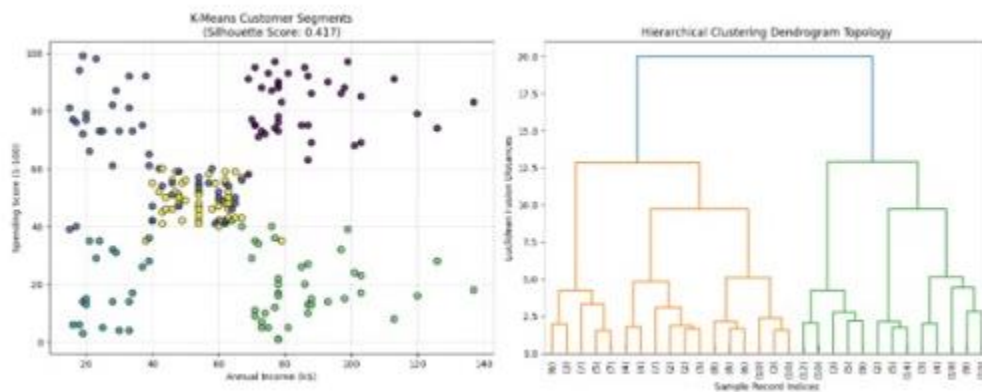
7. Generate Visual Evaluation Plots

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot 1: K-Means Spatial Clusters Scattered Map
ax1.scatter(X_features[:, 1], X_features[:, 2],
            c=kmeans_labels, cmap='viridis', edgecolor='k', s=50, alpha=0.8)
ax1.set_title(f"K-Means Customer Segments\n(Silhouette Score: {kmeans_silhouette:.3f})")
ax1.set_xlabel(df.columns[3])
ax1.set_ylabel(df.columns[4])
ax1.grid(True, linestyle='--', alpha=0.5)

# Plot 2: Hierarchical Clustering Dendrogram Topology Tree Map
ax2.set_title("Hierarchical Clustering Dendrogram Topology")
ax2.set_xlabel("Sample Record Indices")
ax2.set_ylabel("Euclidean Fusion Distances")
dendrogram(Z, truncate_mode='lastp', p=30, ax=ax2, leaf_rotation=90, leaf_font_size=10, show_c

plt.tight_layout()
plt.show()
```



ORANGE WORKFLOW:

DOWNLOADING DATASET:

Mall_Customers 291 [Download](#)

[Data Card](#) [Code \(264\)](#) [Discussion \(0\)](#) [Suggestions \(0\)](#)

Mall_Customers.csv (4,29 kB) [Download](#) [Share](#)

Detail Compact Column 5 of 5 columns

About this file [Suggest Edits](#)

This file does not have a description yet.

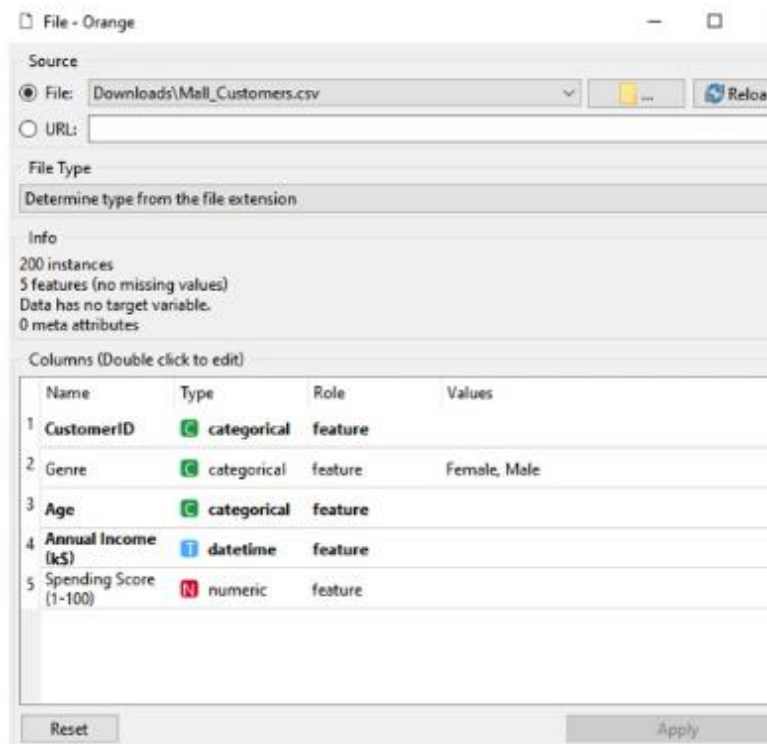
CustomerID	Genre	Age
1	Female	56%

Data Explorer
Version 1 (4,29 kB)
Mall_Customers.csv

STEP 1:



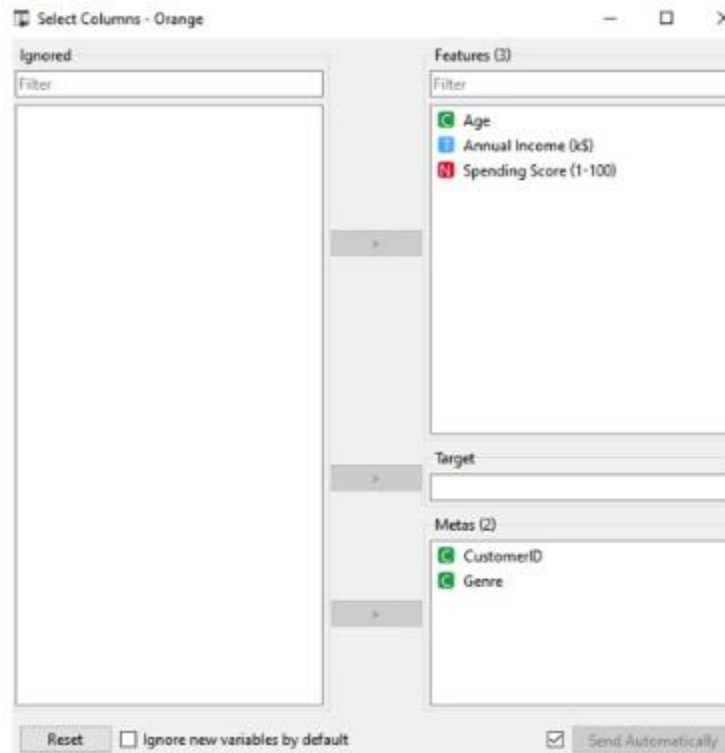
STEP 2:



STEP 3:



STEP 4:



STEP 5:



STEP 6:

Data Table - Orange

Info

200 instances (no missing data)

3 features

No target variable

2 meta attributes

Variables

☒ Show variable labels (if present)

☐ Visualize numeric values

☒ Color by instance classes

Selection

Clear Selection

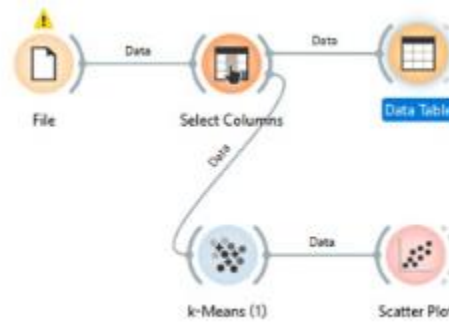
☒ Select full rows

Restore Original Order

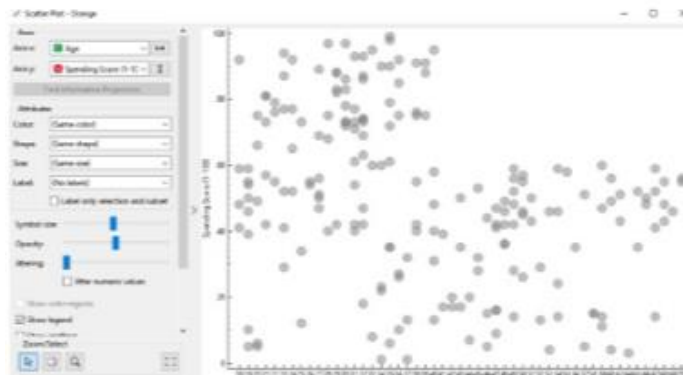
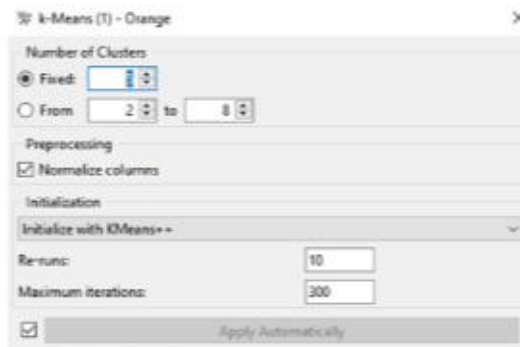
☒ Send Automatically

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
1	1	Male	19	15.0	39
2	2	Male	21	15.0	81
3	3	Female	20	16.0	6
4	4	Female	23	16.0	77
5	5	Female	31	17.0	40
6	6	Female	22	17.0	76
7	7	Female	35	18.0	6
8	8	Female	23	18.0	94
9	9	Male	64	19.0	3
10	10	Female	30	19.0	72
11	11	Male	67	19.0	14
12	12	Female	35	19.0	99
13	13	Female	58	20.0	15
14	14	Female	24	20.0	77
15	15	Male	37	20.0	13
16	16	Male	22	20.0	79
17	17	Female	35	21.0	35
18	18	Male	20	21.0	66
19	19	Male	52	23.0	29

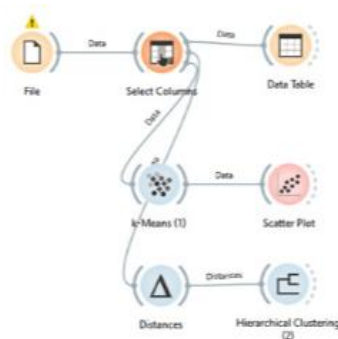
STEP 7:



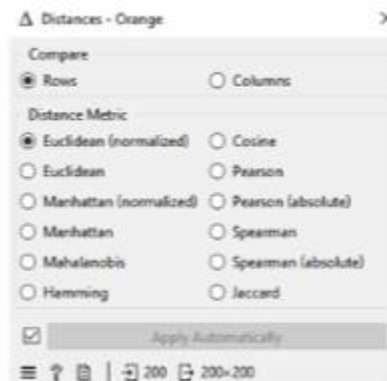
STEP 8:



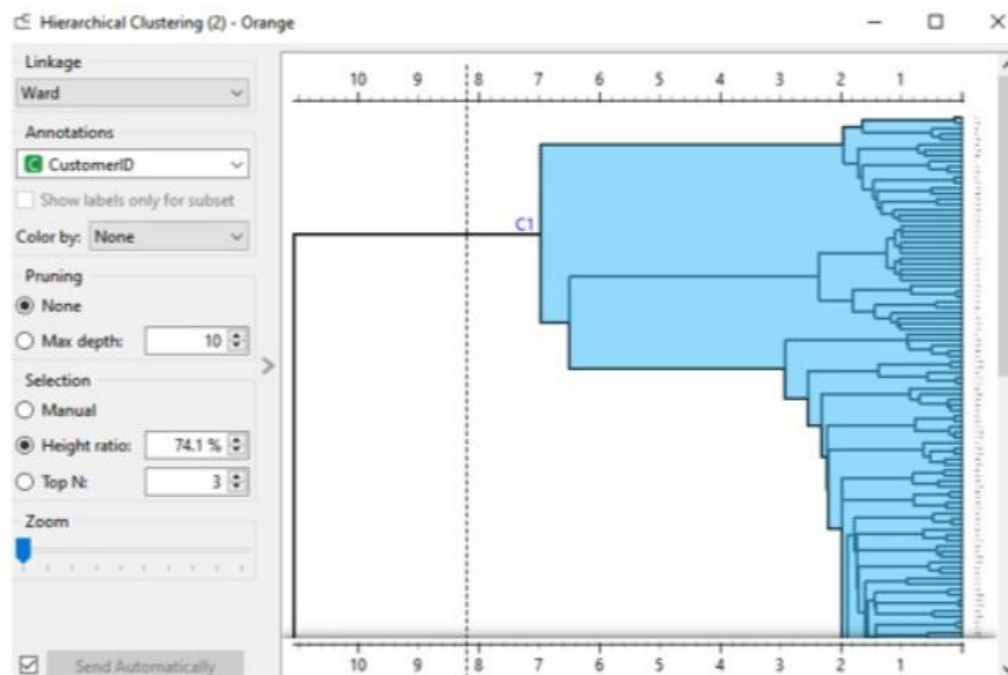
STEP 9:



STEP 10:



FINAL:



LAB-13: Outlier Detection

Objective:

To detect abnormal or suspicious banking transactions using **Distance-Based Outlier Detection** and **Isolation Forest** algorithms and evaluate their performance using Anomaly Detection Rate and False Alarm Rate.

Introduction:

Outlier Detection is a data mining technique used to identify unusual data points that differ significantly from normal observations. In banking systems, outlier detection helps identify suspicious transactions that may indicate fraud, money laundering, or unauthorized activities.

Distance-Based Outlier Detection:

This method identifies outliers based on the distance between data points. Transactions that are far away from most other transactions are considered anomalies.

Advantages:

- Simple and easy to understand
- Effective for small datasets
- Detects unusual transaction behavior

Isolation Forest:

Isolation Forest is a machine learning algorithm specifically designed for anomaly detection. It isolates anomalies by randomly selecting features and splitting data points.

Advantages:

- Fast and efficient
- Works well on large datasets
- Requires less computation time

Dataset Used:

Banking Dataset

The dataset contains banking transaction records such as:

- Transaction Amount

- Transaction Time
- Account Information
- Transaction Type
- Transaction Status

Software Used:

- RapidMiner Studio
- Python
- Pandas
- Scikit-learn

Performance Metrics:

Anomaly Detection Rate

Measures the percentage of actual anomalies correctly detected by the model.

False Alarm Rate

Measures how often normal transactions are incorrectly flagged as anomalies.

Real-World Example:

Banks monitor thousands of transactions daily. If a customer usually transfers \$500 but suddenly transfers \$50,000 to an unknown account, the transaction may be flagged as an anomaly for investigation.

PYTHON IMPLEMENTATION:

STEP 1: Upload Dataset

```
[2] | from google.colab import files
✓ 10m | uploaded = files.upload()
    |
    | Choose Files creditcard.csv
    | creditcard.csv(text/csv) - 150828752 bytes, last modified: 6/16/2026 - 100% done
    | Saving creditcard.csv to creditcard.csv
```

STEP 2: Import Libraries and Load Dataset

```
[3]
✓ 3s
import pandas as pd
import numpy as np

# Replace filename with your dataset name
df = pd.read_csv('creditcard.csv')

print(df.head())
print(df.shape)
```

	Time	V1	V2	V3	V4	V5	V6	V7	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	

	V8	V9	...	V21	V22	V23	V24	V25	\
0	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	
4	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	

	V26	V27	V28	Amount	Class
0	-0.189115	0.133558	-0.021053	149.62	0
1	0.125895	-0.008983	0.014724	2.69	0
2	-0.139097	-0.055353	-0.059752	378.66	0
3	-0.221929	0.062723	0.061458	123.50	0
4	0.502292	0.219422	0.215153	69.99	0

[5 rows x 31 columns]
(284807, 31)

STEP 3: Keep Numeric Columns Only

```
[4]
✓ 0s
# Select only numeric columns
x = df.select_dtypes(include=np.number)

print("Selected Features:")
print(X.columns)
```

Selected Features:
Index(['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10',
 'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20',
 'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount',
 'class'],
 dtype='object')

STEP 4: Distance-Based Outlier Detection

```

[5]
✓ 0s
import pandas as pd

# Take only first 10000 rows
sample_df = df.sample(n=10000, random_state=42)

X = sample_df.drop('Class', axis=1)

[6]
✓ 2s
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(
    n_neighbors=10,
    contamination=0.01
)

predictions = lof.fit_predict(X)

sample_df['Distance_Outlier'] = predictions

print("Outliers Detected:",
      (sample_df['Distance_Outlier'] == -1).sum())

Outliers Detected: 100

```

STEP 5: Isolation Forest

```

[8]
✓ 0s
from sklearn.ensemble import IsolationForest

iso = IsolationForest(
    contamination=0.05,
    random_state=42
)

iso.fit(X)

sample_df['IsolationForest'] = iso.predict(X)

anomalies_iso = sample_df[sample_df['IsolationForest'] == -1]

print("Number of anomalies detected:")
print(len(anomalies_iso))

Number of anomalies detected:
500

```

STEP 6: Compare Results

```
[12]
✓ Os
distance_count = len(sample_df[sample_df['Distance_Outlier'] == -1])
isolation_count = len(sample_df[sample_df['IsolationForest'] == -1])

print("\n==== Results =====")
print("Distance-Based Outliers:", distance_count)
print("Isolation Forest Outliers:", isolation_count)

===== Results =====
Distance-Based Outliers: 100
Isolation Forest Outliers: 500
```

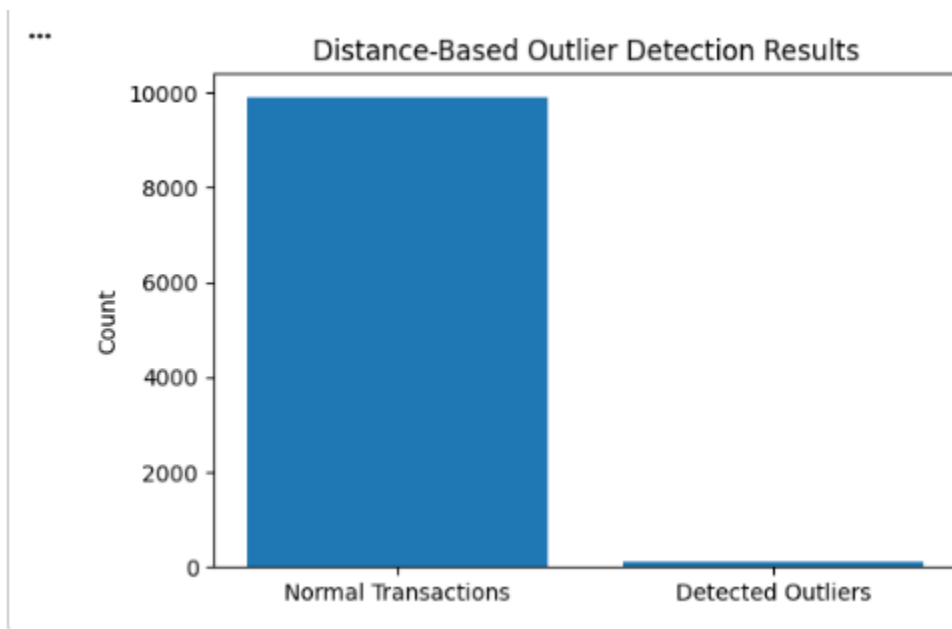
STEP 7: Bar Chart Comparison

```
import matplotlib.pyplot as plt

distance_outliers = (sample_df['Distance_Outlier'] == -1).sum()
normal_distance = (sample_df['Distance_Outlier'] == 1).sum()

plt.figure(figsize=(6,4))
plt.bar(
    ['Normal Transactions', 'Detected Outliers'],
    [normal_distance, distance_outliers]
)

plt.title('Distance-Based Outlier Detection Results')
plt.ylabel('Count')
plt.show()
```



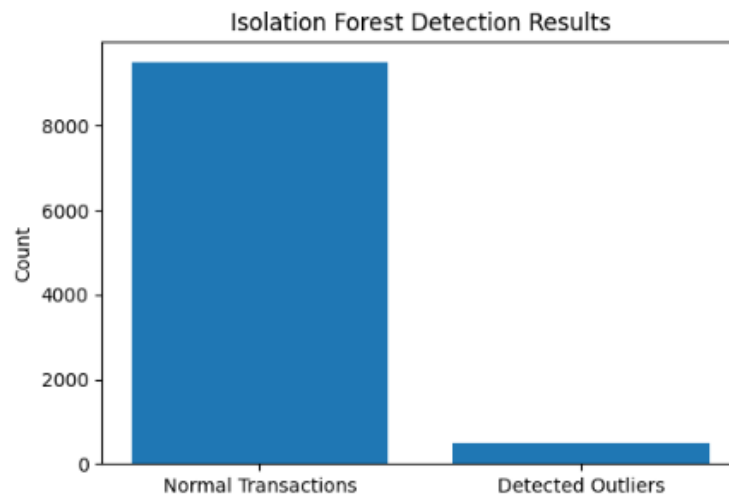
STEP 8: Isolation Forest Results

[13]
✓ 0s

```
isolation_outliers = (sample_df['IsolationForest'] == -1).sum()
normal_isolation = (sample_df['IsolationForest'] == 1).sum()

plt.figure(figsize=(6,4))
plt.bar(
    ['Normal Transactions', 'Detected Outliers'],
    [normal_isolation, isolation_outliers]
)

plt.title('Isolation Forest Detection Results')
plt.ylabel('Count')
plt.show()
```



STEP 9: Transaction Amount Scatter Plot

[15]
✓ 0s

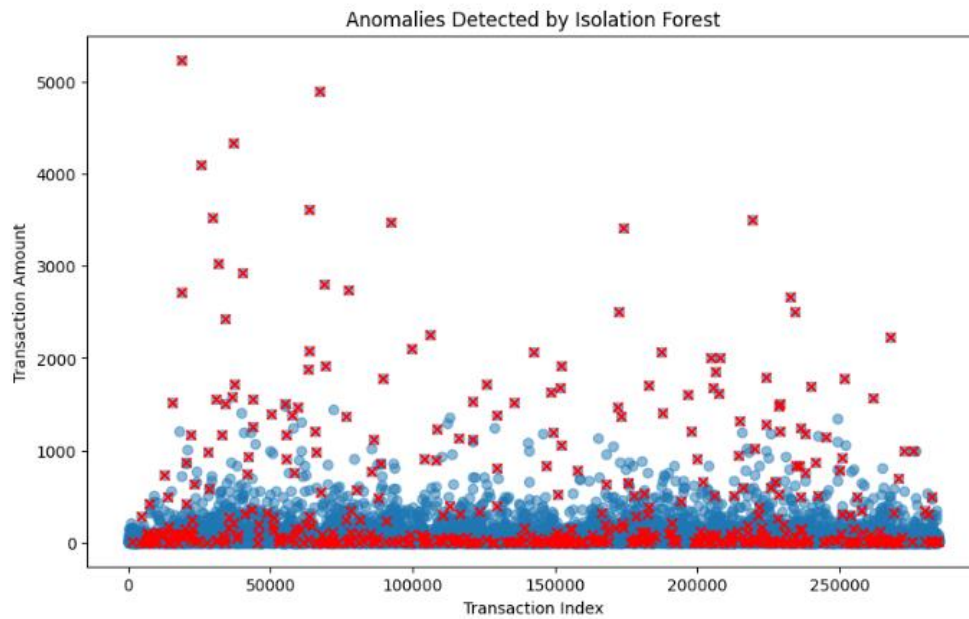
```
plt.figure(figsize=(10,6))

plt.scatter(
    sample_df.index,
    sample_df['Amount'],
    alpha=0.5
)

anomalies = sample_df[sample_df['IsolationForest'] == -1]

plt.scatter(
    anomalies.index,
    anomalies['Amount'],
    marker='x',
    color='red'
)

plt.title('Anomalies Detected by Isolation Forest')
plt.xlabel('Transaction Index')
plt.ylabel('Transaction Amount')
plt.show()
```



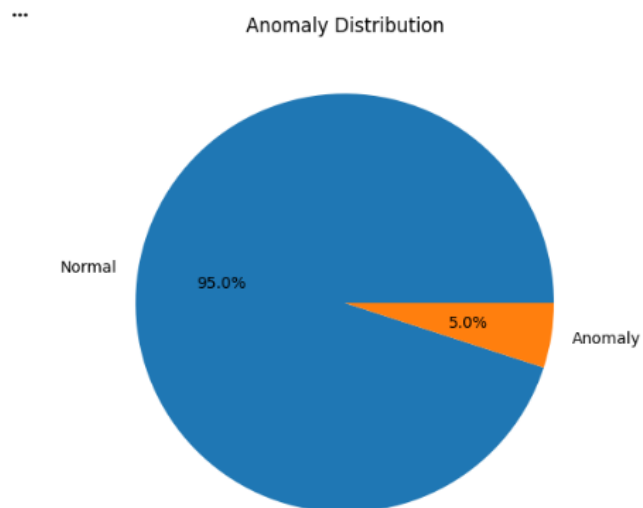
STEP 10: Pie Chart

```
[16]
✓ 0s

labels = ['Normal', 'Anomaly']
sizes = [normal_isolation, isolation_outliers]

plt.figure(figsize=(6,6))
plt.pie(
    sizes,
    labels=labels,
    autopct='%1.1f%%'
)

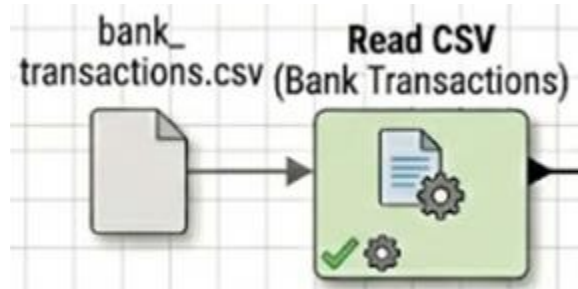
plt.title('Anomaly Distribution')
plt.show()
```



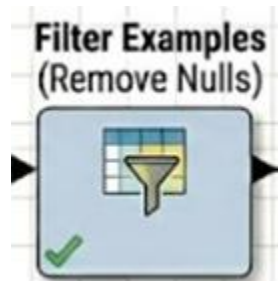
RAPIDMINER IMPLEMENTATION:

Step 1: Data Ingestion & Preprocessing

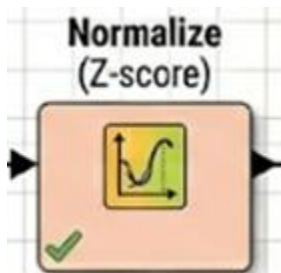
Read Data: Use the Read CSV operator to load the banking transaction data.



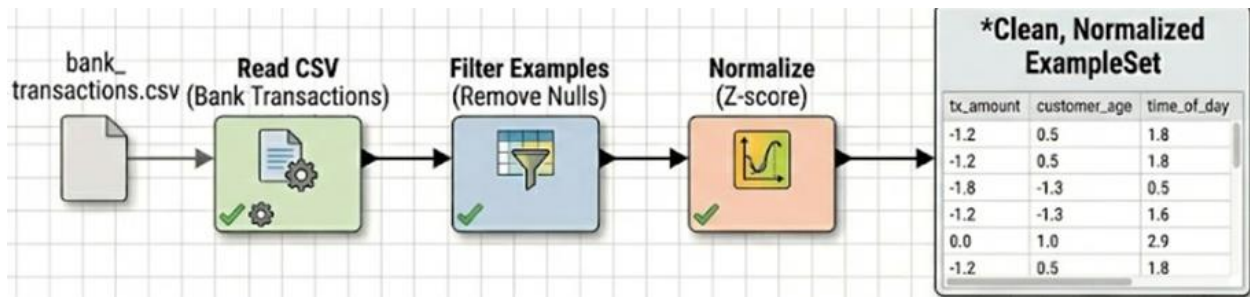
Cleaning: Use the Filter Examples operator to remove any null values.



Scaling: Apply the Normalize operator (Z-score) to ensure all features contribute equally to distance calculations, which is essential for outlier detection.

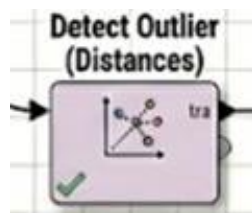


Connecting nodes:

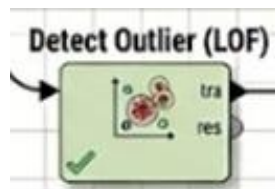


Step 2: Implementing Outlier Detection

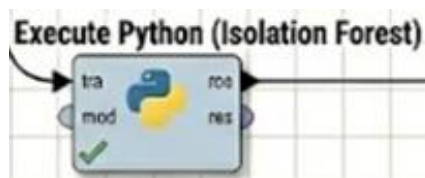
Distance-Based Detection: Use the Detect Outlier (Distances) operator.



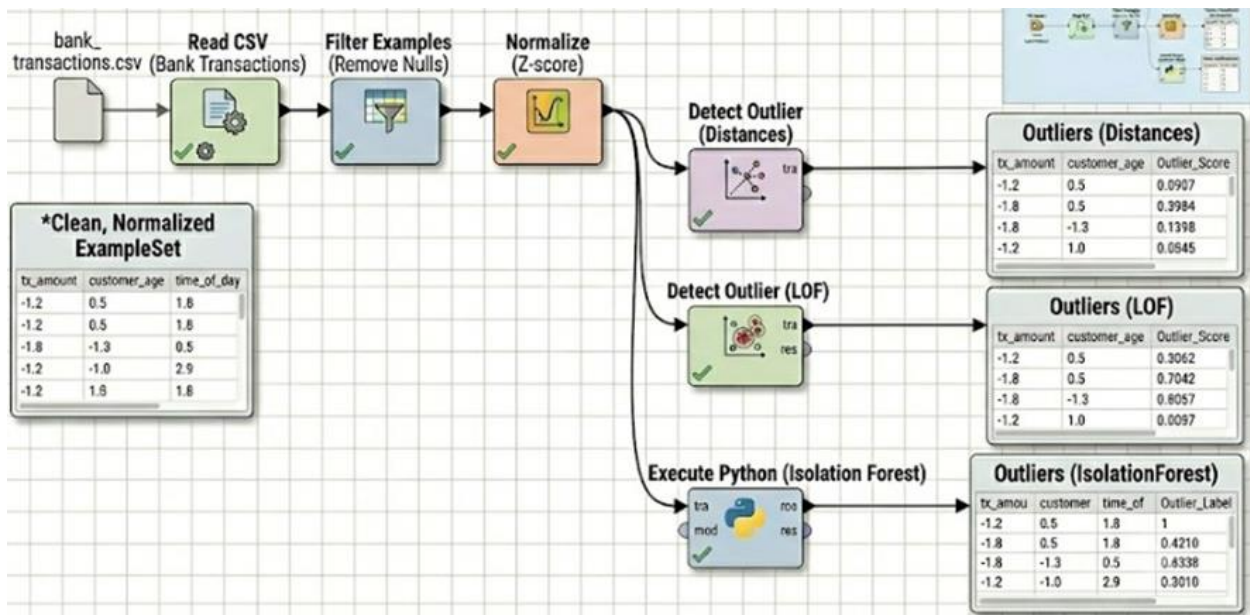
Density-Based Detection: Use the Detect Outlier (LOF) (Local Outlier Factor) operator.



Isolation Forest (via Python): Use the Execute Python operator to apply the Isolation Forest algorithm.



Combining all:



LAB-14: Web Scraping & Sentiment Analysis

Objective:

To collect product reviews from online sources, clean the text data, and perform sentiment analysis to determine whether reviews are positive, negative, or neutral.

Introduction:

Web Scraping is the process of extracting data from websites such as social media platforms, e-commerce sites, and review websites. The collected data can be analyzed to gain useful insights.

Sentiment Analysis is a text mining technique used to identify the sentiment or opinion expressed in text. It classifies reviews as **positive, negative, or neutral** based on the emotions conveyed by users.

Dataset Used:

Social Media / Review Dataset

The dataset contains customer reviews, comments, ratings, or feedback collected from online platforms. These reviews help understand customer opinions about products and services.

Software Used:

- CiteSpace / Gephi
- Python
- Pandas
- TextBlob / NLTK

Methodology:

1. Collect reviews using web scraping.
2. Clean and preprocess the text data.
3. Remove stopwords and unnecessary symbols.
4. Perform sentiment classification.
5. Calculate polarity scores.
6. Visualize sentiment distribution.

Performance Metrics:

Sentiment Percentage Distribution

Shows the percentage of positive, negative, and neutral reviews.

Polarity Score

Measures the sentiment strength of a review.

- Positive value → Positive sentiment
- Negative value → Negative sentiment
- Zero value → Neutral sentiment

Real-World Example:

An online shopping company analyzes customer reviews of a smartphone to understand customer satisfaction. Positive reviews indicate satisfaction, while negative reviews highlight issues that need improvement.

PYTHON IMPLEMENTATION:

Cell 1: Upload Dataset

```
[1]
✓ 28s

from google.colab import files

uploaded = files.upload()

Choose Files retail_sales_dataset.csv
retail_sales_dataset.csv(text/csv) - 51673 bytes, last modified: 4/30/2026 - 100% done
Saving retail_sales_dataset.csv to retail_sales_dataset.csv
```

Cell 2: Install Required Libraries

```
[2]
✓ 7s

!pip install mlxtend

Requirement already satisfied: mlxtend in /usr/local/lib/python3.12/dist-packages (0.23.4)
Requirement already satisfied: scipy>=1.2.1 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (1.16.3)
Requirement already satisfied: numpy>=1.16.2 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (2.0.2)
Requirement already satisfied: pandas>=0.24.2 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (2.2.2)
Requirement already satisfied: scikit-learn>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (1.6.1)
Requirement already satisfied: matplotlib>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (3.10.0)
Requirement already satisfied: joblib>=0.13.2 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (1.5.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->mlxtend) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->mlxtend) (4.55.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->mlxtend) (1.4.7)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->mlxtend) (2.9.0)
Requirement already satisfied: pyparsing>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->mlxtend) (3.2.0)
Requirement already satisfied: packaging>=23.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->mlxtend) (24.1)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->mlxtend) (10.4.0)
```

Cell 3: Load Dataset

```
[3]
✓ 1s
import pandas as pd

df = pd.read_csv("retail_sales_dataset.csv")

print(df.head())
print(df.columns)
print(df.shape)
```

Transaction ID Date Customer ID Gender Age Product Category \

0	1	2023-11-24	CUST001	Male	34	Beauty
1	2	2023-02-27	CUST002	Female	26	Clothing
2	3	2023-01-13	CUST003	Male	50	Electronics
3	4	2023-05-21	CUST004	Male	37	Clothing
4	5	2023-05-06	CUST005	Male	30	Beauty

Quantity Price per Unit Total Amount

0	3	50	150
1	2	500	1000
2	1	30	30
3	1	500	500
4	2	50	100

Index(['Transaction ID', 'Date', 'Customer ID', 'Gender', 'Age',
'Product Category', 'Quantity', 'Price per Unit', 'Total Amount'],
dtype='object')
(1000, 9)

Cell 4: Check Column Names

```
print(df.columns)
```

Index(['Transaction ID', 'Date', 'Customer ID', 'Gender', 'Age',
'Product Category', 'Quantity', 'Price per Unit', 'Total Amount'],
dtype='object')

Cell 5: Create Transaction Basket

```
[6]
✓ 0s
basket = (
    df.groupby(['Transaction ID', 'Product Category'])['Product Category']
      .count()
      .unstack()
      .fillna(0)
)

basket = basket.astype(bool)

print(basket.head())
```

Product Category Beauty Clothing Electronics

Transaction ID

1	True	False	False
2	False	True	False
3	False	False	True
4	False	True	False
5	True	False	False

Cell 6: FP-Growth

[7]
✓ Os

```
from mlxtend.frequent_patterns import fpgrowth
from mlxtend.frequent_patterns import association_rules
import time

start_fp = time.time()

fp_itemsets = fpgrowth(
    basket,
    min_support=0.01,
    use_colnames=True
)

fp_rules = association_rules(
    fp_itemsets,
    metric="confidence",
    min_threshold=0.5
)

end_fp = time.time()

fp_time = end_fp - start_fp

print("FP-Growth Frequent Itemsets")
print(fp_itemsets.head())

print("\nFP-Growth Rules")
print(fp_rules.head())
```

```
*** FP-Growth Frequent Itemsets
      support  itemsets
0    0.307    (Beauty)
1    0.351  (Clothing)
2    0.342 (Electronics)
```

FP-Growth Rules

Empty DataFrame

Columns: [antecedents, consequents, antecedent support, consequent support, support, confidence, lift, representativit
Index: []

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
return datetime.utcnow().replace(tzinfo=utc)

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
return datetime.utcnow().replace(tzinfo=utc)

Cell 7: Apriori

```
[8]
✓ Os
from mlxtend.frequent_patterns import apriori

start_ap = time.time()

ap_itemsets = apriori(
    basket,
    min_support=0.01,
    use_colnames=True
)

ap_rules = association_rules(
    ap_itemsets,
    metric="confidence",
    min_threshold=0.5
)

end_ap = time.time()

ap_time = end_ap - start_ap

print("Apriori Frequent Itemsets")
print(ap_itemsets.head())

print("\nApriori Rules")
print(ap_rules.head())
```

```
... Apriori Frequent Itemsets
      support  itemsets
0    0.307    (Beauty)
1    0.351  (Clothing)
2    0.342 (Electronics)
```

Apriori Rules

Empty DataFrame

Columns: [antecedents, consequents, antecedent support, consequent support, support, confidence, lift, representativeness]

Index: []

Cell 8: Compare Execution Time and Rule Count

```
[9]
✓ Os
results = pd.DataFrame({
    "Algorithm": ["FP-Growth", "Apriori"],
    "Execution Time": [fp_time, ap_time],
    "Rule Count": [len(fp_rules), len(ap_rules)]
})

print(results)
```

```
▼
   Algorithm  Execution Time  Rule Count
0  FP-Growth      0.012605           0
1    Apriori      0.007352           0
```

Cell 9: Execution Time Graph

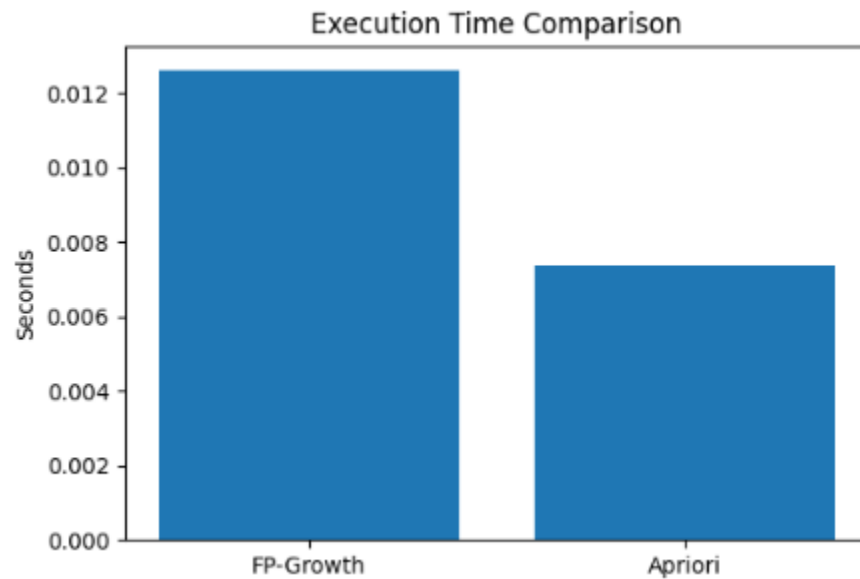
```
[10] ✓ 0s import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))

plt.bar(
    results["Algorithm"],
    results["Execution Time"]
)

plt.title("Execution Time Comparison")
plt.ylabel("Seconds")

plt.show()
```



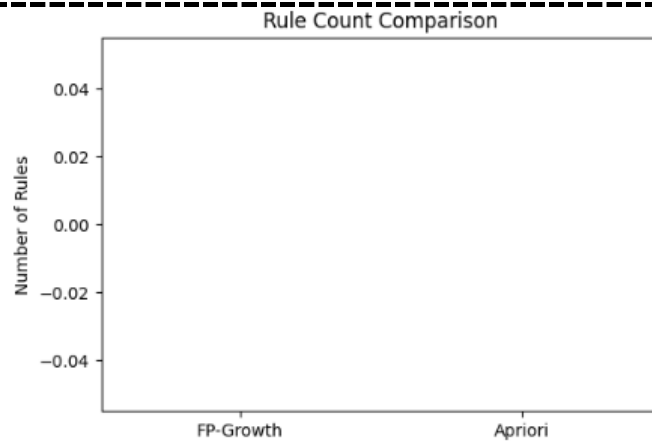
Cell 10: Rule Count Graph

```
[12] ✓ 0s plt.figure(figsize=(6,4))

plt.bar(
    results["Algorithm"],
    results["Rule Count"]
)

plt.title("Rule Count Comparison")
plt.ylabel("Number of Rules")

plt.show()
```

Cell 11: Top 10 FP-Growth Rules

```
[13]
✓ Os
top_rules = fp_rules.sort_values(
    by='confidence',
    ascending=False
).head(10)

print(top_rules[
    ['antecedents',
     'consequents',
     'support',
     'confidence',
     'lift']
])
```

Empty DataFrame
Columns: [antecedents, consequents, support, confidence, lift]
Index: []

Cell 12: FP-Tree Style Visualization

```
[14]
✓ Os
top_items = fp_itemsets.sort_values(
    by='support',
    ascending=False
).head(10)

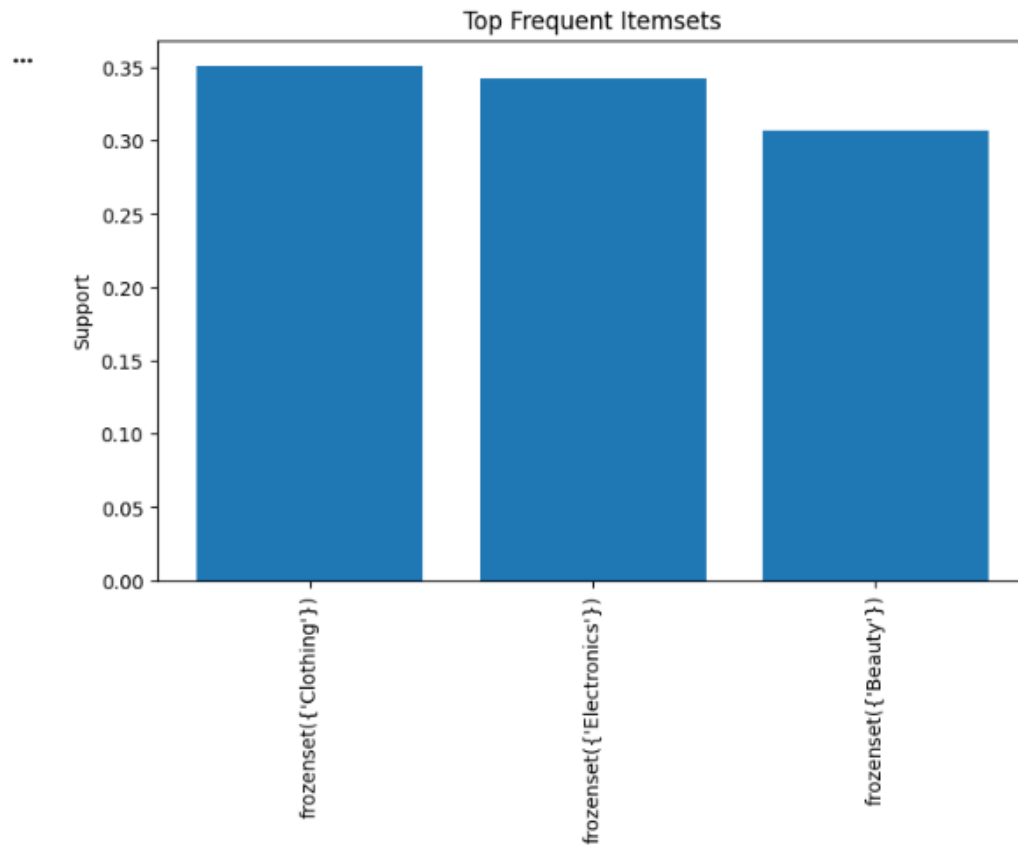
plt.figure(figsize=(8,5))

plt.bar(
    top_items['itemsets'].astype(str),
    top_items['support']
)

plt.xticks(rotation=90)

plt.title("Top Frequent Itemsets")
plt.ylabel("Support")

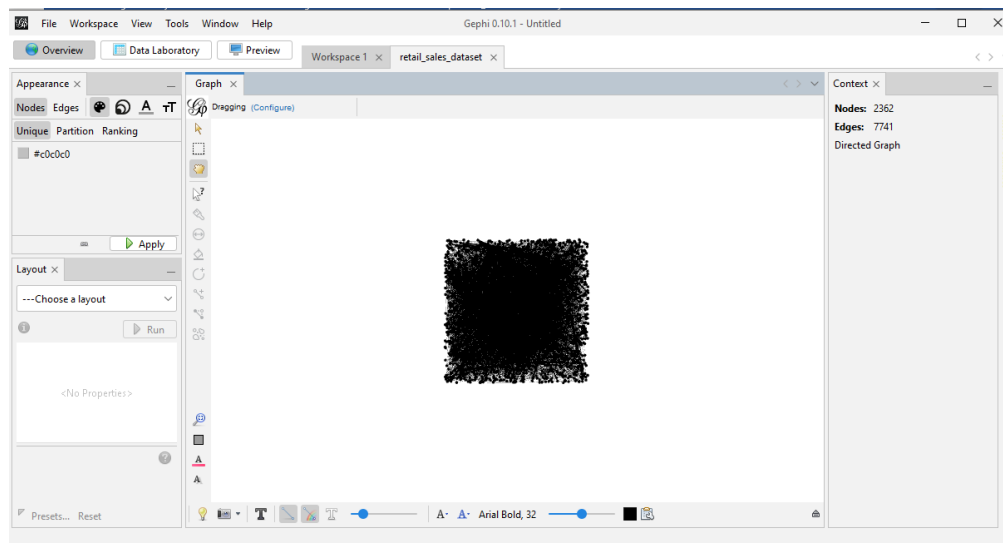
plt.show()
```



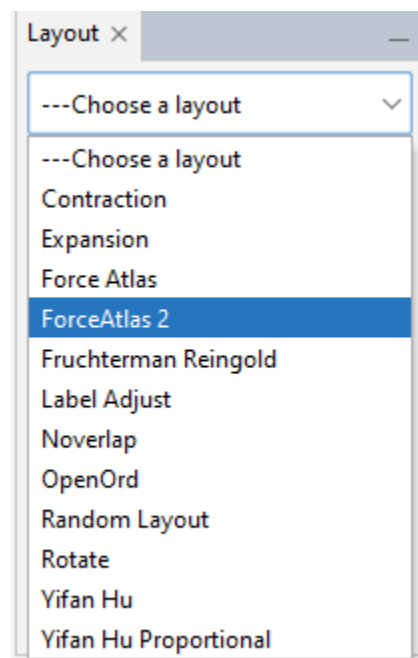
WORK ON GEPHI:



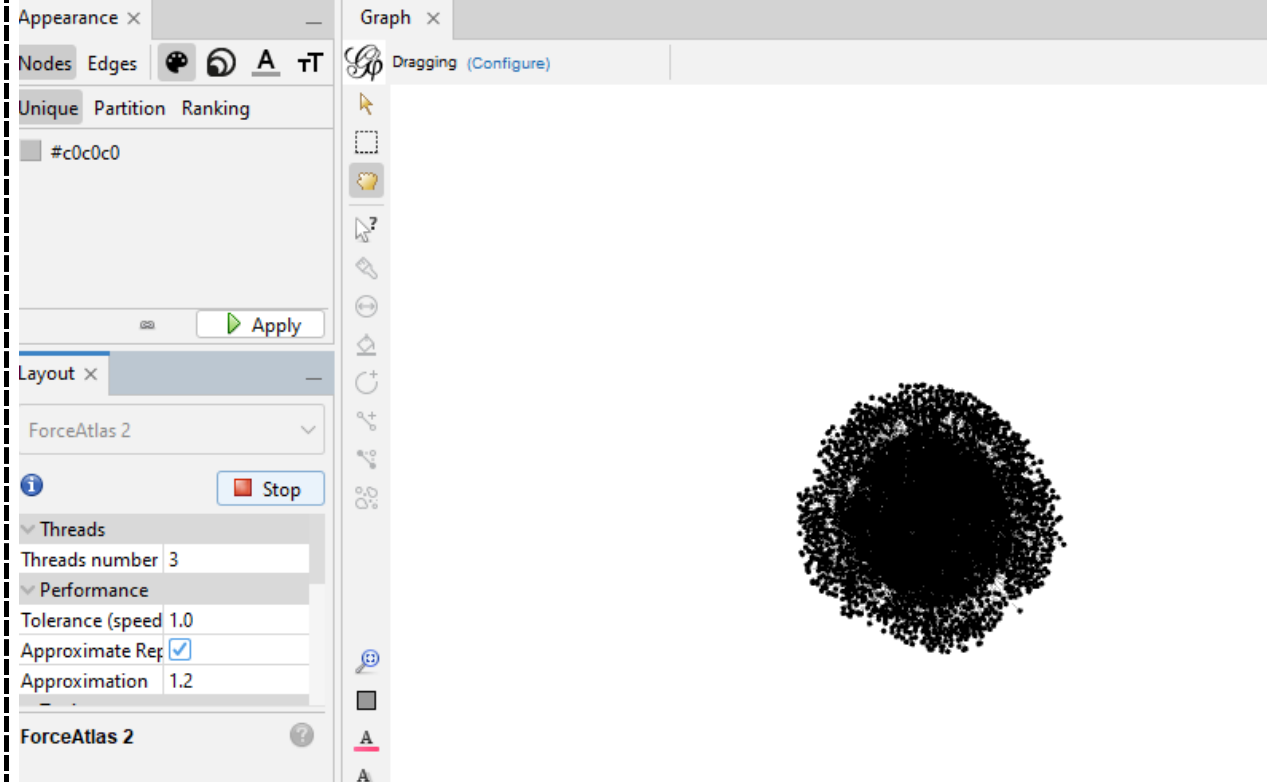
Upload dataset:



Choose layout:



After running:



From statistics

Modularity

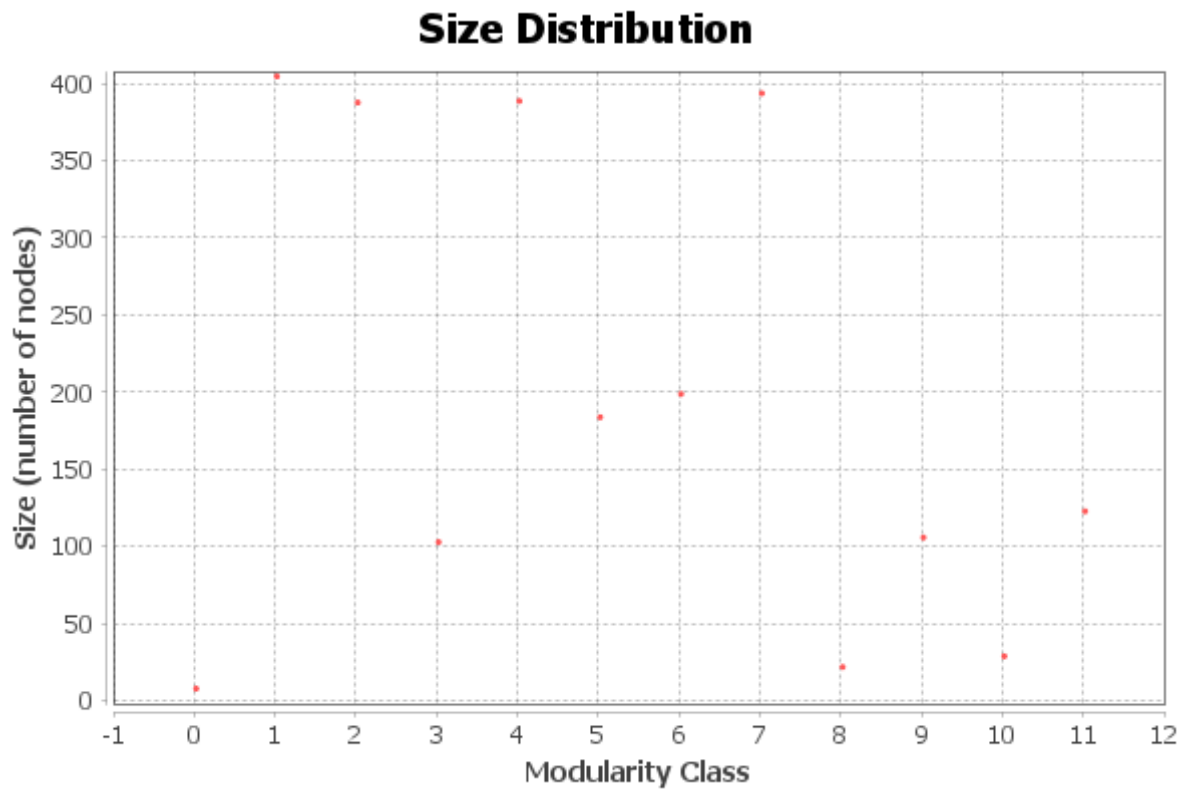
Modularity Report

Parameters:

Randomize: On
Use edge weights: On
Resolution: 1.0

Results:

Modularity: 0.350
Modularity with resolution: 0.350
Number of Communities: 12



Graph Density Report

Parameters:

Network Interpretation: directed

Results:

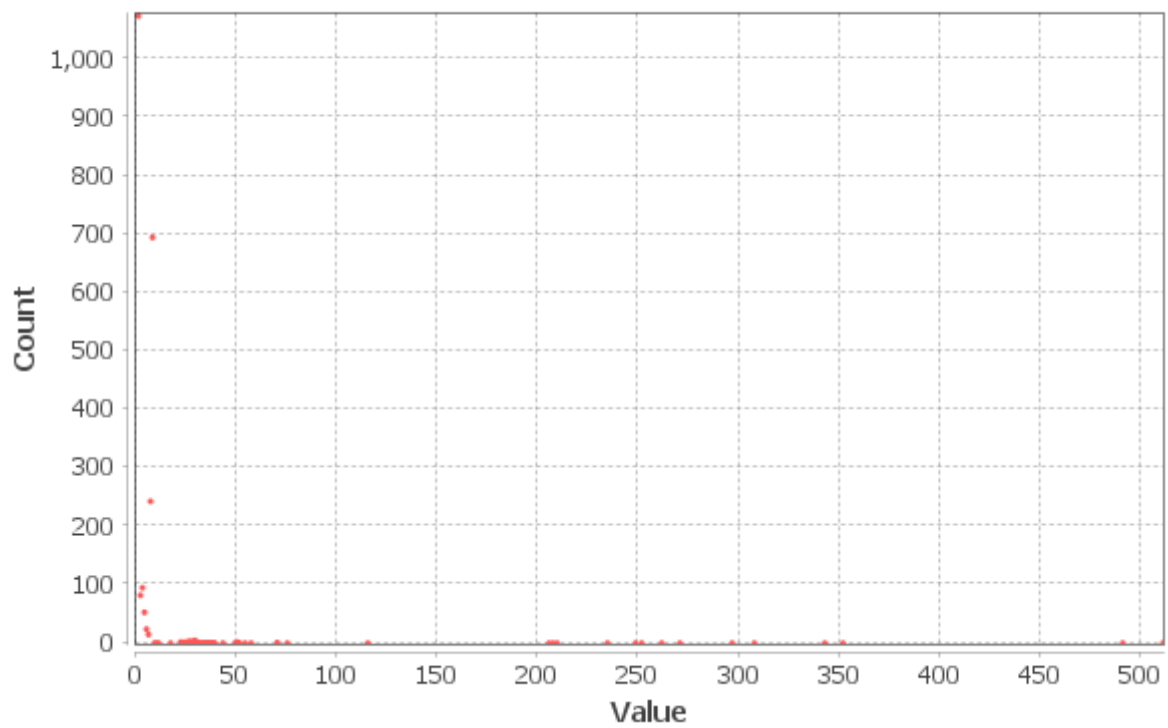
Density: 0.001

Degree Report

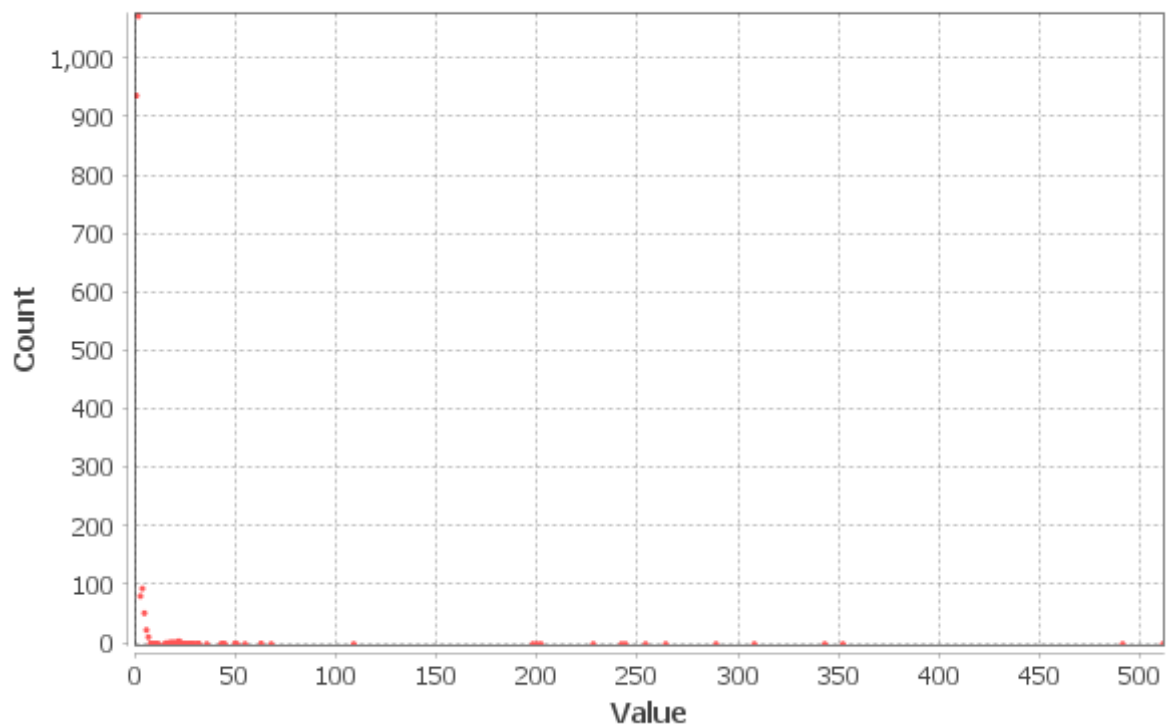
Results:

Average Degree: 3.277

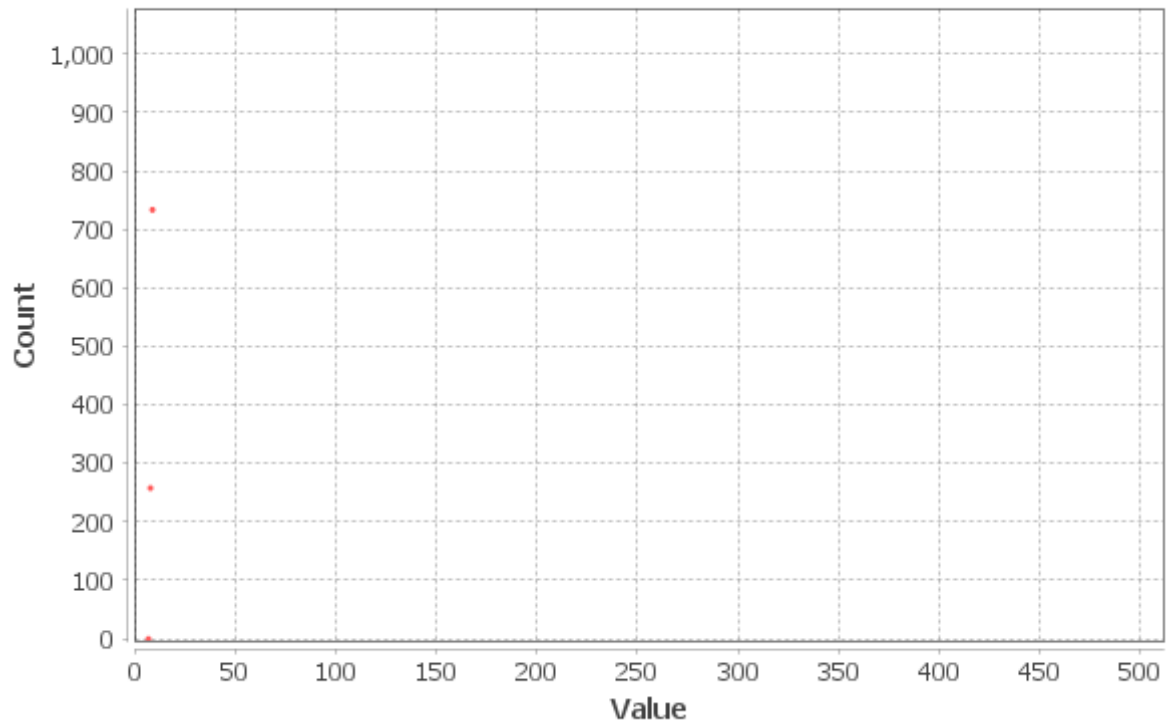
Degree Distribution



In-Degree Distribution



Out-Degree Distribution



Graph Distance Report

Parameters:

Network Interpretation: directed

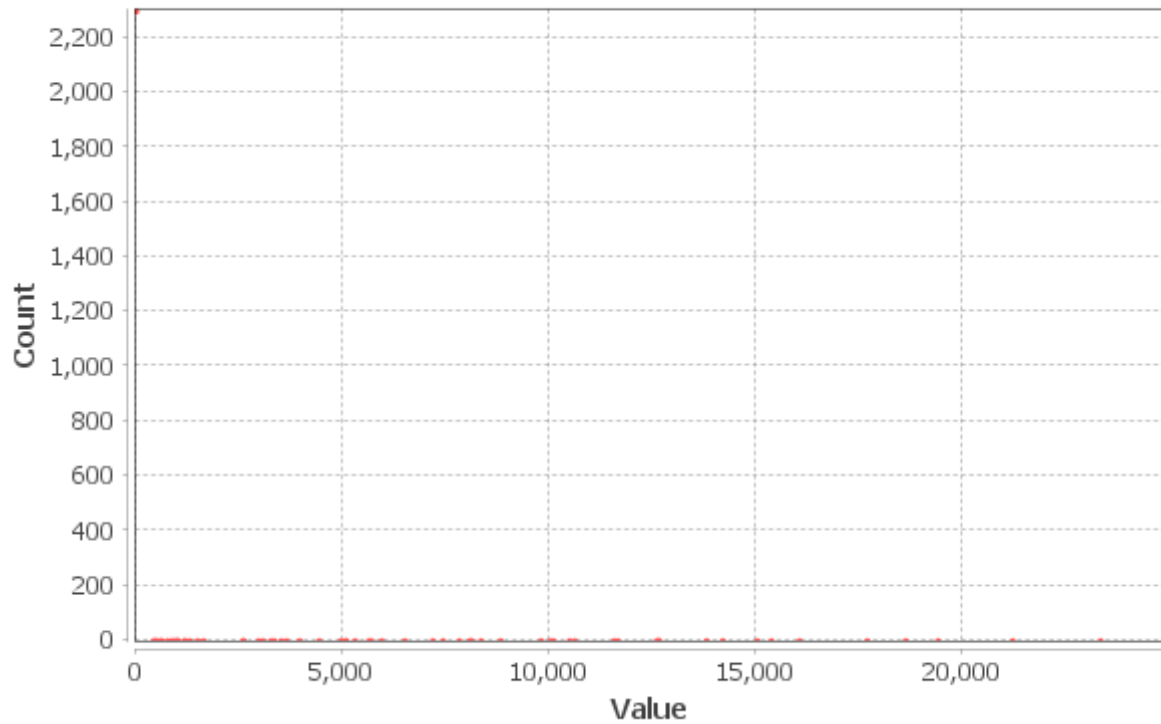
Results:

Diameter: 12

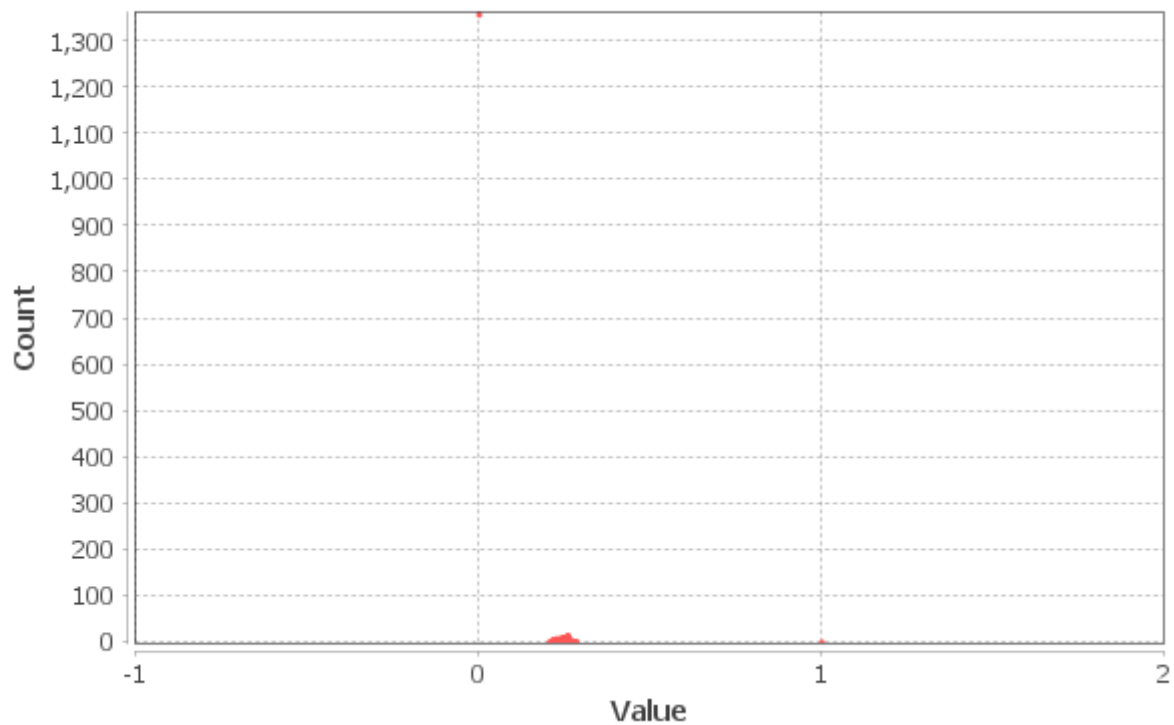
Radius: 0

Average Path length: 4.136677325899202

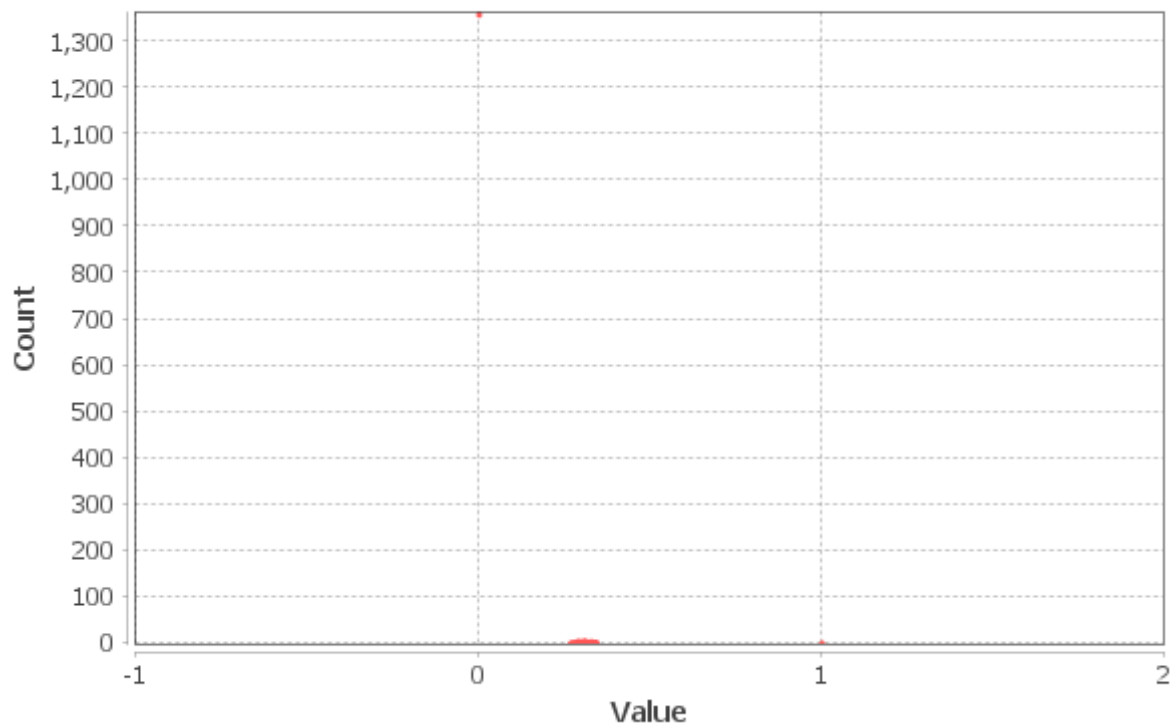
Betweenness Centrality Distribution



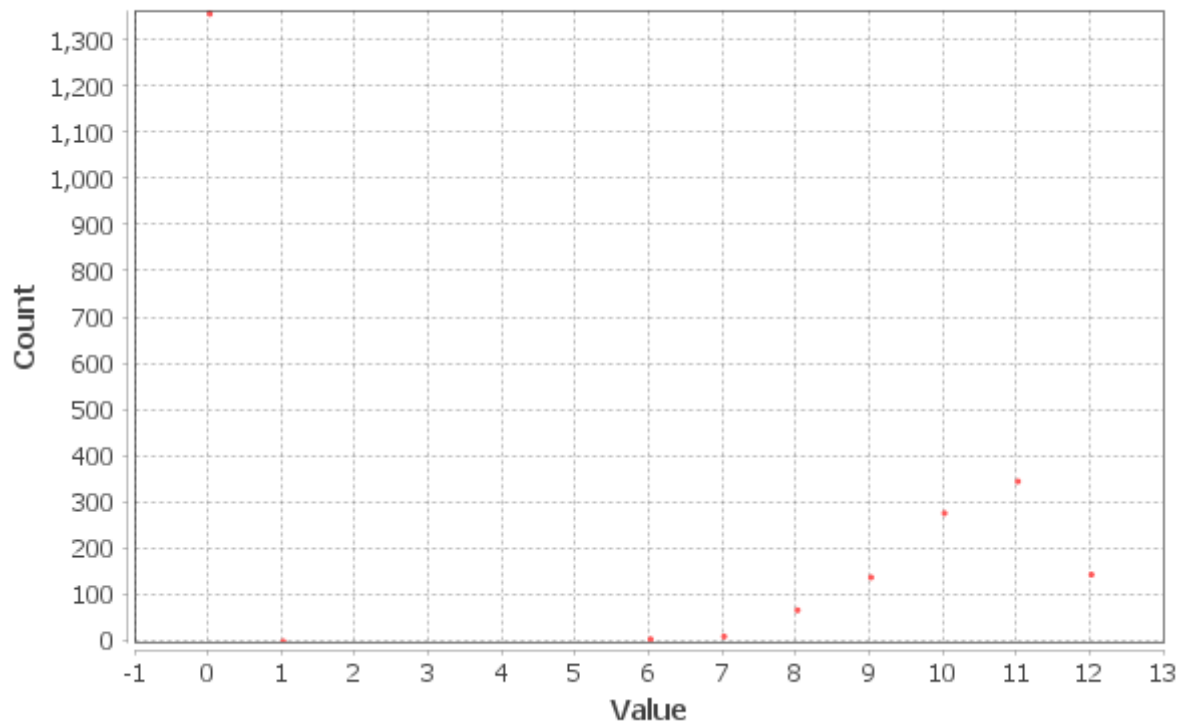
Closeness Centrality Distribution



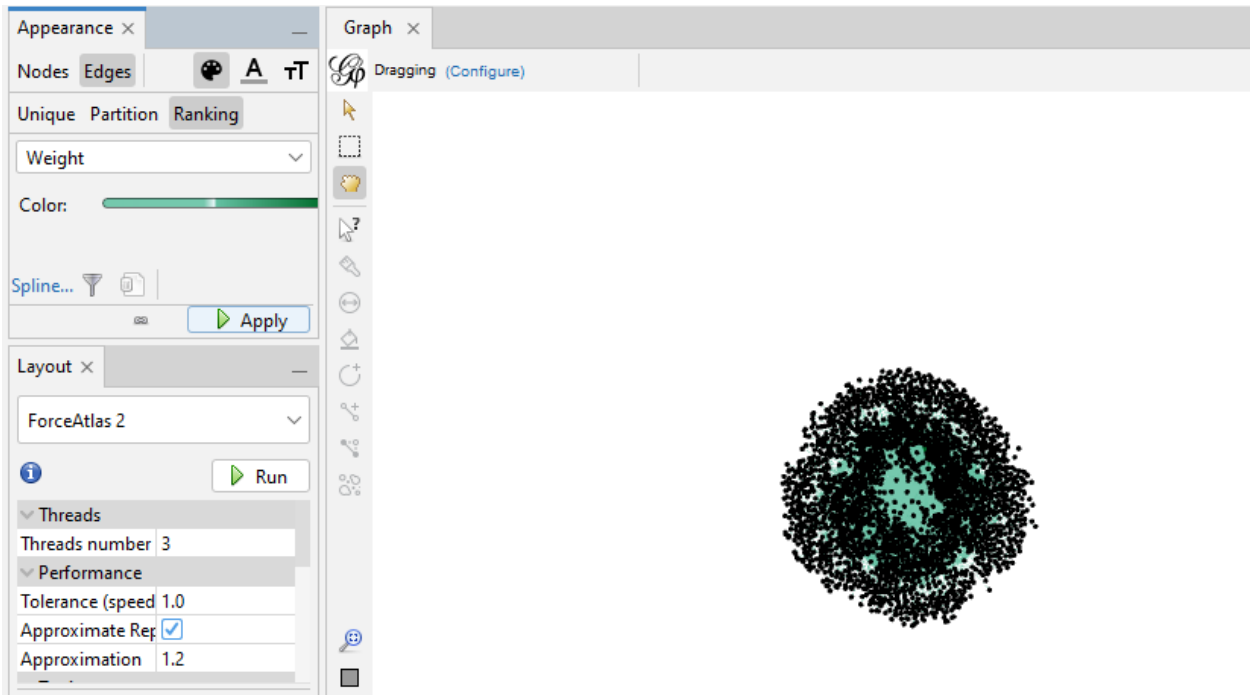
Harmonic Closeness Centrality Distribution



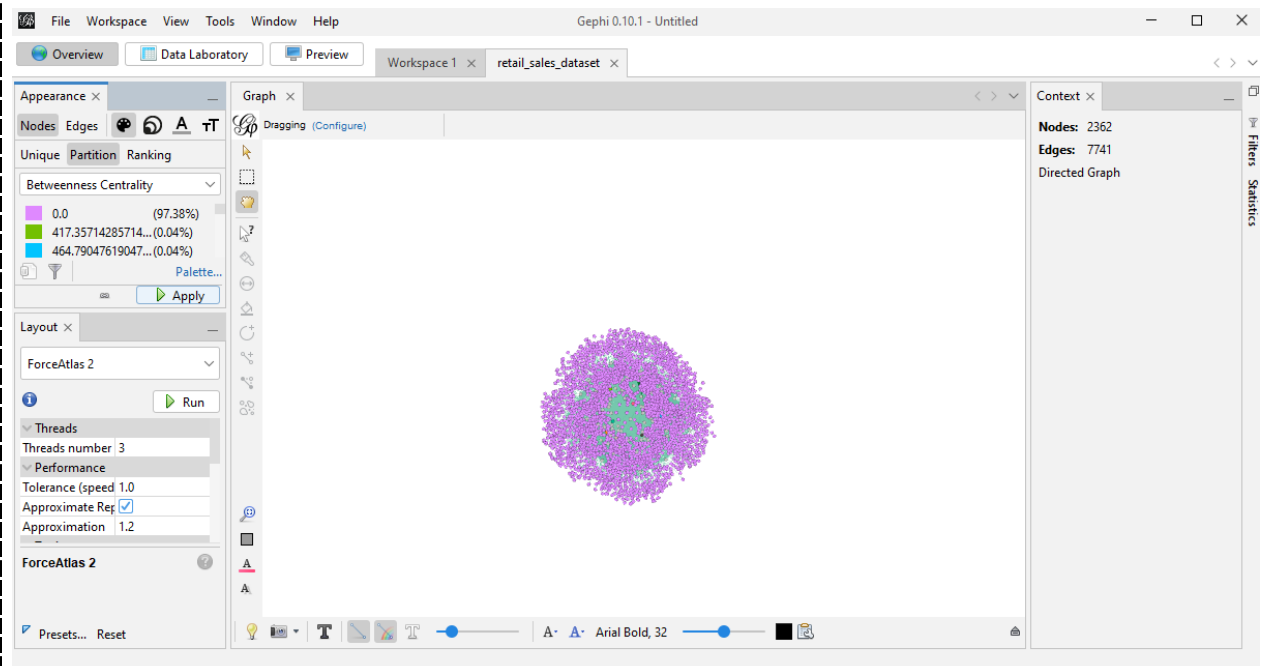
Eccentricity Distribution

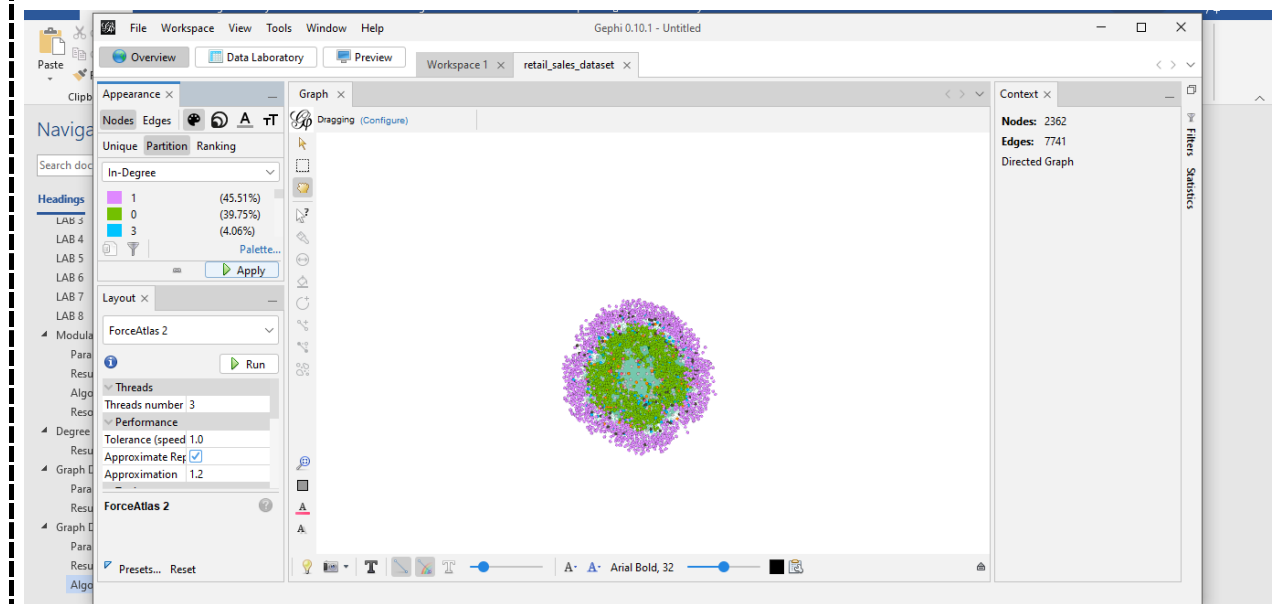
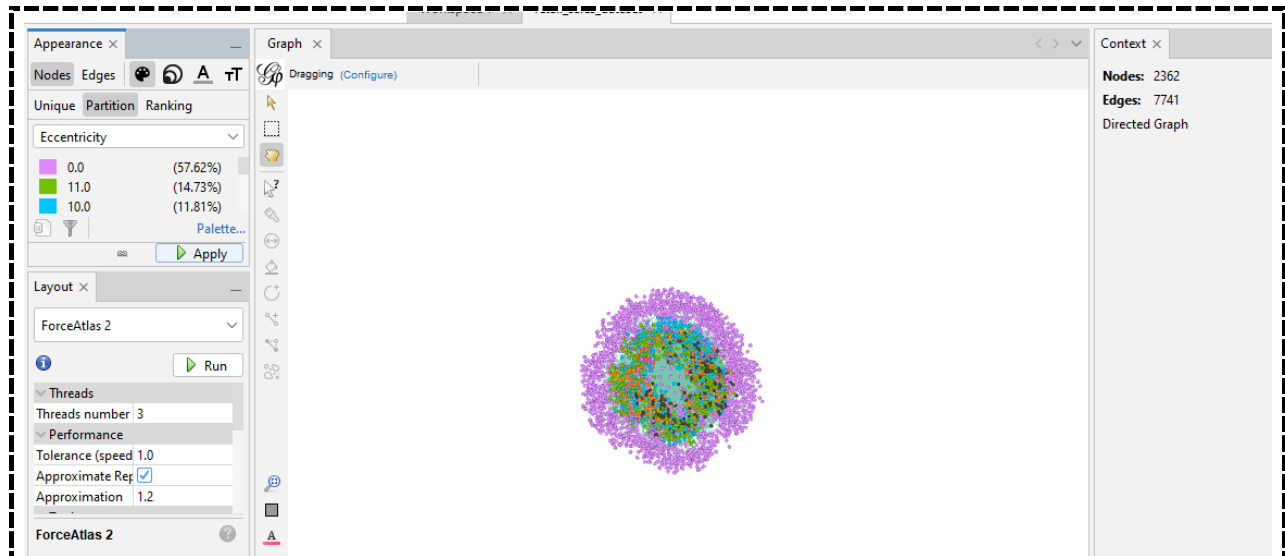


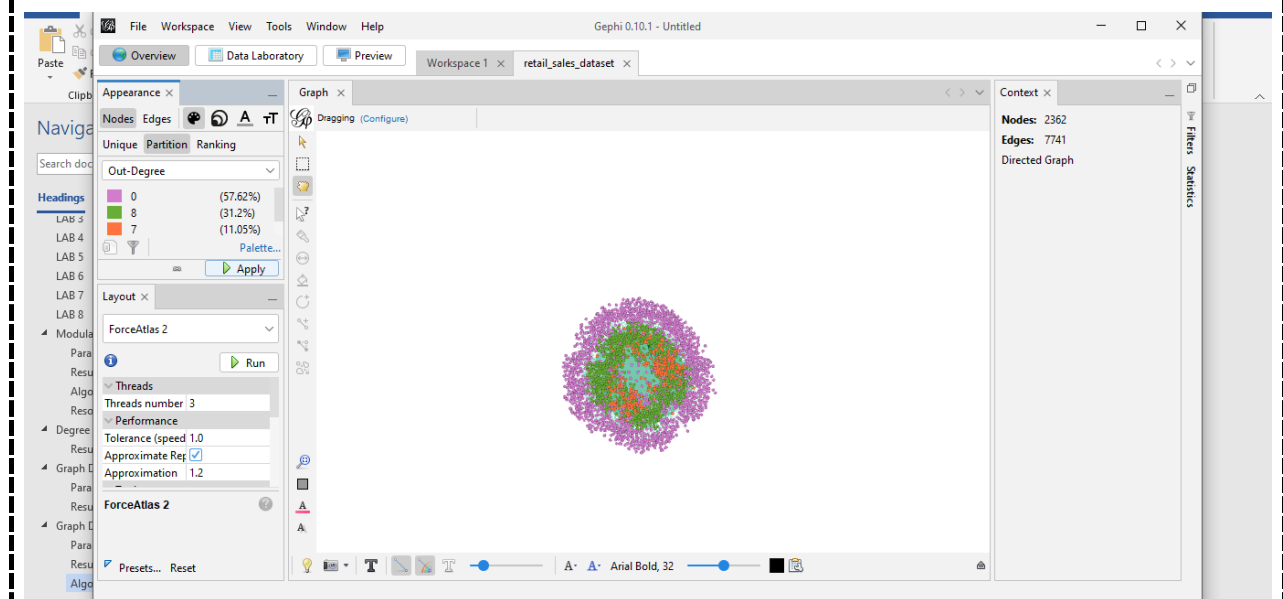
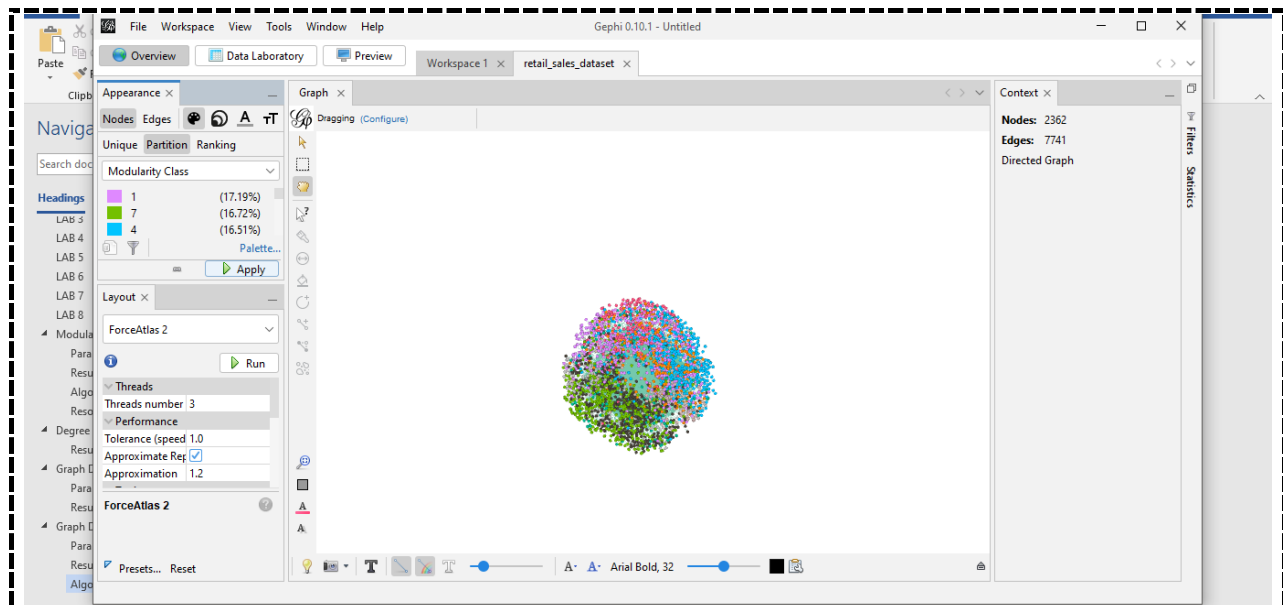
From edges:

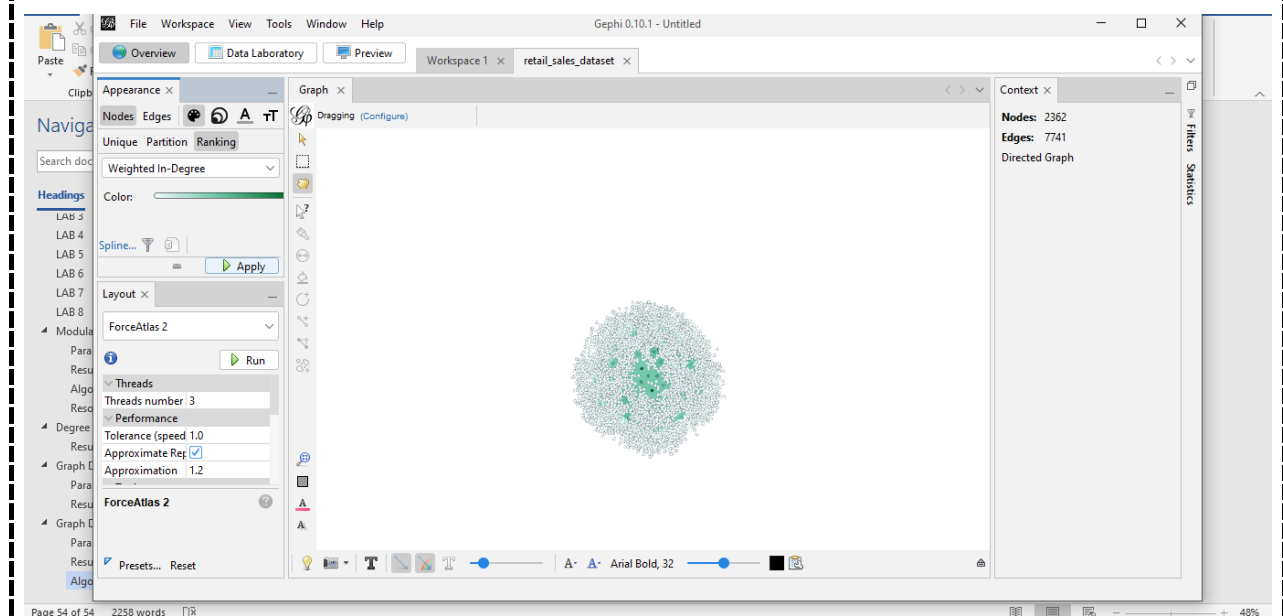
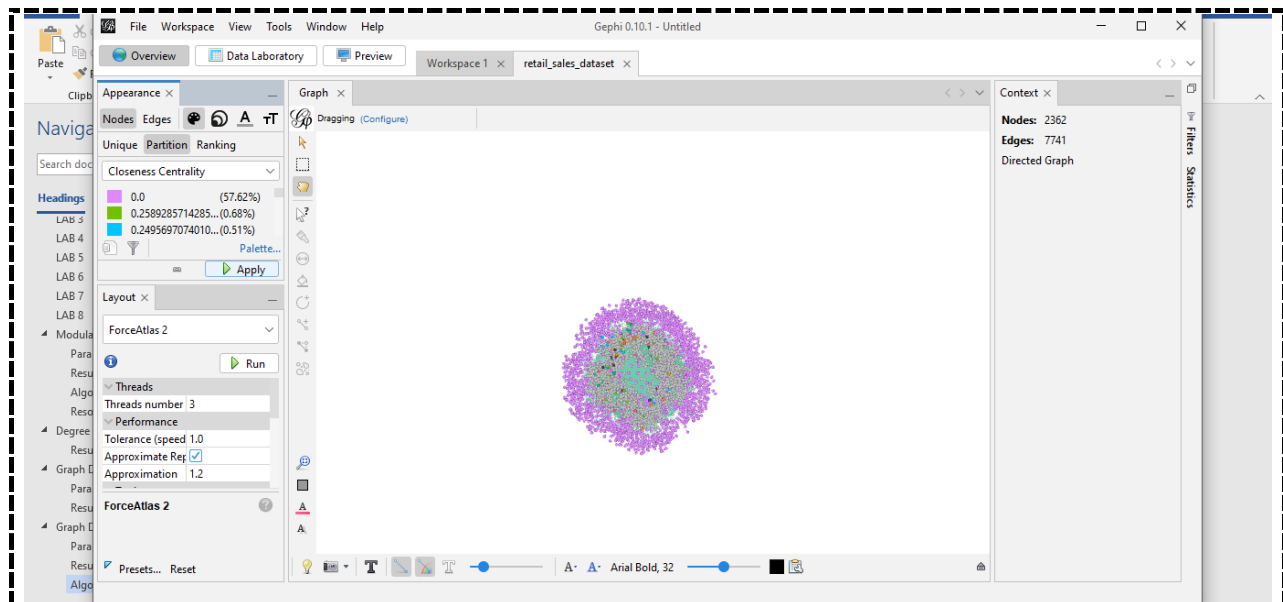


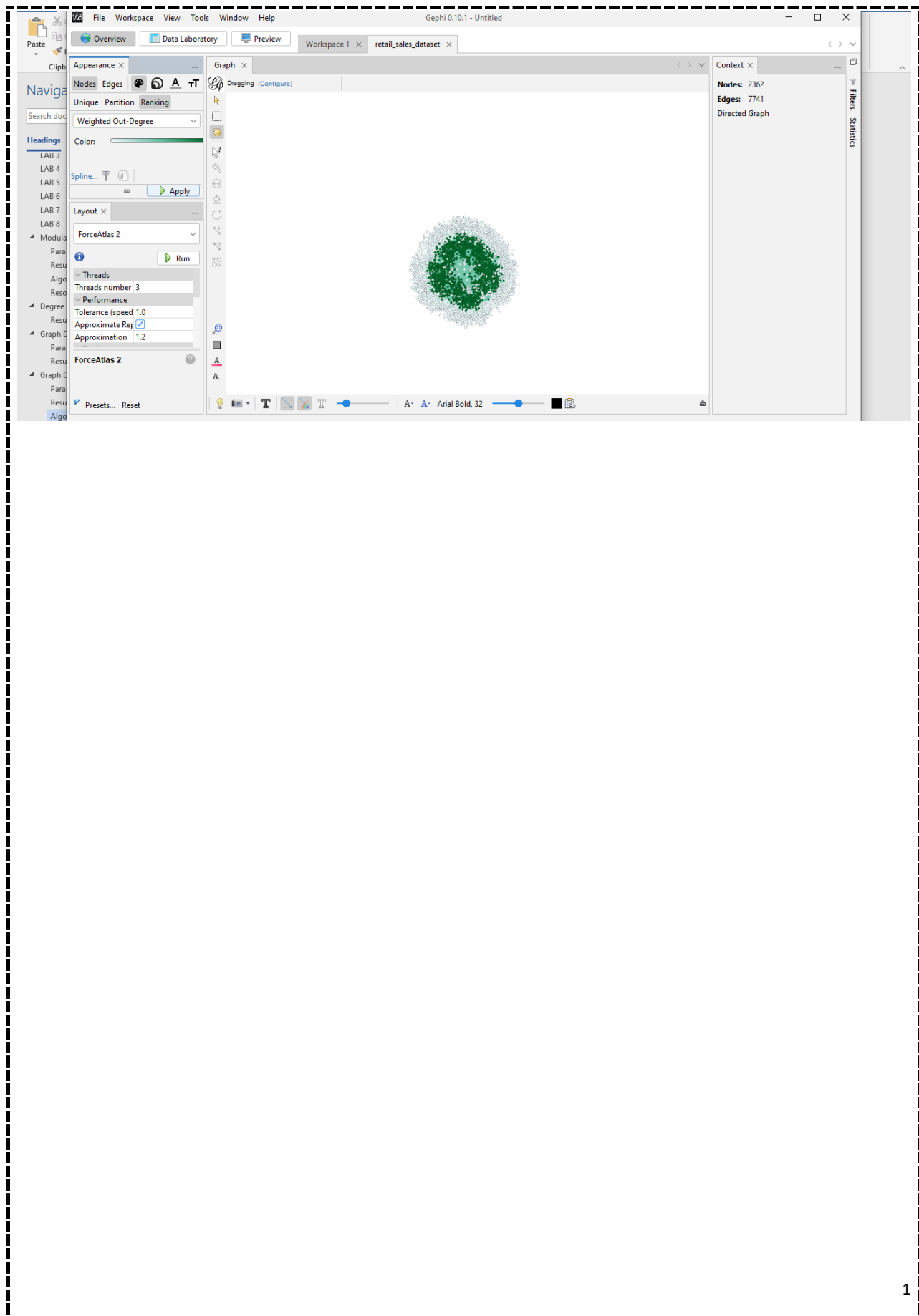
From nodes:











LAB-15: Final Capstone

Objective:

To develop a complete data mining solution for a real-world business problem by preprocessing data, applying multiple machine learning models, comparing their performance, and providing business recommendations.

Introduction:

A Capstone Project combines all the concepts learned in data mining and machine learning. It involves analyzing a real-world industry dataset, preparing the data, building predictive models, evaluating their performance, and selecting the best model for decision-making.

Dataset Used:

Any Industry Dataset (Retail, Banking, Healthcare, Education, Marketing, etc.)

The dataset contains business-related information used to solve a specific problem such as customer churn prediction, sales forecasting, fraud detection, or loan approval prediction.

Methodology:

1. Load and preprocess the dataset.
2. Handle missing values and outliers.
3. Perform feature selection and transformation.
4. Apply 8 machine learning models.
5. Compare model performance.
6. Select the best-performing model.
7. Interpret results and provide business recommendations.

Models Used:

- Decision Tree
- Random Forest
- Naïve Bayes
- KNN
- SVM
- Logistic Regression
- Neural Network
- Gradient Boosting (or any suitable model)

Performance Metrics:

- Accuracy
- Precision
- Recall
- F1-Score

Expected Output:

- Model Comparison Table
- Best Accuracy Score
- Best Performing Model
- Business Recommendation

PYTHON IMPLEMENTATION:

STEP 1: Upload Dataset

```
[1] | from google.colab import files
✓ 3m | uploaded = files.upload()

    | Choose Files Churn_Modelling.csv
    | Churn_Modelling.csv(text/csv) - 684858 bytes, last modified: 6/16/2026 - 100% done
    | Saving Churn_Modelling.csv to Churn_Modelling.csv
```

STEP 2: Import Libraries

```
[2] | import pandas as pd
✓ 2s | import numpy as np
    | import matplotlib.pyplot as plt

    | from sklearn.model_selection import train_test_split
    | from sklearn.preprocessing import LabelEncoder
    | from sklearn.metrics import accuracy_score
```


STEP 3: Load Dataset

```
[3] ✓ 0s df = pd.read_csv('Churn_Modelling.csv')

print(df.head())
print(df.shape)
```

	RowNumber	CustomerId	Surname	Creditscore	Geography	Gender	Age	\
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	

	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	\
0	2	0.00	1	1	1	
1	1	83807.86	1	0	1	
2	8	159660.80	3	1	0	
3	1	0.00	2	0	0	
4	2	125510.82	1	1	1	

	EstimatedSalary	Exited
0	101348.88	1
1	112542.58	0
2	113931.57	1
3	93826.63	0
4	79084.10	0

(10000, 14)

STEP 4: Data Preprocessing

```
[4] ✓ 0s # Remove unnecessary columns
df = df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1)

# Encode categorical variables
le = LabelEncoder()

df['Geography'] = le.fit_transform(df['Geography'])
df['Gender'] = le.fit_transform(df['Gender'])

# Features and Target
X = df.drop('Exited', axis=1)
y = df['Exited']

print(X.head())
```

...	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	\
0	619	0	0	42	2	0.00		1
1	608	2	0	41	1	83807.86		1
2	502	0	0	42	8	159660.80		3
3	699	0	0	39	1	0.00		2
4	850	2	0	43	2	125510.82		1

	HasCrCard	IsActiveMember	EstimatedSalary
0	1	1	101348.88
1	0	1	112542.58
2	1	0	113931.57
3	0	0	93826.63
4	1	1	79084.10

STEP 5: Train-Test Split

```
[5]
✓ 0s
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

STEP 6: Import Models

```
[6]
✓ 1s
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import GradientBoostingClassifier
```

STEP 7: Train 8 Models

```
[7] ✓ 13s
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC(),
    "Neural Network": MLPClassifier(max_iter=500),
    "Gradient Boosting": GradientBoostingClassifier()
}

results = []

for name, model in models.items():
    model.fit(X_train, y_train)

    predictions = model.predict(X_test)

    accuracy = accuracy_score(y_test, predictions)

    results.append([name, accuracy])

    print(name, "Accuracy =", round(accuracy*100,2), "%")

n_iter_i = _check_optimize_result(
Logistic Regression Accuracy = 81.6 %
Decision Tree Accuracy = 78.5 %
Random Forest Accuracy = 86.5 %
KNN Accuracy = 76.4 %
Naive Bayes Accuracy = 79.0 %
SVM Accuracy = 80.35 %
Neural Network Accuracy = 45.0 %
Gradient Boosting Accuracy = 86.55 %
```

STEP 8: Comparison Table

```
[8] ✓ 0s
results_df = pd.DataFrame(
    results,
    columns=["Model", "Accuracy"]
)

results_df = results_df.sort_values(
    by="Accuracy",
    ascending=False
)

print(results_df)
```

	Model	Accuracy
7	Gradient Boosting	0.8655
2	Random Forest	0.8650
0	Logistic Regression	0.8160
5	SVM	0.8035
4	Naive Bayes	0.7900
1	Decision Tree	0.7850
3	KNN	0.7640
6	Neural Network	0.4500

STEP 9: Best Model

```
[9] ✓ Os best_model = results_df.iloc[0]

print("Best Model:", best_model['Model'])
print("Best Accuracy:", round(best_model['Accuracy']*100,2), "%")

▼ Best Model: Gradient Boosting
  Best Accuracy: 86.55 %
```

STEP 10: Accuracy Comparison Graph

```
[10] ✓ Os plt.figure(figsize=(10,5))

plt.bar(
    results_df["Model"],
    results_df["Accuracy"]
)

plt.xticks(rotation=45)

plt.ylabel("Accuracy")

plt.title("Model Comparison")

plt.show()
```

...

